

Welcome Back

My Dear Part 2 Students

I hope you all are refreshed
and energized for another
good academic year.



Course Title: PERL AND R PROGRAMMING LANGUAGE FOR BIOINFORMATICS	Course Code: GNKPSBIMJ2504
Unit 1	Introduction to Perl, File handling and Object-oriented Perl
Unit 2	Regular Expression and Database connectivity and Bioperl
Unit 3	Fundamentals of R in Bioinformatics
Unit 4	Introduction to Bioconductor and ggplot2

Lets Understand What we know?

List the languages you are aware and learned

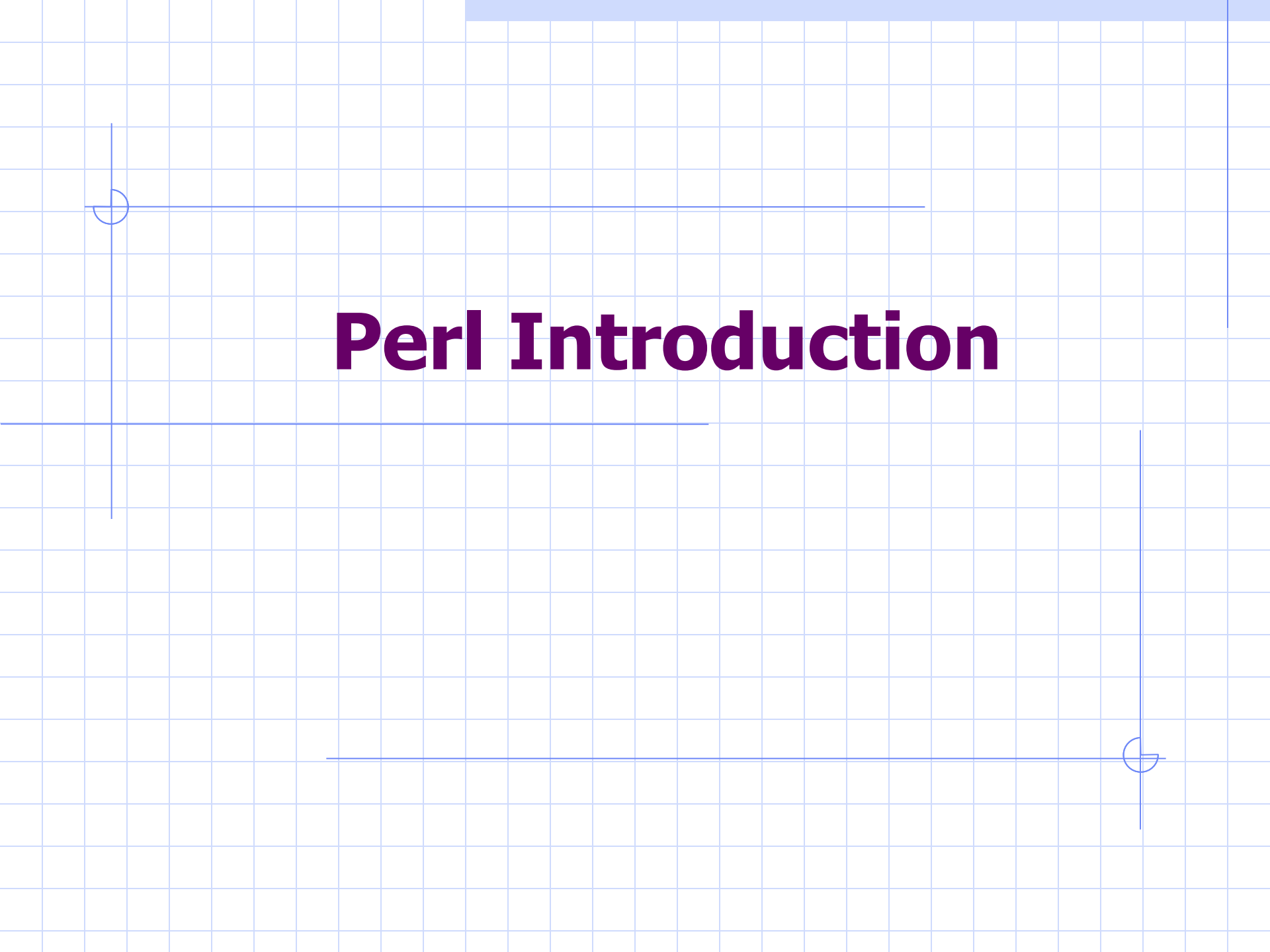
Specify the Purpose of Each Language?

Which two or three language's used widely?

Can we able use these languages to help analyzing genomics and proteomics data?

Easy coding languages for Bioinformaticians (if fundamentals are clear)

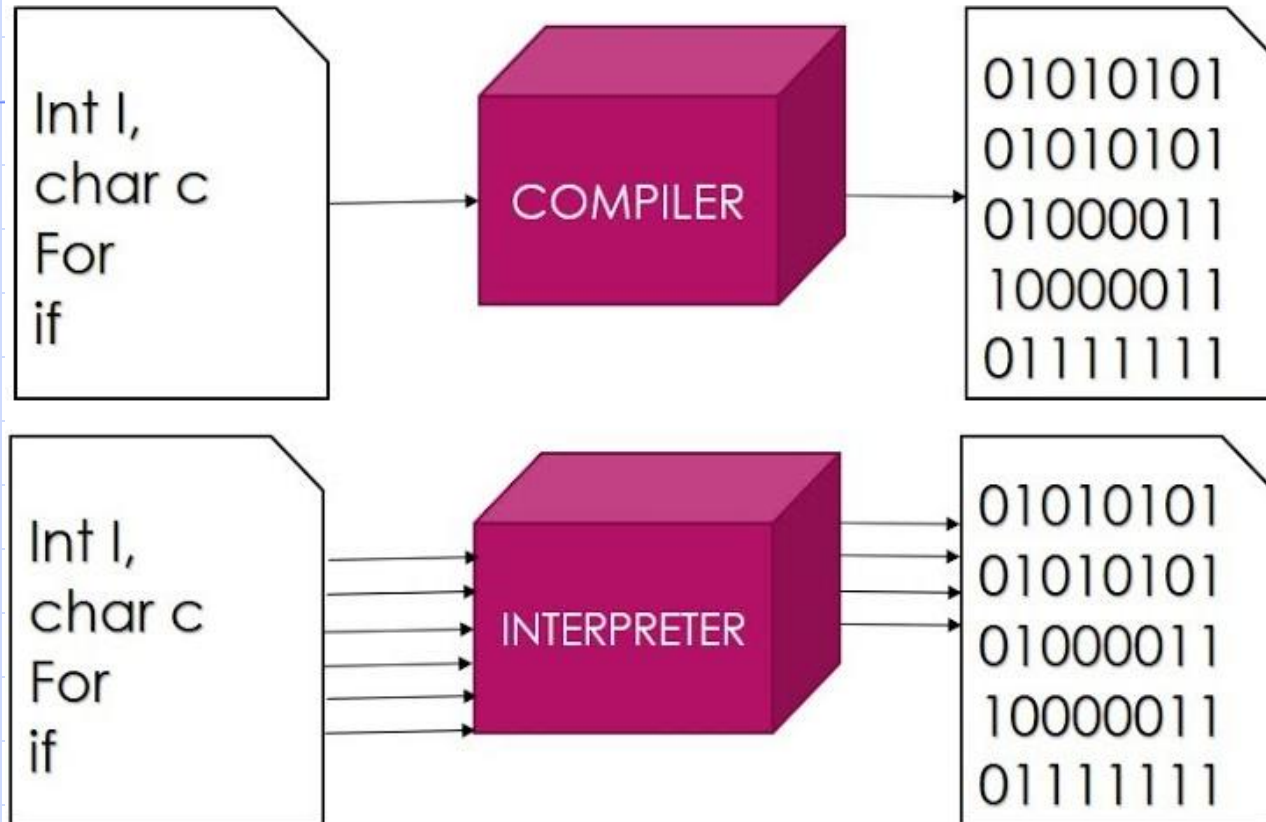
- ❖ Perl
- ❖ Python
- ❖ R
- ❖ Matlab



Perl Introduction

PARAMETERS	SCRIPTING LANGUAGES	PROGRAMMING LANGUAGES
Language Type	Interpreter-based languages	Compiler-based languages
Use	Aid the integration of the existing components of the application	Aid from-scratch development of applications
Compilation	Compilation is not needed	Compilation is needed
Complexity	Easy to write and use	More complex in terms of writing and usage
Conversion	Convert high-level instructions into machine language	Help in the conversion of the whole program into machine language at once
Running of Language	Program-dependent	Program-independent
Maintenance	Low maintenance	High maintenance
Cost	Relatively easy and cheap to maintain	More expensive to maintain

SCRIPTING V/S PROGRAMMING



Perl

- **"Practical Extraction and Reporting Language"**
- written by **Larry Wall** and first released in **1987**
- "Perl is a first language used easily for **manipulating text, files and processes**" and now used for a wide range of tasks including **system administration, web development, network programming, GUI development, and more.**
- Perl is a **High-level** Scripting language and case in-sensitive
- Perl is an **Open Source software**, licensed under its Artistic License, or the GNU General Public License (GPL).

What's Perl Good For?

- Quick scripts, complex scripts
- Parsing & restructuring data files
- High-level programming
 - Networking libraries
 - Graphics libraries
 - Database interface libraries

What's Perl Bad For?

- Hardware interfacing (device drivers...)
- Many modules like oops are not complete.

Use of Perl in Bioinformatics:-

It has Bioperl ,one of the first **biological module** repositories that increase the usability from.

For example, Change formats (Bio::SeqIO) to do phylogenetic analysis. There are some biological software that uses Perl such as GBrowse

Perl Basics

- Comment lines begin with: #
- Multi-Line Comment
=begin comment
=cut(end comment)
- File Naming Scheme
 - *filename.pl* (programs)
- Example program: print "Hello,
World!\n";

Perl Basics

■ Statements must end with semicolon

- `$a = 0;`
- Whitespaces in Perl(does not care)

`Print "Hello`

`world\n";`

This will produce the following result –

Hello

world

Single and Double Quotes in Perl

Ex:

```
print "Hello, world\n";  
print 'Hello, world\n';
```

This will produce the following result –

```
Hello, world  
Hello, world\n
```

Ex:

```
$a = 10;  
print "Value of a = $a\n";  
print 'Value of a = $a\n';
```

This will produce the following result –

```
Value of a = 10  
Value of a = $a\n
```

Data Types

■ Integer

- 25 750000 1_000_000_000
- 8#100 16#FFFF0000

■ Floating Point

- 1.25 50.0 6.02e23 -1.6E-8

■ String

- `"hi there"`
- `print "Text Utility, version $ver\n";`

Data Types

- Boolean
 - 0 0.0 represent False
 - 1 1.0 represent True

Variable Types

Scalar –

Scalars are simple variables. They are preceded by a dollar sign (\$). A scalar is either a number, a string, or a reference. A reference is actually an address of a variable.

Arrays –

Arrays are ordered lists of scalars that you access with a numeric index which starts with 0. They are preceded by an "at" sign (@).

Hashes –

Hashes are unordered sets of key/value pairs that you access using the keys as subscripts. They are preceded by a percent sign (%).

1. Scalar variables

■ Declaration of variable and Assignment of value

- `$num = 14;`
- `$fullname = "John H. Smith";`
- Variable Names are Case Sensitive

`print ("number is equal to: $num\n");`

2. Arrays

Declaration of variable and Assignment of value:

```
@ages = (25, 30, 40);  
@names = ("John Paul", "Lisa", "Kumar");  
print "\$ages[0] = $ages[0]\n";  
print "\$ages[1] = $ages[1]\n";  
print "\$ages[2] = $ages[2]\n";  
print "\$names[0] = $names[0]\n";  
print "\$names[1] = $names[1]\n";  
print "\$names[2] = $names[2]\n";
```

Output:-

\$ages[0] = 25

\$ages[1] = 30

\$ages[2] = 40

\$names[0] = John Paul

\$names[1] = Lisa

\$names[2] = Kumar

3. Hashes

Declaration of variable and Assignment of value:

```
%data = ('John Paul'=> 45, 'Lisa'=>30, 'Kumar'=>40);  
print %data;  
print "\$data{'John Paul'} = $data{'John Paul'}\n";  
print "\$data{'Lisa'} = $data{'Lisa'}\n";  
print "\$data{'Kumar'} = $data{'Kumar'}\n";
```

Output:

LisaKumarJohn Paul453040

`$data{'John Paul'} = 45`

`$data{'Lisa'} = 30`

`$data{'Kumar'} = 40`

VIVA QUESTION

- Full form of PERL
- Is Perl case sensitivity?[YES/NO]
- Perl is good for hardware.[True/False]
- List any one type of variable used in Perl?
- How to declare arrays variables in Perl?
- Disadvantages of Perl
- Uses of Perl in bioinformatics
- How do we save Perl script?



THANK YOU

Scalar Perl Operators

Perl Arithmetic Operators

Assume variable \$a holds 10 and variable \$b holds 20
then –

Operator Example

1. Addition –

Adds values on either side of the operator

$\$a + \b will give 30

2. Subtraction –

Subtracts right hand operand from left hand operand

$\$a - \b will give -10

3. Multiplication –

Multiplies values on either side of the operator

$\$a * \b will give 200

4. Division –

Divides left hand operand by right hand operand

$\$b / \a will give 2

5. Modulus –

Divides left hand operand by right hand operand and returns remainder $\$b \% \a will give 0

6. Exponent –

Performs exponential (power) calculation on operators

$\$a ** \b will give 10 to the power 20

Comparison Operators :-

1) Integer-Comparison Operators

Operator

< Less than

> Greater than

== Equal to

<= Less than or equal to

>= Greater than or equal to

!= Not equal to

<=> Comparisons operator[Return 1 if first no. is greater,-1 if first no. is smaller,Return 0 if both no. is same]

2) String-Comparison Operators

Operator

lt **Less than**

gt **Greater than**

eq **Equal to**

le **Less than or equal to**

ge **Greater than or equal to**

ne **Not equal to**

cmp **Comparison operator**

Perl Logical Operators

There are following logical operators supported by Perl language. Assume variable \$a holds true and variable \$b holds false then –

➤ and(&&)

Called Logical AND operator. If both the operands are true then condition becomes true.

Ex: (\$a and \$b) is false.

➤ or(||)

Called Logical OR Operator. If any of the two operands are non zero then condition becomes true.

Ex: (\$a or \$b) is true.

➤ not

Called Logical NOT Operator. Use to reverse the logical state of its operand. If a condition is true then Logical NOT operator will make false.

Ex: not(\$a and \$b) is true.

Miscellaneous Operators

There are following miscellaneous operators supported by Perl language. Assume variable a holds 10 and variable b holds 20 then –

•

Binary operator dot (.) concatenates two strings.

If \$a="abc", \$b="def" then \$a.\$b will give "abcdef"

x

The repetition operator x returns a string consisting of the left operand repeated the number of times specified by the right operand. ('-' x 3) will give ---.

..

The range operator .. returns a list of values counting (up by ones) from the left value to the right value (2..5) will give (2, 3, 4, 5)

++

Auto Increment operator increases integer value by one \$a++ will give 11

--

Auto Decrement operator decreases integer value by one \$a-- will give 9

->

The arrow operator is mostly used in dereferencing a method or variable from an object or a class name \$obj->\$a is an example to access variable \$a from object \$obj.

The ? : Operator

Let's check the **conditional operator ? :** which can be used to replace **if...else** statements. It has the following general form –

Exp1 ? Exp2 : Exp3;

Below is a simple example making use of this operator –

```
$name = "Ali";
```

```
$age = 10;
```

```
$status = ($age > 60 )? "A senior citizen" : "Not a senior citizen";
```

```
print "$name is - $status\n";
```

This will produce the following result –

Ali is - Not a senior citizen

VIVA QUESTIONS:

Write a perl script to calculate average marks of a students with respect to 5 subject.

Which function we use to calculate 3 raised to 5.

Is any difference between `<=>` and `cmp()` operators?

Which operators used to concatenate two strings?

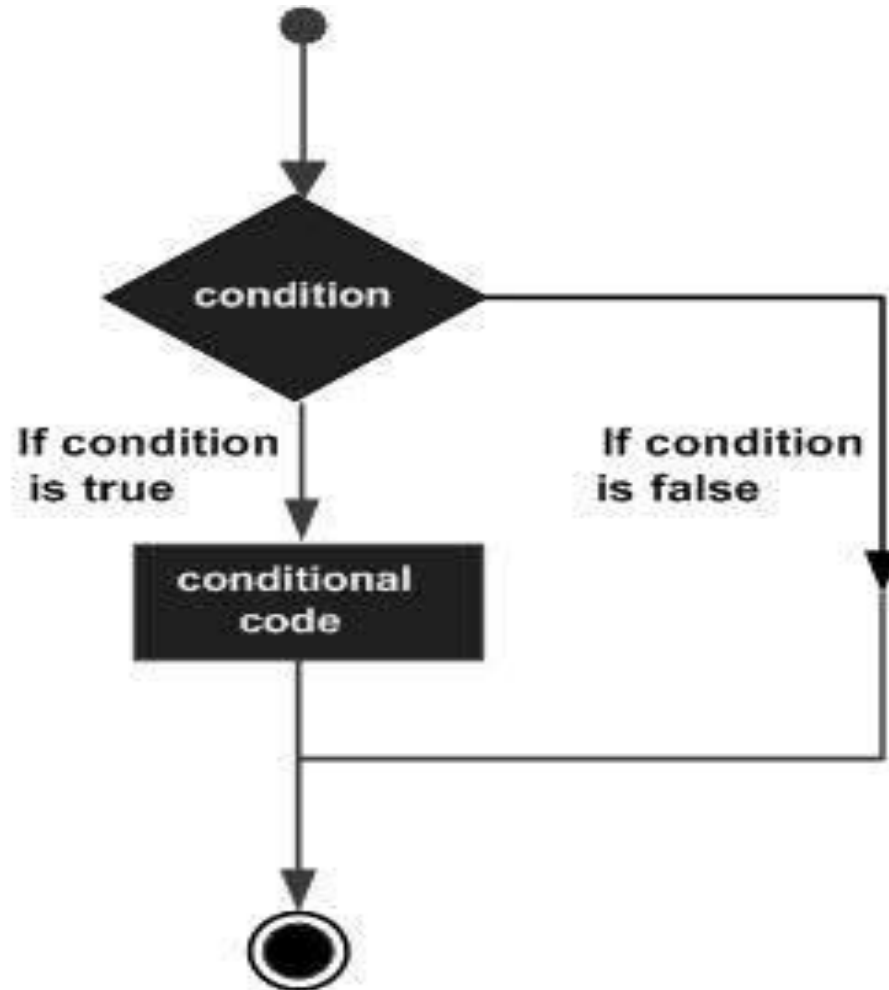
Give syntax for repetition operator.

What is the output for following code?

```
$a=(2..5);  
print $a;
```

Perl Conditional Statements -

Perl conditional statements helps in the **decision making**, which require that the programmer specifies **one or more conditions to be evaluated or tested by the program**, along with a statement or statements to be executed if the condition is determined to be **true**, and optionally, other statements to be executed if the condition is determined to be **false**.



Types of conditional statements.

1) if statement

An if statement consists of a boolean expression followed by one or more statements.

Syntax

The syntax of an if statement in Perl programming language is –

```
if(boolean_expression){
```

```
    # statement(s) will execute if the given condition is true
```

```
}
```

2) if...else statement

An if statement can be followed by an optional else statement.

Syntax

The syntax of an **if...else** statement in Perl programming language is –

```
if(boolean_expression){
```

```
    # statement(s) will execute if the given condition is true
```

```
}else{
```

```
    # statement(s) will execute if the given condition is false
```

```
}
```

if...elif...else statement

An if statement can be followed by an optional elif statement and then by an optional else statement.

Syntax

The syntax of an **if...elsif...else** statement in Perl programming language is

–

```
if(boolean_expression 1){
```

```
    # Executes when the boolean expression 1 is true
```

```
}
```

```
elsif( boolean_expression 2){
```

```
    # Executes when the boolean expression 2 is true
```

```
}
```

```
elsif( boolean_expression 3){
```

```
    # Executes when the boolean expression 3 is true
```

```
}else{
```

```
    # Executes when the none of the above condition is true
```

```
}
```

unless statement

An unless statement consists of a boolean expression followed by one or more statements.

Syntax

The syntax of an unless statement in Perl programming language is –

```
unless(boolean_expression){
```

```
    # statement(s) will execute if the given condition is  
false
```

```
}
```

unless...else statement

An unless statement can be followed by an optional else statement.

Syntax

The syntax of an **unless...else** statement in Perl programming language is –

```
unless(boolean_expression){
```

```
    # statement(s) will execute if the given condition is false
```

```
}else{
```

```
    # statement(s) will execute if the given condition is true
```

```
}
```


unless...elsif..else statement

An unless statement can be followed by an optional elsif statement and then by an optional else statement.

Syntax

The syntax of an **unless...elsif...else** statement in Perl programming language is –

```
unless(boolean_expression 1){  
  # Executes when the boolean expression 1 is false  
}  
elsif( boolean_expression 2){  
  # Executes when the boolean expression 2 is true  
}  
elsif( boolean_expression 3){  
  # Executes when the boolean expression 3 is true  
}  
else{  
  # Executes when the none of the above condition is met  
}
```

switch statement

With the latest versions of Perl, you can make use of the switch statement. which allows a simple way of comparing a variable value against various conditions.

Syntax

The synopsis for a **switch** statement in Perl programming language is as follows –

use Switch;

```
switch(argument){
    case 1 { print "number 1" }
    case "a" { print "string a" }
    case [1..10,42] { print "number in list" }
    case (\@array) { print "number in list" }
    case /\w+/ { print "pattern" }
    case qr/\w+/ { print "pattern" }
    case (\%hash) { print "entry in hash" }
    case (\&sub) { print "arg to subroutine" }
    else { print "previous case not true" }
}
```

Viva Questions:

What is the difference between if and switch statement

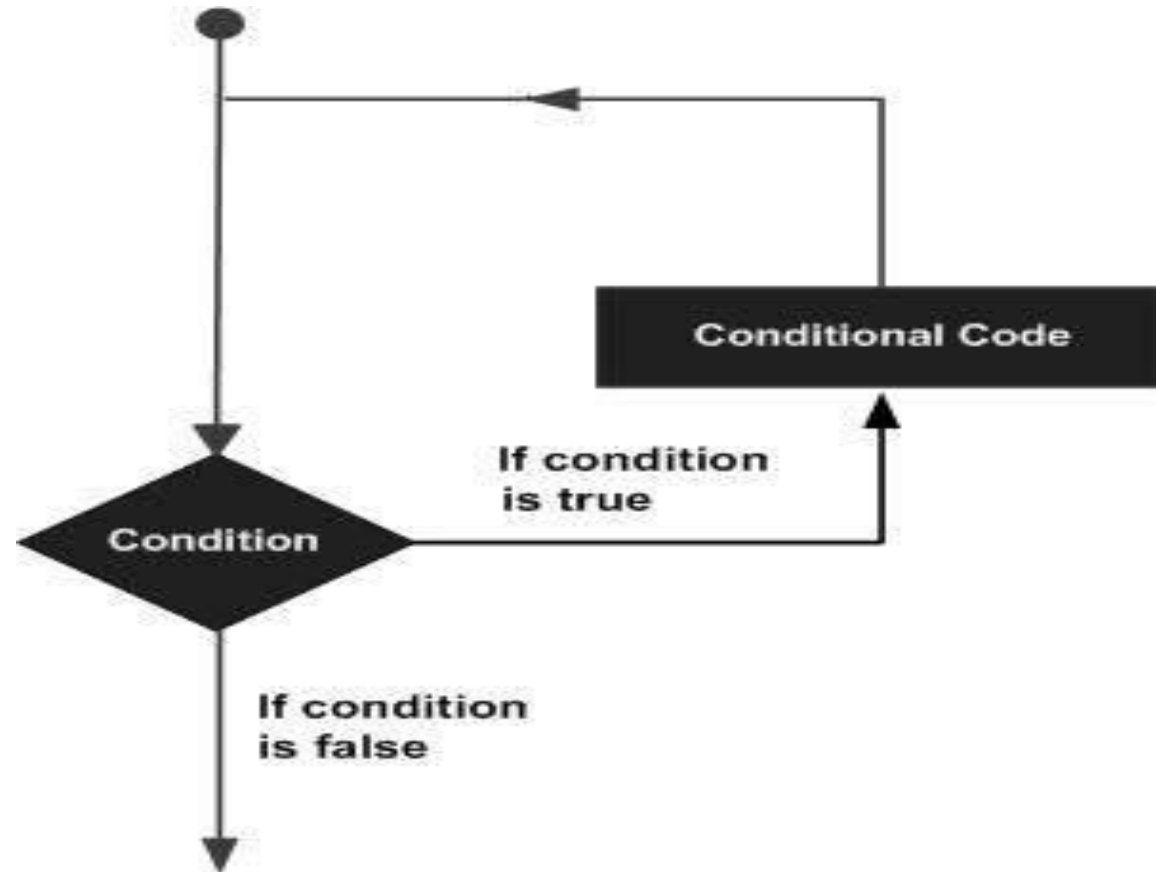
Difference between if and unless

Write a perl script to display the level of sugar entered by user and also whether is normal or high.

Write a perl script Check whether entered base pair is dna or not using switch case.

Perl - Loops

A loop statement allows us to **execute a statement or group of statements multiple times** and following is the general form of a loop statement in most of the programming languages –



Perl programming language provides the following **types of loop** to handle the looping requirements.

1. **while loop:-**

A **while loop** statement in Perl programming language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a while loop in Perl programming language is –

```
while(condition)
{
    statement(s);
    increment statement;
}
```

2. until Loop

An until loop statement in Perl programming language repeatedly executes a target statement as long as a given condition is false.

Syntax

The syntax of an until loop in Perl programming language is –

```
until(condition)
{
    statement(s);
}
```

3. FOREACH Loops:

A foreach loop runs a block of code for each element of a list.

```
my @numbers = ( 1, 3, 7 );
```

Example:

1. Using arrays

```
foreach $n( @numbers ) {  
    print $a;  
}
```


4. for loop:-

A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

Syntax

The syntax of a while loop in Perl programming language is –

```
for(start;condition; increment)
{
    statement(s);
}
```

5. do...while loop

- Unlike for and while loops, which test the loop condition at the top of the loop, the do...while loop checks its condition at the bottom of the loop.
- A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.
- Syntax

The syntax of a do...while loop in Perl is –

```
do {  
    statement(s);  
}while( condition );
```

7. nested for loop

A loop can be nested inside of another loop. Perl allows to nest all type of loops to be nested.

Syntax

The syntax for a nested for loop statement in Perl is as follows –

```
for ( init; condition; increment )  
{  
    for ( init; condition; increment )  
    {  
        statement(s);  
    }  
    statement(s);  
}
```

Viva Questions:

- Difference between for and foreach.
- Display 4 to 30(range operator) numbers using foreach loop
- How this loop runs and output for the same?

```
for ($i=0;$i<=2;$i++)  
{  
    for ($j=0;$j<=2;$j++)  
    {  
        print $i," ";  
    }  
    print "\n";  
}
```

ARRAY OPERATORS

ARRAY

- An array is a variable **that stores an ordered list of scalar values.**
- Array variables are preceded by an "at" (@) sign.
- To refer to a **single element** of an array, you will use the **dollar sign (\$)** with the variable name followed by the **index of the element in square brackets.**

Adding and Removing Elements in Array

Perl provides a number of useful functions to add and remove elements in an array.

1. **push @ARRAY, LIST**

Pushes the values of the list onto the end of the array.

2. **pop @ARRAY**

Pops off and returns the last value of the array.

3. **shift @ARRAY**

Shifts the first value of the array off and returns it, shortening the array by 1 and moving everything down.

4. **unshift @ARRAY, LIST**

Prepends list to the front of the array, and returns the number of elements in the new array.

```
@coins = ("Quarter","Dime","Nickel");  
print "1. \@coins = @coins\n";
```

add one element at the end of the array

```
push(@coins, "Penny");  
print "2. \@coins = @coins\n";
```

add one element at the beginning of the array

```
unshift(@coins, "Dollar");  
print "3. \@coins = @coins\n";
```

remove one element from the last of the array.

```
pop(@coins);  
print "4. \@coins = @coins\n";
```

remove one element from the beginning of the array.

```
shift(@coins);  
print "5. \@coins = @coins\n";
```

```
$len=$#coins+1;  
print "6. $len = $len\n";
```


This will produce the following result –

1. @coins = Quarter Dime Nickel
2. @coins = Quarter Dime Nickel Penny
3. @coins = Dollar Quarter Dime Nickel Penny
4. @coins = Dollar Quarter Dime Nickel
5. @coins = Quarter Dime Nickel
6. \$len = 3

Slicing Array Elements

You can also extract a "slice" from an array - that is, you can select more than one item from an array in order to produce another array.

Ex:

```
@days = qw/Mon Tue Wed Thu Fri Sat Sun/;
```

```
@weekdays = @days[3,4,5];
```

```
print "@weekdays\n";
```

This will produce the following result –

Thu Fri Sat

The specification for a slice must have a list of valid indices each separated by a comma. For speed, you can also use the .. range operator –

Ex:

```
@days = qw/Mon Tue Wed Thu Fri Sat Sun/;
```

```
@weekdays = @days[3..5];
```

```
print "@weekdays\n";
```

This will produce the following result –

```
Thu Fri Sat
```

Replacing Array Elements

Now we are going to introduce one more function called `splice()`, which has the following syntax –

`splice (@ARRAY, OFFSET , LENGTH , LIST)`

This function will remove the elements of `@ARRAY` designated by `OFFSET` and `LENGTH`, and replaces them with `LIST`, if specified. Finally, it returns the elements removed from the array. Following is the example –

Ex:

```
@nums = (1..20);  
print "Before - @nums\n";
```

```
splice(@nums, 5, 5, 21..25);  
print "After - @nums\n";
```

This will produce the following result –

Before - 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

After - 1 2 3 4 5 21 22 23 24 25 11 12 13 14 15 16 17 18 19 20

sort() function

The sort() function sorts each element of an array according to the ASCII Numeric standards. This function has the following syntax –

sort LIST

```
# define an array
```

```
@foods = qw(pizza steak chicken burgers);  
print "Before: @foods\n";
```

```
# sort this array
```

```
@foods = sort(@foods);  
print "After: @foods\n";
```

This will produce the following result –

Before: pizza steak chicken burgers

After: burgers chicken pizza steak

Merging Arrays

Because an array is just a comma-separated sequence of values, you can combine them together as shown below –

Ex:

```
@odd = (1,3,5);
```

```
@even = (2, 4, 6);
```

```
@numbers = (@odd, @even);
```

```
print "numbers = @numbers\n";
```

This will produce the following result –

```
numbers = 1 3 5 2 4 6
```

Reverse of an Arrays:

Syntax:

```
reverse(list_name);
```

Ex:

```
@odd = (1,3,5);
```

```
@numbers = reverse(@odd);
```

```
print "numbers = @numbers\n";
```

Viva Questions:

How to add one element at the end of the array?

How to add one element at the start of the array?

How to remove one element at the end of the array?

How to remove one element at the start of the array?

What's the use of Slicing Operator?

How to do sort in descending order of numeric values in an array. Give syntax ?

How to do sort in descending order of string values in an array. Give syntax ?

Give syntax for adding two arrays in one.

How to reversing an array?

Which function used for replacing some elements of an array?

How to get last index number of an array?

OOP in Perl

Object-Oriented Programming Concepts

1. Object

- An object is a **single entity that combines both data and methods**.
- Actions or methods describe what it can do. It is the code part of the object.
- Attributes or properties describe what information the object conveys. It is the data part of the object.

Example:

An object can be anything tangible or intangible. A **phone** is a tangible object that has attributes: serial, name, price, etc. You can change one of this information through methods that the phone object provides e.g., `set_name`, `get_name`, etc.

2. Class

A class is a blueprint or template of similar objects.

Example: We often say an object is an instance of a class e.g., a phone is an instance of Product class.

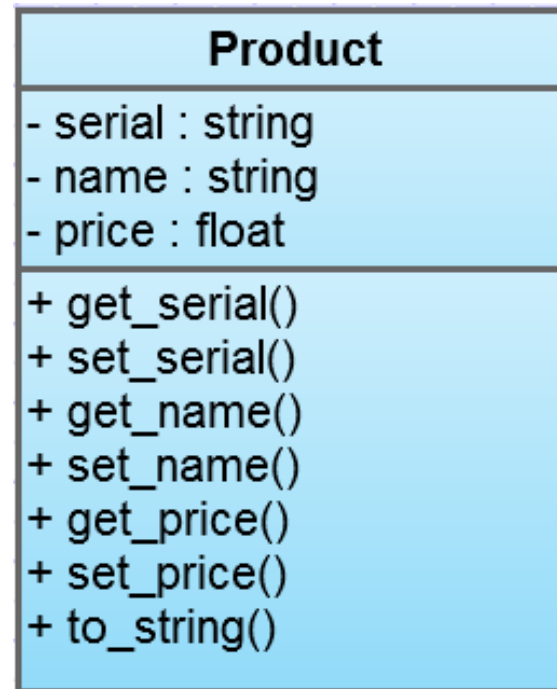


Fig: Class Product UI

3. Encapsulation

Through object, you can hide its complexity which is known as abstraction or encapsulation in object oriented programming. It means the client of the object does not need to care about the internal logic of the object but still can use the object through its interfaces (methods).

4. Inheritance

Inheritance means one class inherits both attributes and methods from another class. Inheritance enables you to reuse and extend existing classes without copy-n-paste the code or re-invent the wheel.

- A class that other classes inherit from is called base class or superclass.
- A class that inherits from other class called subclass or derived class.

5. Polymorphism

Polymorphism means “many forms” in Greek. It means a method call can behave differently depending on the type of the object that calls it.

Perl OOP rules

There are three important rules in Perl object oriented programming:

1. A class is a package.
2. An object is a reference that knows its class.
3. A method is a subroutine.

- 1) Class:
Package is itself a class.
Ex: package <Package Name>
- 2) To create an object, a function having a array reference or hash reference is created.
Ex: sub <Function Name> {
 my \$className = shift;
 my \$ref = {};

 bless \$ref, \$className;
 return \$ref;
 }
 → bless function creates a object's reference by blessing a reference to the package's class.

- 3) To create an object:
my \$object = new <Package or class Name>(<Parameters>);

Example 1:

Test1.pm

```
package Test1;
sub new
{
    $class=shift;
    $ref={id=>shift};
    bless $ref,$class;
    return $ref;
}
1;
```

oop.pl

```
use Test1;
$obj = new Test1(1);
print $obj->{id};
```

Example 2: Create a class product and display the product details.

Example 2: Create a class product and constructor and function display to display the product details.

Product.pm

```
package Product;
sub new{
    $class = shift;
    $ref = {serial=>shift,name=>shift,price=>shift};
    bless $ref,$class;
    return $ref;
}
sub to_string{
    $ref=shift;
    print "Serial: $ref->{serial}\nName: $ref->{name}\nPrice: $ref->{price}USD\n";
}

1;
```

oop.pl

```
use Product;
$obj = new Product("100","iPhone 5",650.00);
$obj->to_string();
```


Inheritance

Object-oriented programming sometimes involves inheritance. Inheritance simply means allowing one class called the Child to inherit methods and attributes from another, called the Parent, so you don't have to write the same code again and again. For example, we can have a class Employee which inherits from Person. This is referred to as an "isa" relationship because an employee is a person. Perl has a special variable, @ISA, to help with this. @ISA governs (method) inheritance.

Following are notable points while using inheritance

- Perl searches the class of the specified object.
- Perl searches the classes defined in the object class's @ISA array.
- If no method is found in steps 1 or 2, then Perl uses an AUTOLOAD subroutine, if one is found in the @ISA tree.
- If a matching method still cannot be found, then Perl searches for the method within the UNIVERSAL class (package) that comes as part of the standard Perl library.
- If the method still hasn't been found, then Perl gives up and raises a runtime exception.

```
# Creating parent class
package Employee;

# Creating constructor
sub new
{
    # shift will take package name 'employee'
    # and assign it to variable 'class'
    my $class = shift;

    my $self = {
        'name' => shift,
        'employee_id' => shift
    };

    # Bless function to bind object to class
    bless $self, $class;

    # returning object from constructor
    return $self;
}
1;
```

```
package Department;

use Employee;

# Using class employee as parent
our @ISA = 'Employee';

1;

oo.pl
use Department;

# Creating object and assigning values
$a=new Department("Shikhar",18017);
# Printing the required fields
print "$a->{'name'}\n";
print "$a->{'employee_id'}\n";
```

VIVA QUESTIONS

- What is the use of bless function? `bless $ref,$class_name`
- How to create class? `package ABC`
- How to create constructor in perl?

```
sub new{$class_name=shift; $ref={id=>shift,name=>shift} bless $ref,$class_name; return return;}
```

- Give syntax for creating object of a class.

```
$Classnameobj=new ABC(1,'DIMPAL');
```

- How to implement inheritance of any superclass?

```
Package XYZ;
```

```
Use ABC;
```

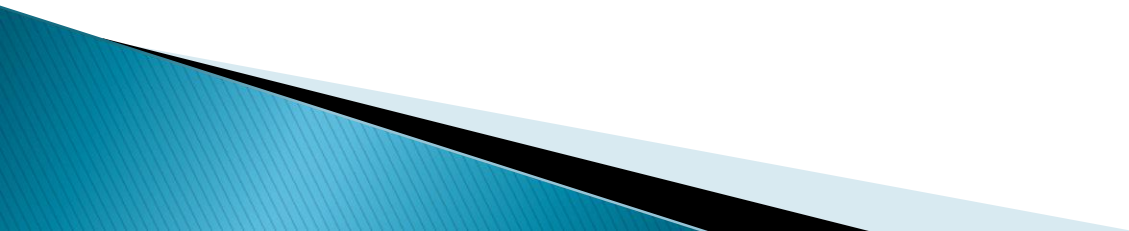
```
Our @ISA='ABC'
```

Homework:

Q.) Write a Perl script to create class Square and find area of square.

Thank You

Regular Expressions & Pattern Matching



Why we use Regular Expressions?



How to find
the date and
time from the
log file



```
Jul 1 03:27:12  
syslog:  
[m_java]  
1/Jul/2013  
03:27:12.818[j:  
[SessionThread  
<]^lat  
com/avc/abc/m  
agr/service/find  
.something(abc  
/1235/locator/a  
bc;Ljava/lang/S  
tring;)Labc/abc/  
abcd/abcd;(byt  
ecode:7)
```

→ Date and Time

Email or Phone	Password	Log In
Forgotten account?		



Sc # .com

sk@gmail.com

dk@gmail.com

sd@gmail.com

Cf @ cm



How to
verify these
e-mail
addresses?

Phone no.:
444-123-2344
10-1029-4562
109-2937-36
100-2939-9390
9292-2929-47



How to verify the phone
number and find the
country to which it
belongs?

Student Database:

Name: Saurabh

Age: 32

Address: F-127, Starc
tower, NewYork

59006

Name: Neel

Age: 65

Address: d-2, NewYork

59006

.....
.....



59006

59006

59006

59006



Replace the string

59076

59076

59076

59076

Find a particular string
from student data

What are Regular Expressions?

A Regular Expression is a special text string for describing a search pattern.

?



Can you identify the pattern to
get the Name and Age



NameAge = '''

Janice is 22 and Theon is 33

Gabriel is 44 and Joey is 21

'''



{'Janice': '22', 'Theon': '33',
'Gabriel': '44', 'Joey': '21'}

?



Can you identify the pattern to
get the Name and Age



NameAge = '''

Janice is 22 and Theon is 33
Gabriel is 44 and Joey is 21
'''



{'Janice': '22', 'Theon': '33',
'Gabriel': '44', 'Joey': '21'}



First letter of all the Names is
an uppercase letter and Age is
represented by numbers

```
ages = re.findall(r'\d{1,3}', NameAge)  
names = re.findall(r'[A-Z][a-z]*', NameAge)
```

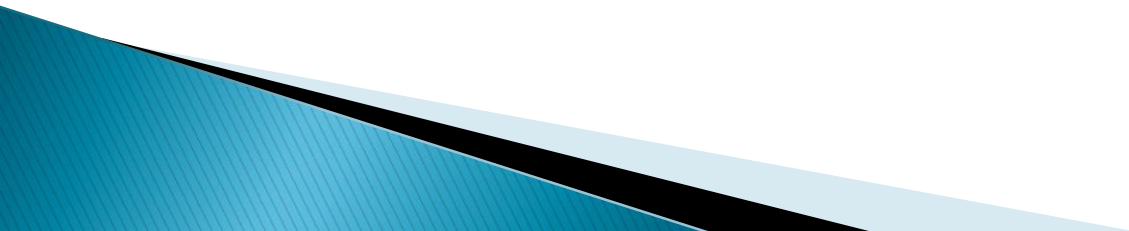


Definitions

Pattern Match – searching for a specified pattern within string.

For example:

A sequence like motif etc



Regular Expressions

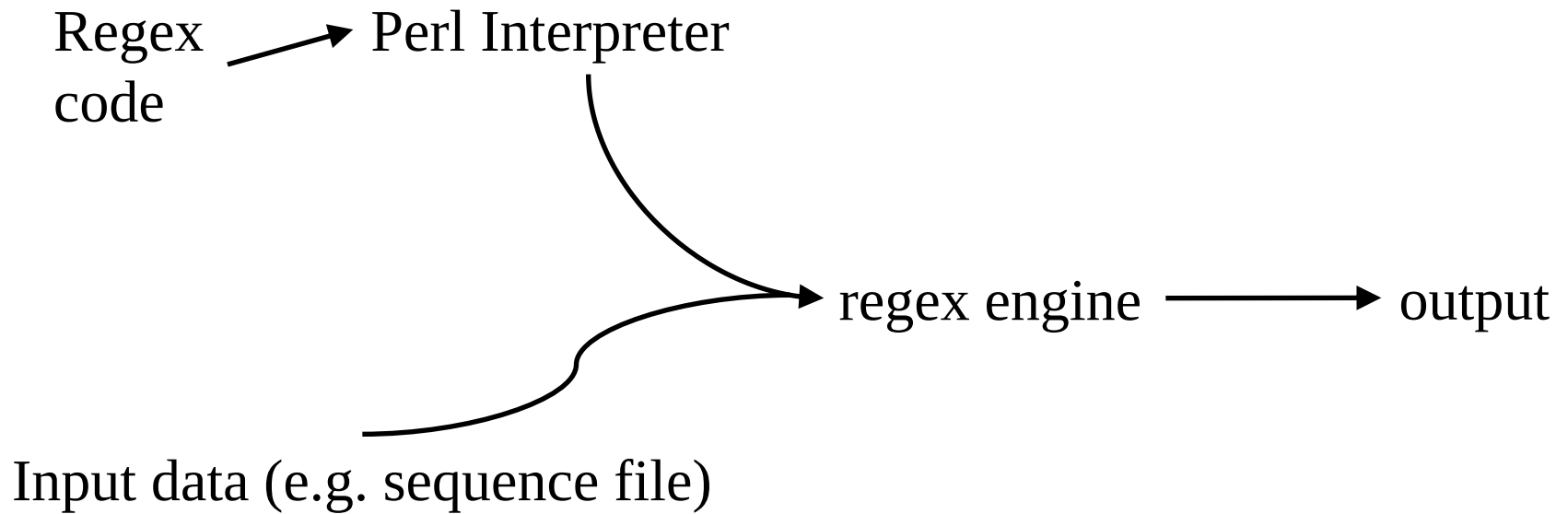
It is string of characters that defines the pattern or patterns you are searching.

Utilised in most popular languages – usually as separate library

Perl – fully incorporated.

Popular programming languages have Regex capabilities Python, Perl, JavaScript, Ruby ,Tcl, C++,C#,javascript

How Regex work



Binding Operator

Matching Operator:

Want to match against a scalar variable?

Binding Operator “=~” matches pattern on right against string on left.

Usually add the `m` operator – clarity of code.

```
$string =~ m/pattern/
```


There are **three regular expression operators** within Perl

- Match Regular Expression - `m//`
- Substitute Regular Expression - `s///`
- Transliterate Regular Expression - `tr///`

The Match Operator:-

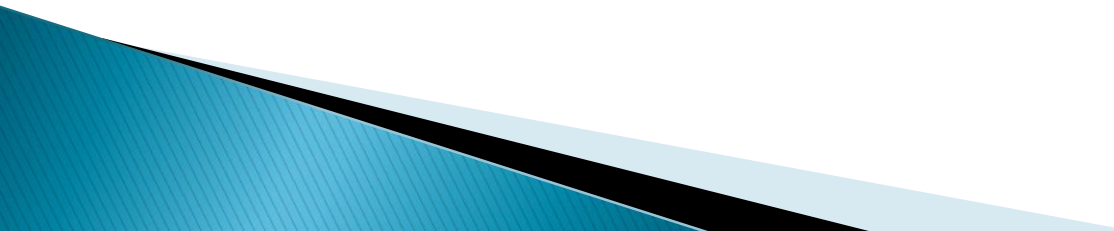
- The match operator, **m//**, is used to **match a string or statement** to a regular expression.
- For example, to match the character sequence "foo" against the scalar \$bar, you might use a statement like this –

```
$bar = "This is foo and again foo";  
if ($bar =~ m/foo/){  
    print "First time is matching\n";  
}else{  
    print "First time is not matching\n";  
}
```

m{}, m() are all valid

Match Operator Modifiers

The match operator supports its own set of modifiers. The **/g** modifier allows for global matching. The **/i** modifier will make the match case insensitive. Here is the complete **list of modifiers**



Modifier	Description
i	Makes the match case insensitive.
m	Specifies that if the string has newline or carriage return characters, the ^ and \$ operators will now match against a newline boundary, instead of a string boundary .
o	Evaluates the expression only once.
x	Allows you to use white space in the expression for clarity.
g	Globally finds all matches.
cg	Allows the search to continue even after a global match fails.



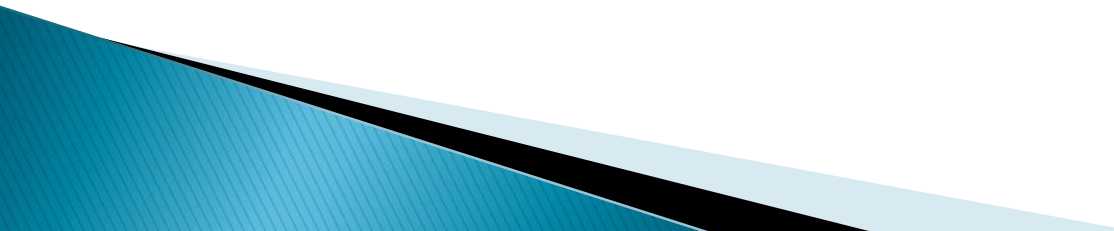
Substitution Operator

The substitution operator, `s///`, is really just an extension of the match operator that allows you to replace the text matched with some new text. The basic form of the operator is –

```
s/PATTERN/REPLACEMENT/;
```

```
$string = "The cat sat on the mat";  
$string =~ s/cat/dog/;
```

```
print "$string\n";
```



Translation Operator:-

Translation is similar, but not identical, to the principles of substitution

```
tr/SEARCHLIST/REPLACEMENTLIST/
```

```
$string = 'The cat sat on the mat';
```

```
$string =~ tr/a/o/;
```

```
print "$string\n";
```

We can also replace by ranges:-

```
$string =~ tr/a-z/A-Z/;
```

Modifier	Description
c	Complements SEARCHLIST.
s	Squashes duplicate replaced characters.

```
$string = 'food';  
$string = 'food';  
$string =~ tr/a-z/a-z/s;  
  
print "$string\n";
```

Regular Expressions

- In regular expressions, the following character set symbols match a single character. So, to handle arbitrary blank space you must use `\s+`. These symbols may be used in a character set. So, when looking for a hex integer you could use `[a-fA-F\d]`.

Symbol	Equiv	Description
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	A "word" character (alphanumeric plus "_")
<code>\W</code>	<code>[^a-zA-Z0-9_]</code>	Match a non-word character
<code>\s</code>	<code>[\t\n\f\r]</code>	Match a whitespace character
<code>\S</code>	<code>[^\s]</code>	Match a non-whitespace character
<code>\d</code>	<code>[0-9]</code>	Match a digit character
<code>\D</code>	<code>[^0-9]</code>	Match a non-digit character

Metacharacters

char meaning

^ beginning of string

\$ end of string

.

any character except newline

*

match 0 or more times

+

match 1 or more times

?

match 0 or 1 times; or: shortest match

|

alternative

()

grouping; “storing”

[]

set of characters

{ }

repetition modifier

\

quote or special

expression matches...

abc (that exact character sequence, but anywhere in the string)

^abc abc at the *beginning* of the string

abc\$ abc at the *end* of the string

a|b either of a and b

^abc|abc\$ the string abc at the beginning or at the end of the string

Quantifiers in regular expressions

In order to provide more flexibility there are a number of special characters called quantifiers, that will change the number of times each character can match.

There are 3 main quantifiers:

- ? matching 0 or 1 occurrence.

- + matching 1 or more occurrence.

- * matching 0 or more occurrence.

In addition, using a pair of curly braces {} we can have even more flexible quantifiers.

Repetition

a^* zero or more a's

a^+ one or more a's

$a?$ zero or one a's (i.e., optional a)

$a\{m\}$ exactly m a's

$a\{m,\}$ at least m a's

$a\{m,n\}$ at least m but at most n a's repetition?

*** Any number of characters**

The star * means the character in-front of it can appear any number of times. Including 0 times.

+ One or more characters

The third common quantifier is the plus character + which means 1 or more occurrence.

{ } curly braces

Using curly braces we can express a lot of different amounts. Normally it is used to express a range so $x\{2, 4\}$ would mean 2, 3 or 4 x-es.

We can leave out the second number thereby removing the upper limit. $x\{2, \}$ means 2 or more x-es. If we also remove the comma the $x\{2\}$ means exactly 2 x-es. So in order to express the 3 common quantifiers we could use the curly braces:

$$? = \{0,1\}$$

$$+ = \{1,\}$$

$$* = \{0,\}$$

Regex # examples it would match

/ax*a/ # aa, axa, axxa, axxxa, ...

/ax+a/ # axa, axxa, axxxa, ...

/ax?a/ # aa, axa

/ax{2,4}a/ # axxa, axxxa, axxxxxa

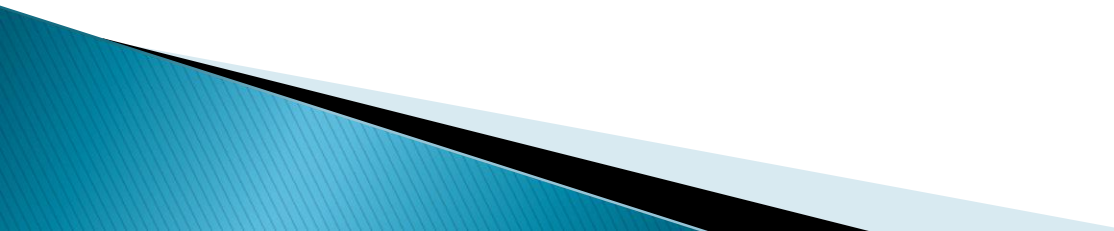
/ax{3,}a/ # axxxa, axxxxxa, ...

/ax{17}a/ # axxxxxxxxxxxxxxxxxxxxxxa

```
if ($str =~ /colou?r/) {  
    # match  
}
```

Note:

Here the question-mark ? is a quantifier that says, the letter 'u' can appear 0 or 1 times. That is, it might be there, or it might not be there. Exactly the two cases we wanted.



split() function

The **split() function** is used to split a string into smaller sections. You can split a string on a single character, a group of characters or a regular expression (a pattern).

Example 1. Splitting on a character

```
$data = 'Becky Alcorn,25,female,Melbourne';  
@values = split(',', $data);  
foreach $val (@values)  
{  
    print "$val\n";  
}
```

Example 2. Splitting on a string

```
$data = 'Bob the Builder~~~10:30am~~~1,6~~~ABC';  
@values = split('~~~', $data);  
foreach my $val (@values)  
{  
    print "$val\n";  
}
```

Example 3. Splitting on a pattern

```
my $data = 'Home1Work2Cafe3Work4Home';  
# \d+ matches one or more integer numbers  
@values = split(/\d+/, $data);  
foreach my $val (@values)  
{ print "$val\n"; }
```

Example Splitting into a hash

```
$data = 'FIRSTFIELD=1;SECONDFIELD=2;THIRDFIELD=3';  
%values = split(/[=;]/, $data);  
foreach $k (keys %values)  
{  
    print "$k: $values{$k}\n";  
}
```

join Function

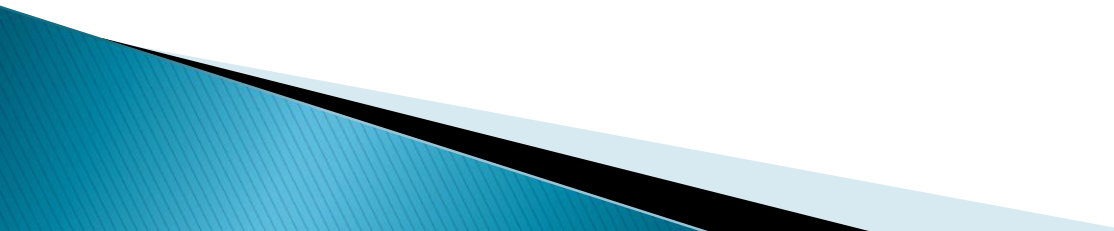
This function combines the elements of LIST into a single string using the value of EXPR to separate each element.

Syntax

Following is the simple syntax for this function –
join EXPR, LIST

Return Value

This function returns the joined string.

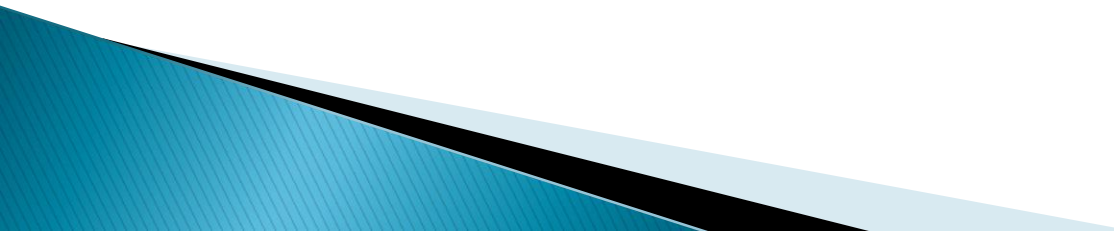


Example

Following is the example code showing its basic usage –

```
$string = join( “ “, "one", "two", "three" );
```

```
print "Joined String is $string\n";
```



Perl substr() function

String functions

1. lc, uc, length:-

There are a number of simple functions such as **lc** and **uc** to return the lower case and upper case versions of the original string respectively. Then there is **length** to return the number of characters in the given string.

Ex:

```
use 5.010;  
$str = 'HeLlo';
```

```
say lc $str; # hello  
say uc $str; # HELLO  
say length $str; # 5
```

index

Returns Index number of string
use 5.010;

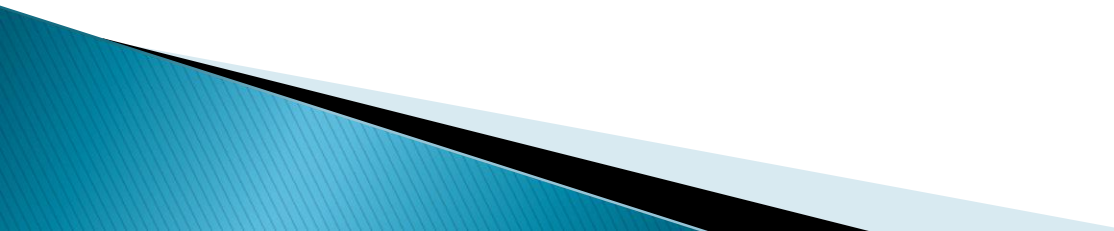
Ex:-

```
$str = "The black cat climbed the green tree";  
say index $str, 'cat'; # 10  
say index $str, 'dog'; # -1  
say index $str, "The"; # 0  
say index $str, "e "; # 2  
say index $str, "e ", 3; # 24
```

Substring Function:-

This function returns a substring of **EXPR**, starting at **OFFSET** is **zero** within the string. If **OFFSET** is negative, starts that many characters from the end of the string. If **LEN** is specified, returns that number of bytes, or all bytes up until end-of-string if not specified. If **LEN** is negative, leaves that many characters off the end of the string.

If **REPLACEMENT** is specified, replaces the substring with the **REPLACEMENT** string.



Syntax

Following is the simple syntax for this function –
substr EXPR, OFFSET, LEN, REPLACEMENT

substr EXPR, OFFSET, LEN

substr EXPR, OFFSET

Example

```
$temp = substr("okay", 2);  
print "Substring value is $temp\n";  
$temp = substr("okay", 1,2);  
print "Substring valuye is $temp\n";
```

Example:-

use 5.010;

\$str = "The black cat climbed the green tree";

say substr \$str, 4, 5; #black

say substr \$str, 4, -11; # black cat climbed the

say substr \$str, 14; # climbed the green tree

say substr \$str, -4; # tree

say substr \$str, -4, 2; # tr

Replace:

my \$z = substr \$str, 14, 7, "jumped from";

say \$str; # The black cat jumped from the green tree

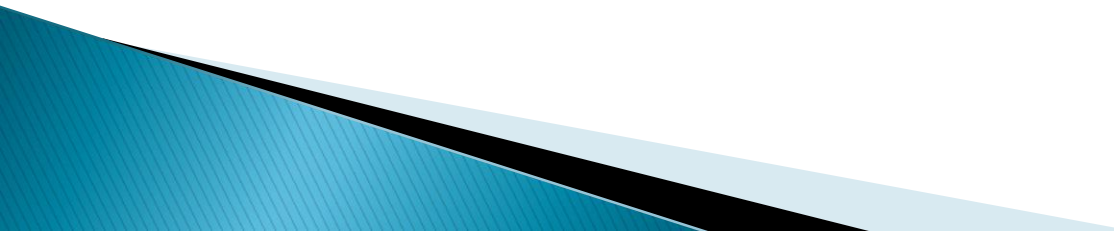
Perl Formatting Data

Perl uses a writing template called a 'format' to output reports. To use the format feature of Perl, you must –

- **Define a Format**
- **Pass the data that will be displayed on the format**
- **Invoke the Format**

Define a Format

```
format FormatName =  
fieldline  
value_one, value_two, value_three  
fieldline  
value_one, value_two  
.  
write;
```

- **FormatName** represents the name of the format.
 - The **fieldline** is the specific way the data should be formatted.
 - The **values** lines represent the values that will be entered into the fieldline.
 - You **end the format** with a single period.
 - **fieldline can contain any text or fieldholders. Fieldholders hold space for data**
- 

A fieldholder has the format –

@<<<<	left-justified
@>>>>	right-justified
@	centered
@#####.##	numeric field holder
@*	multiline field holder

Example 1:-

format STDOUT =

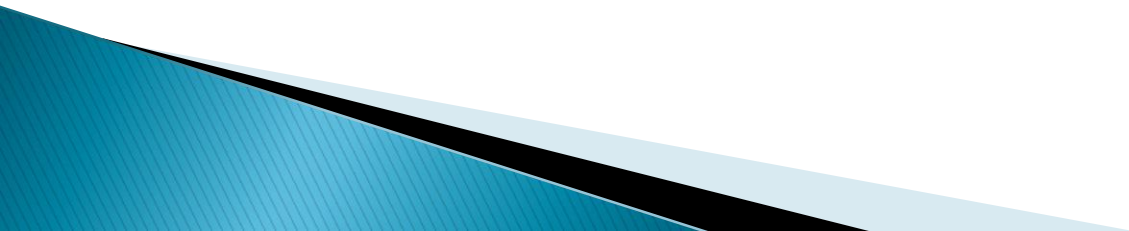
=====

Here is the text I want to display.

=====

.

write;



Example 2:-

format STDOUT =

@|||||

"CD Collection of David Medinets",

Album	Artist	Price
-----	-----	-----

.
write;

Example 2:-

```
$name='hello';
```

```
format STDOUT =
```

```
@||||||||||||||||||||
```

```
$name
```

```
.
```

```
write;
```


Example 3:-

```
@name=(['kani'],['vani'],['ravi']);
```

```
foreach $row (@name) {
format STDOUT =
```

@<<<<<<<<<<<<<

@\$row

•

```
write;
```

}

Using the Format

In order to invoke this format declaration, we would use the write keyword –
write EMPLOYEE;

The problem is that the format name is usually the name of an open file handle, and the write statement will send the output to this file handle. **As we want the data sent to the STDOUT**, we must associate EMPLOYEE with the STDOUT filehandle. First, however, we must make sure that that STDOUT is our selected file handle, using the **select() function**.

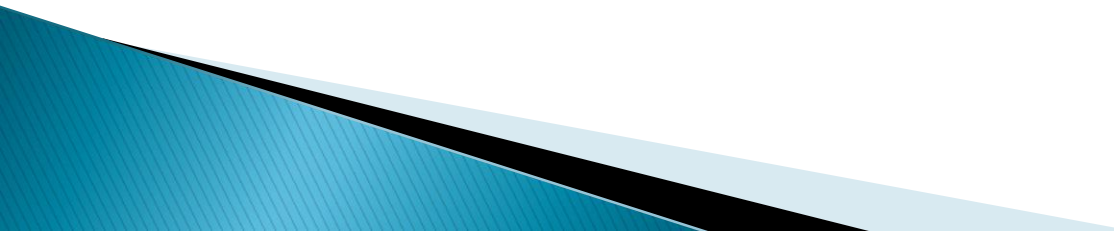
select(STDOUT);

We would then associate EMPLOYEE with STDOUT by setting the new format name with STDOUT, **using the special variable \$~ or \$FORMAT_NAME as follows** –

\$~ = "EMPLOYEE";

When we now do a write(), the data would be sent to STDOUT.

Remember: if you are going to write your report in any other file handle instead of STDOUT then you can use select() function to select that file handle and rest of the logic will remain the same.



white,
}

```
=====
@<<<<<<<<<<<<<<<<<<<< @<<
$name,$age
@#####.##
$salary
=====
```

```
@n = ("Ali", "Raza", "Jaffer");
@a = (20,30, 40);
@s = (2000.00, 2500.00, 4000.000);
```

```
for($i=0;$i<3;$i++) {
    $name = $n[$i];
    $age = $a[$i];
    $salary = $s[$i];
    write;
}
```

```
$salary = $$[$1];  
write;  
}
```

=====

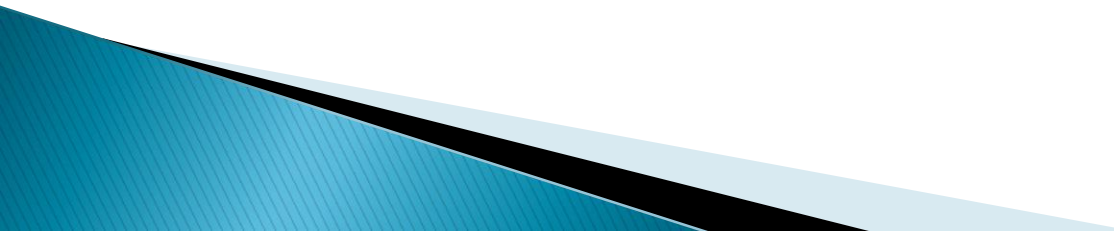
=====

.

=====

•

Packages in Perl

- A Perl package is a collection of code **which resides in its own namespace.**
 - Perl module is a package defined in a file having the same name as that of the package and having extension **.pm.**
 - **Two different modules** may contain a variable or a function of the same name.
 - **1;** is written at the end of the code to return a true value to the interpreter.
 - To use module or package or libraries, we use **require or use functions**
- 

Syntax

1. Create Package

package <name_of_package>; // .pm

2. Use package

use <name_of_package>; // .pl

Declaration of a Perl Module

Name of the module must be the same as that of the package name and has a .pm extension.

Example : Calculator.pm

```
package Calculator;
```

```
sub addition
```

```
{
```

```
    $a = $_[0];
```

```
    $b = $_[1];
```

```
    $a = $a + $b;
```

```
    print "\n***Addition is $a";
```

```
}
```

```
sub subtraction
```

```
{
```

```
    $a = $_[0];
```

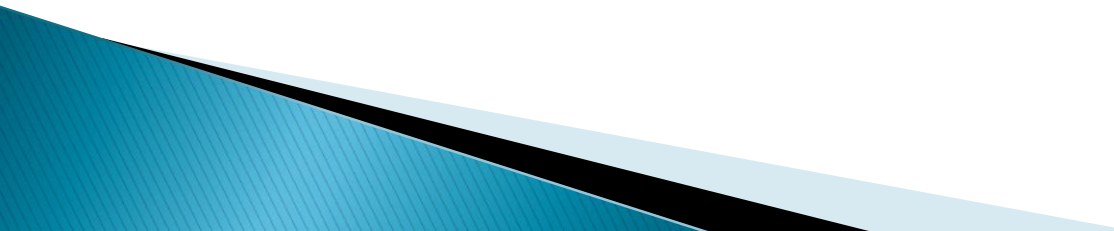
```
    $b = $_[1];
```

```
    $a = $a - $b;
```

```
    print "\n***Subtraction is $a";
```

```
}
```

```
1;
```



Examples: Test.pl

use Calculator;

\$a = 10;

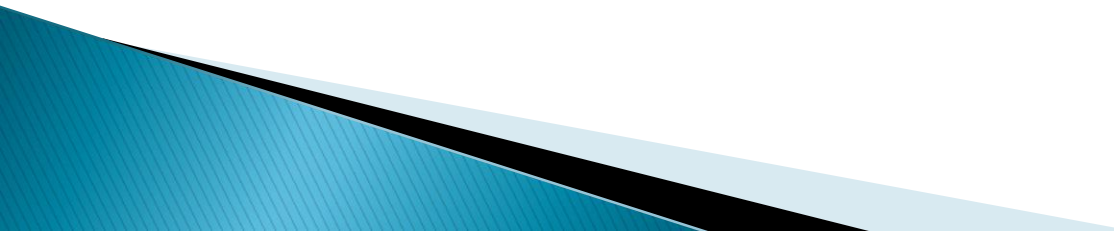
\$b = 20;

Calculator::addition(\$a, \$b);

\$a = 30;

\$b = 10;

Calculator::subtraction(\$a, \$b);



THANK YOU

DBI

The Perl Database Module

What you mean by DBI?

- DBI stands for **Database Independent** Interface/module for Perl which means DBI provides an **abstraction layer between the Perl code and the underlying database**, allowing you to switch database implementations really easily.
- The DBI is a **database access** module for the Perl programming language.
- It provides a **set of methods, variables, and conventions** that provide a consistent database interface, independent of the actual database being used.

DBI History – The Bad Old Days

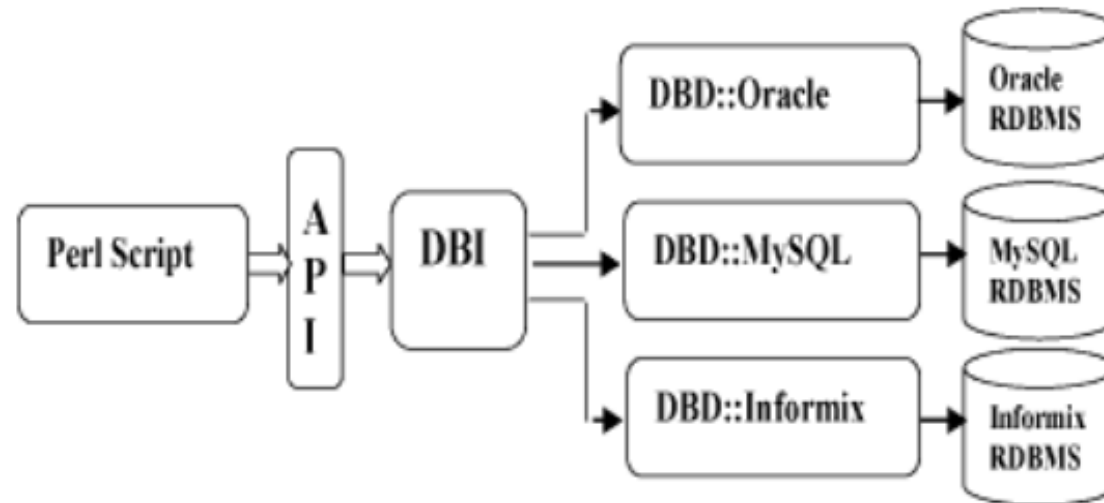
Different builds of perl for each database

- oraperl
- syperl
- INGperl

Each with it's own, incompatible API

Architecture of a DBI Application

DBI is independent of any database available in backend. You can use DBI whether you are working with Oracle, MySQL or Informix etc. This is clear from the following architecture diagram.



Here DBI is responsible of taking all SQL commands through the API, (ie. Application Programming Interface) and to dispatch them to the appropriate driver for actual execution. And finally DBI is responsible of taking results from the driver and giving back it to the calling script.

Notation and Conventions

\$dsn	Database source name
\$dbh	Database handle object
\$sth	Statement handle object
\$h	Any of the handle types above (\$dbh, \$sth, or \$drh)
@ary	List of values returned from the database.
\$rows	Number of rows processed (if available, else -1)
\$fh	A filehandle
undef	NULL values are represented by undefined values in Perl

Database Connection

Assuming we are going to work with MySQL database. Before connecting to a database make sure followings.

- You have created a database with a name TESTDB.
- You have created a table with a name TEST_TABLE in TESTDB.
- This table is having fields FIRST_NAME, LAST_NAME, AGE, SEX and INCOME.
- User ID "testuser" and password "test123" are set to access TESTDB
- Perl Module DBI is installed properly on your machine.
- You have gone through MySQL tutorial to understand MySQL Basics.

How to download and install mysql

- <https://dev.mysql.com/downloads/windows/installer/8.0.html>
- <https://www.onlinetutorialspoint.com/mysql/install-mysql-on-windows-10-step-by-step.html>
- **After installation run this command at mysql:**

```
mysql >ALTER USER 'root'@'localhost' IDENTIFIED WITH  
mysql_native_password BY '1234';
```

```
mysql>FLUSH PRIVILEGES;
```

```
mysql>quit
```


Following is the example of connecting with MySQL database "TESTDB"

Example:

```
use DBI;
```

```
$dbh=DBI->connect("DBI:mysql:college","root","1234") or die $DBI::errstr;
```

```
print "Database connected sucessfully";
```

INSERT Operation

INSERT operation is required when you want to create some records into a table. Here we are using table TEST_TABLE to create our records. So once our database connection is established, we are ready to create records into TEST_TABLE. Following is the procedure to create single record into TEST_TABLE. You can create as many as records you like using the same concept.

Record creation takes following steps

- **Preparing SQL statement with INSERT statement. This will be done using prepare() API.**
- **Executing SQL query to select all the results from the database. This will be done using execute() API.**
- **Releasing Statement handle. This will be done using finish() API**

If everything goes fine then commit this operation otherwise you can rollback complete transaction. Commit and Rollback are explained in next sections.

Example:

```
my $sth = $dbh->prepare("INSERT INTO TEST_TABLE
                        (FIRST_NAME, LAST_NAME, SEX, AGE, INCOME )
                        values
                        ('john', 'poul', 'M', 30, 13000)");
$sth->execute() or die $DBI::errstr;
$sth->finish();
$dbh->commit or die $DBI::errstr;
```

Using Bind Values

There may be a case when values to be entered is not given in advance. So you can use bind variables which will take required values at run time. Perl DBI modules makes use of a question mark in place of actual value and then actual values are passed through execute() API at the run time. Following is the example:

Example:

```
my $first_name = "john";
my $last_name = "poul";
my $sex = "M";
my $income = 13000;
my $age = 30;
my $sth = $dbh->prepare("INSERT INTO TEST_TABLE
                        (FIRST_NAME, LAST_NAME, SEX, AGE, INCOME )
                        values(?,?,?,?)");
$sth->execute($first_name,$last_name,$sex, $age, $income)
    or die $DBI::errstr;
$sth->finish();
```

READ Operation

READ Operation on any database means to fetch some useful information from the database ie one or more records from one or more tables. So once our database connection is established, we are ready to make a query into this database. Following is the procedure to query all the records having AGE greater than 20. This will take four steps

- **Preparing SQL SELECT query based on required conditions. This will be done using prepare() API.**
- **Executing SQL query to select all the results from the database. This will be done using execute() API.**
- **Fetching all the results one by one and printing those results. This will be done using fetchrow_array() API.**
- **Releasing Statement handle. This will be done using finish() API**

```
my $sth = $dbh->prepare("SELECT FIRST_NAME, LAST_NAME
                        FROM TEST_TABLE
                        WHERE AGE > 20");
$sth->execute() or die $DBI::errstr;
print "Number of rows found :" + $sth->rows;
while (my @row = $sth->fetchrow_array()) {
    my ($first_name, $last_name) = @row;
    print "First Name = $first_name, Last Name = $last_name\n";
}
$sth->finish();
```

C/W: write a perl script to display employee data whose salary is less than entered value

```
use DBI;
print "Enter id and name";
chomp($sal=<stdin>);

$dbh=DBI->connect("DBI:mysql:company","root","12345") or die DBI->errstr;
print "Database connected sucessfully\n";
$stmt=$dbh->prepare("select id,name from employee where salary<?") or die DBI->errstr;
$stmt->execute($sal) or die DBI->errstr;
print "Sucessfully executed\n";
print "\n\n Employee Details\n";
while (my @row = $stmt->fetchrow_array()) {
    my ($id, $name ) = @row;
    print "First Name = $id, Last Name = $name\n";
}
$stmt->finish();
```

UPDATE Operation:

UPDATE Operation on any database means to update one or more records already available in the database tables. Following is the procedure to update all the records having SEX as 'M'. Here we will increase AGE of all the males by one year. This will take three steps

- **Preparing SQL query based on required conditions. This will be done using prepare() API.**
- **Executing SQL query to select all the results from the database. This will be done using execute() API.**
- **Releasing Statement handle. This will be done using finish() API**

If everything goes fine then commit this operation otherwise you can rollback complete transaction. See next section for commit and rollback APIs.

```
my $sth = $dbh->prepare("UPDATE TEST_TABLE
    SET AGE = AGE + 1
    WHERE SEX = 'M'");
$sth->execute() or die $DBI::errstr;
print "Number of rows updated :" + $sth->rows;
$sth->finish();
$dbh->commit or die $DBI::errstr;
```

Using Bind Values

There may be a case when condition is not given in advance. So you can use bind variables which will take required values at run time. Perl DBI modules makes use of a question mark in place of actual value and then actual values are passed through execute() API at the run time. Following is the example:

```
$sex = 'M';  
my $sth = $dbh->prepare("UPDATE TEST_TABLE  
    SET  AGE = AGE + 1  
    WHERE SEX = ?");  
$sth->execute('$sex') or die $DBI::errstr;  
print "Number of rows updated :" + $sth->rows;  
$sth->finish();  
$dbh->commit or die $DBI::errstr;
```

DELETE Operation

DELETE operation is required when you want to delete some records from your database. Following is the procedure to delete all the records from TEST_TABLE where AGE is equal to 30. This operation will take following steps.

- **Preparing SQL query based on required conditions. This will be done using prepare() API.**
- **Executing SQL query to delete required records from the database. This will be done using execute() API.**
- **Releasing Statement handle. This will be done using finish() API**

If everything goes fine then commit this operation otherwise you can rollback complete transaction.

```
$age = 30;  
my $sth = $dbh->prepare("DELETE FROM TEST_TABLE  
                        WHERE AGE = ?");  
$sth->execute( $age ) or die $DBI::errstr;  
print "Number of rows deleted :" + $sth->rows;  
$sth->finish();
```


Using do Statement

If you're doing an **UPDATE, INSERT, or DELETE** there is no data that comes back from the database, so **there is a short cut to perform this operation**. You can use do statement to execute any of the command as follows.

```
$dbh->do('DELETE FROM TEST_TABLE WHERE age =30');
```

do returns a true value if it succeeded, and a false value if it failed. Actually, if it succeeds it returns the number of affected rows. In the example it would return the number of rows that were actually deleted.

Example:

```
use DBI;
```

```
chomp($id=<stdin>);
```

```
chomp($name=<stdin>);
```

```
$dbh=DBI->connect("DBI:mysql:stud","root","12345") or die
```

```
$DBI->errstr;
```

```
print "successfully connected";
```

```
$dbh->do("insert into stud1 values(?,?)",undef,$id,$name);
```

```
print "successfully inserted";
```

COMMIT Operation

Commit is the operation which gives a green signal to database to finalize the changes and after this operation no change can be reverted to its original position.

Here is a simple example to call commit API.

```
$dbh->commit or die $dbh->errstr;
```

ROLLBACK Operation

If you are not satisfied with all the changes or you encounter an error in between of any operation , you can revert those changes to use rollback API.

Here is a simple example to call rollback API.

```
$dbh->rollback or die $dbh->errstr;
```

Viva question

- How to connect perl and sql
- How to create table in perl.
- Syntax for retrieve data from table using perl
- How to update values into table.
- Which function is used for saving and reversing the saved information from database
- Difference between prepare and do function.

THANK YOU

FILE HANDLE AND DIRECTORIES

File Systems

- A **file** is a named collection of related data
- A **file system** is the logical view that an operating system provides so that users can manage information as a collection of files
- A file system is often organized by grouping files **into directories**

Text and Binary Files

- In a **text file** the bytes of data are organized as characters.
- A **binary file** requires a specific interpretation of the bits based on the information in the file

Text and Binary Files

- The terms ***text file*** and ***binary file*** are somewhat misleading
- They seem to imply that the information in a text file is not stored as binary data
- Ultimately, all information on a computer is stored as binary digits
- These terms refer to how those bits are formatted: as chunks of 8 or 16 bits, interpreted as characters, or in some other special format

File Types

- Most files, whether they are in text or binary format, contain a specific type of information
 - For example, a file may contain a Java program, a JPEG image, or an MP3 audio clip
- The kind of information contained in a document is called the **file type**

File Types

Extensions	File type
txt	text data file
mp3, au, wav	audio file
gif, tiff, jpg	image file
doc, wp3	word processing document
java, c, cpp	program source files

Figure 11.1 Some common file types and their extensions

- File names are often separated, usually by a period, into two parts
 - Main name
 - File extension
- The **file extension** indicates the type of the file

File Access

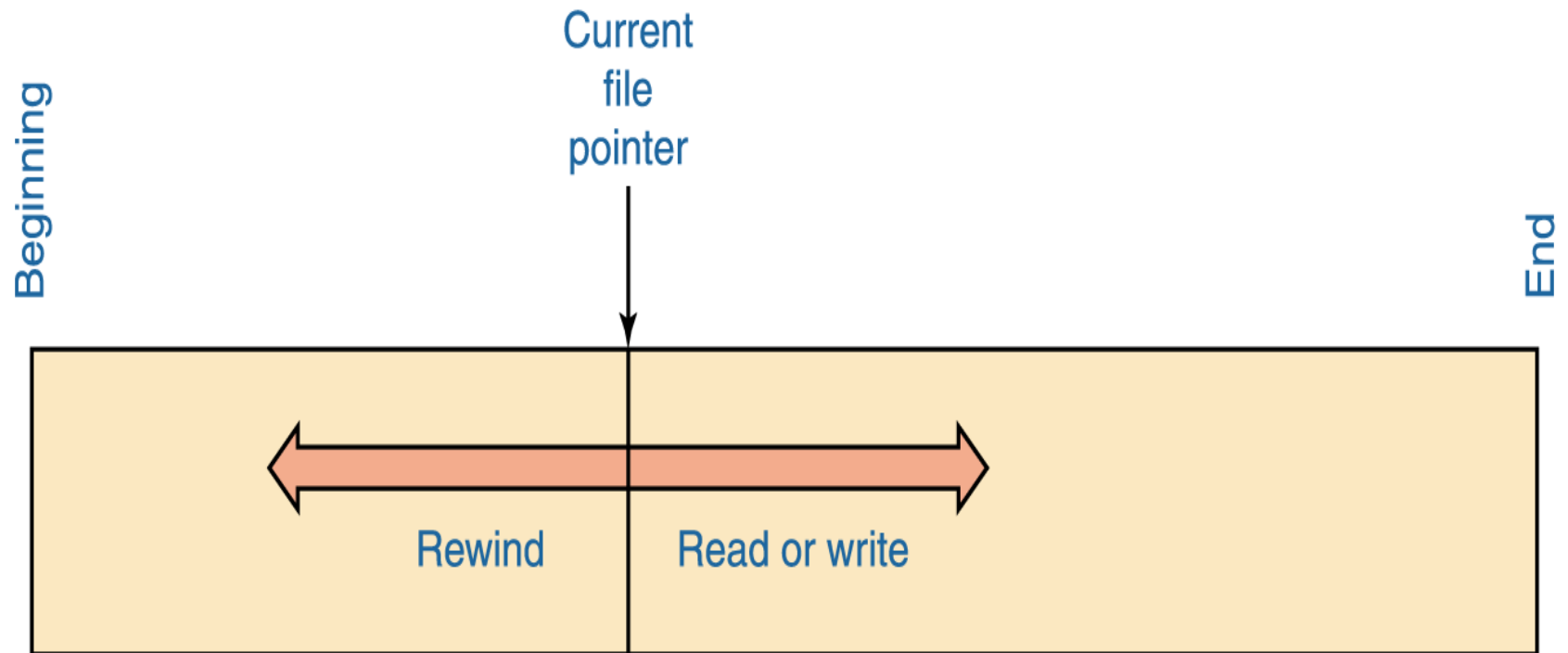


Figure 11.2 Sequential file access

File Access

- The most common access technique, and the simplest to implement, is **sequential access**
 - It requires that the information in the file be processed in order
 - Read and write operations move the current file pointer according to the amount of data that is read or written

File Access

- Files with **direct access** are conceptually divided into numbered logical records
- Direct access allows the user to set the file pointer to any particular record by specifying the record number

File Access

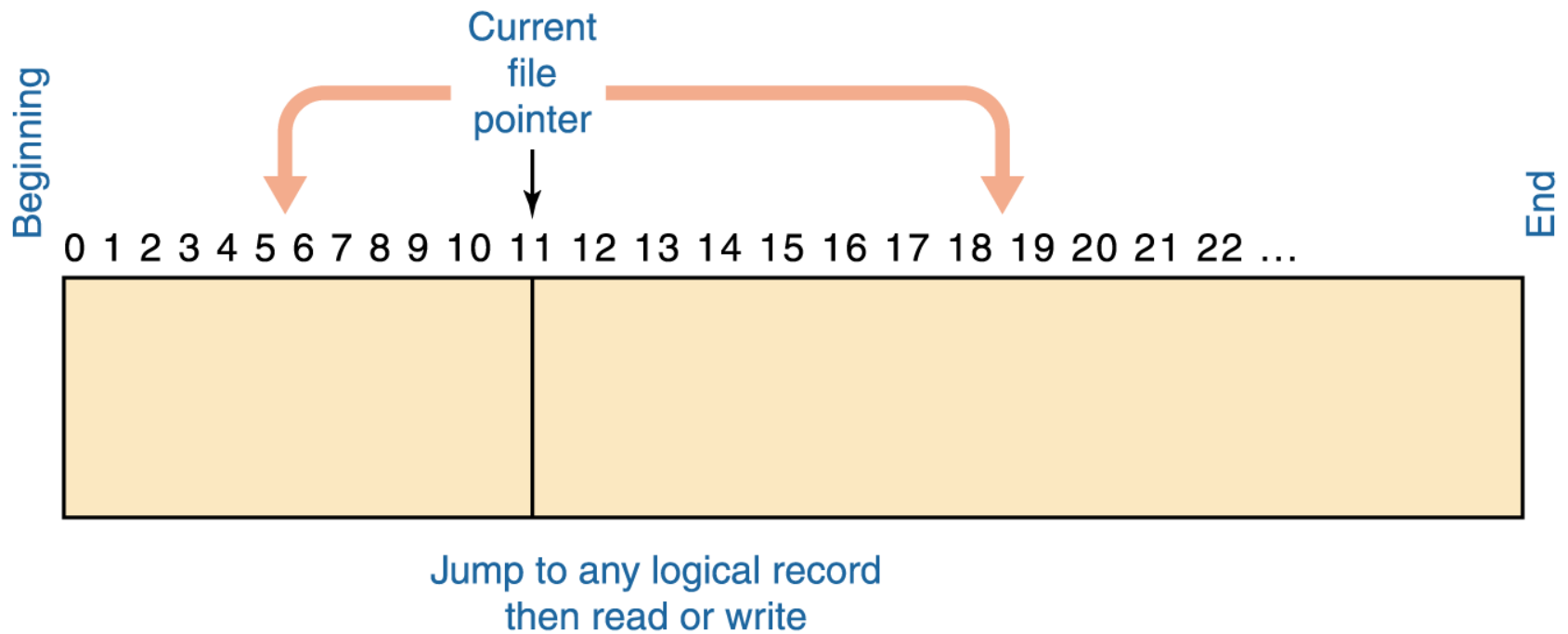


Figure 11.3 Direct file access

File Protection

- In multiuser systems, file protection is of primary importance
- We don't want one user to be able to access another user's files unless the access is specifically allowed
- A file protection mechanism determines who can use a file and for what general purpose

File Protection

- A file's protection settings in the Unix operating system is divided into three categories
 - Owner
 - Group
 - World

	Read	Write/Delete
Owner	Yes	Yes
Group	Yes	No
World	No	No

File Operations

- Create a file
- Delete a file
- Open a file
- Close a file
- Read data from a file
- Write data to a file
- Reposition the current file pointer in a file
- Append data to the end of a file
- Truncate a file (delete its contents)
- Rename a file
- Copy a file

File handling

- `open(handle, "filename")`
- `<handle> read a record from file`
- `close handle`
- `print handle expressionlist`

Opening and closing a file

`open (F, "f.txt")` to read
`open (F, ">f.txt")` to write a new file
`open (F, ">>f.txt")` to append
`open (F, "+<f.txt")` read/write

Return value is non-zero or **undef** in case of error.

`close F` closes the file

getc Function

The getc function returns a single character from the specified FILEHANDLE, or STDIN if none is specified –

Syntax:

getc FILEHANDLE

If there was an error, or the filehandle is at end of file, then undef is returned instead.

read Function

The read function reads a block of information from the buffered filehandle: This function is used to read binary data from the file.

Syntax:

read FILEHANDLE, SCALAR, LENGTH, OFFSET

read FILEHANDLE, SCALAR, LENGTH

The length of the data read is defined by LENGTH, and the data is placed at the start of SCALAR if no OFFSET is specified. Otherwise data is placed after OFFSET bytes in SCALAR. The function returns the number of bytes read on success, zero at end of file, or undef if there was an error.

Reading record from file

```
$my="C:\\Users\\LINO\\Desktop\\abc.txt";  
open(FH,'<',$my)or die $!;  
@row =<FH>;  
print @row,"\n";  
print getc <FH>;  
close FH
```

Example 1:

```
$my="C:\\Users\\LINO\\Desktop\\abc.txt";  
open(FH,'<',$my)or die $!;  
$ch=getc FH;  
print $ch;  
close FH
```

Example 2:

```
$d="";  
$my="C:\\Users\\LINO\\Desktop\\abc.txt";  
open(FH,'<',$my)or die $!;  
read FH,$d,2;  
print $d;  
close FH
```

Write record into file

```
$my="C:\\Users\\LINO\\Desktop\\xyz.txt";  
open(F,'>',$my)or die $!;  
print F "HELLO";  
close FH
```

DIRECTORIES

Create new Directory

You can use **mkdir** function to create a new directory.

Example:

```
$dir = "/tmp/perl";
```

```
# This creates perl directory in /tmp directory.
```

```
mkdir( $dir ) or die "Couldn't create $dir directory, $!";  
print "Directory created successfully\n";
```


Remove a directory

- You can use **rmdir** function to remove a directory.
- You will need to have the required permission to remove.
- A directory additionally this directory should be empty before you try to remove it.

Example:-

```
$dir = "/tmp/perl";
```

```
# This removes perl directory from /tmp directory.
```

```
rmdir( $dir ) or die "Couldn't remove $dir directory, $!";  
print "Directory removed successfully\n";
```

Change a Directory

- You can use **chdir** function to change a directory and go to a new location.
- You will need to have the required permission to change a directory and go inside the new directory.

```
$dir = "/home";
```

```
# This changes perl directory and moves you inside  
/home directory.
```

```
chdir( $dir ) or die "Couldn't go inside $dir directory, $!";  
print "Your new location is $dir\n";
```

Renaming Files

```
rename("oldfilename", "newfilename") || die  
"Cannot rename file.txt: $!";
```

Test Directory Exist or not

Then **-d** to check if it is a directory

Example:-

```
$somedir = "c:/windows";
```

```
if (-d $somedir)
```

```
{
```

```
    print "$somedir exists";
```

```
}
```

```
Else
```

```
{
```

```
    print "$somedir does not exist!";
```

```
}
```

Perl glob Function

- This function returns a list of files matching EXPR
- The * character matches zero or more characters of any type.

Syntax

glob EXPR

Example

```
#!/usr/bin/perl
```

```
@file_list= glob ("/perl/*.pl");
```

```
print "Returned list of file @file_list\n";
```

Directory Handle

1. opendir

Open directory handle

Syntax

```
opendir ( Path)
```

Opens up a directory handle to be used in subsequent `closedir()`, `readdir()`.

2. Opendir using directory handle

```
opendir(NT, path) || die "no $windir?: $!";
```

3. Reading a Directory Handle

After we have a directory handle open, we can read the list of names with **readdir**, which takes a single parameter:

Syntax:-

```
$name = readdir(NT)
```


4. Close Directory Handle

```
closedir(DIRECTORYHANDLE);
```

Example:-

```
$windir = "D:/RANI";  
opendir(NT, $windir) || die "no $windir?: $!";  
while ($name = readdir(NT))  
{  
    print "$name\n"; # prints ., .., system.ini, and so  
on  
}  
closedir(NT);
```

VIVA QUESTIONS:

What does read () command do?

How will you create a file in Perl?

How will you open a file in read-only mode in Perl?

How will you open a file in a write-only mode in Perl?

How to prevent file truncation in Perl?

How to read multi lines from a file in Perl?

How to close a file in Perl?

Which function is used to list all the files name starting with 'p' in folder?

Classwork:

Write a Check whether dna folder is there in your drive using perl script?

THANK YOU

Bioperl Modules

Ms. Sermarani Nadar
Department of Bioinformatics

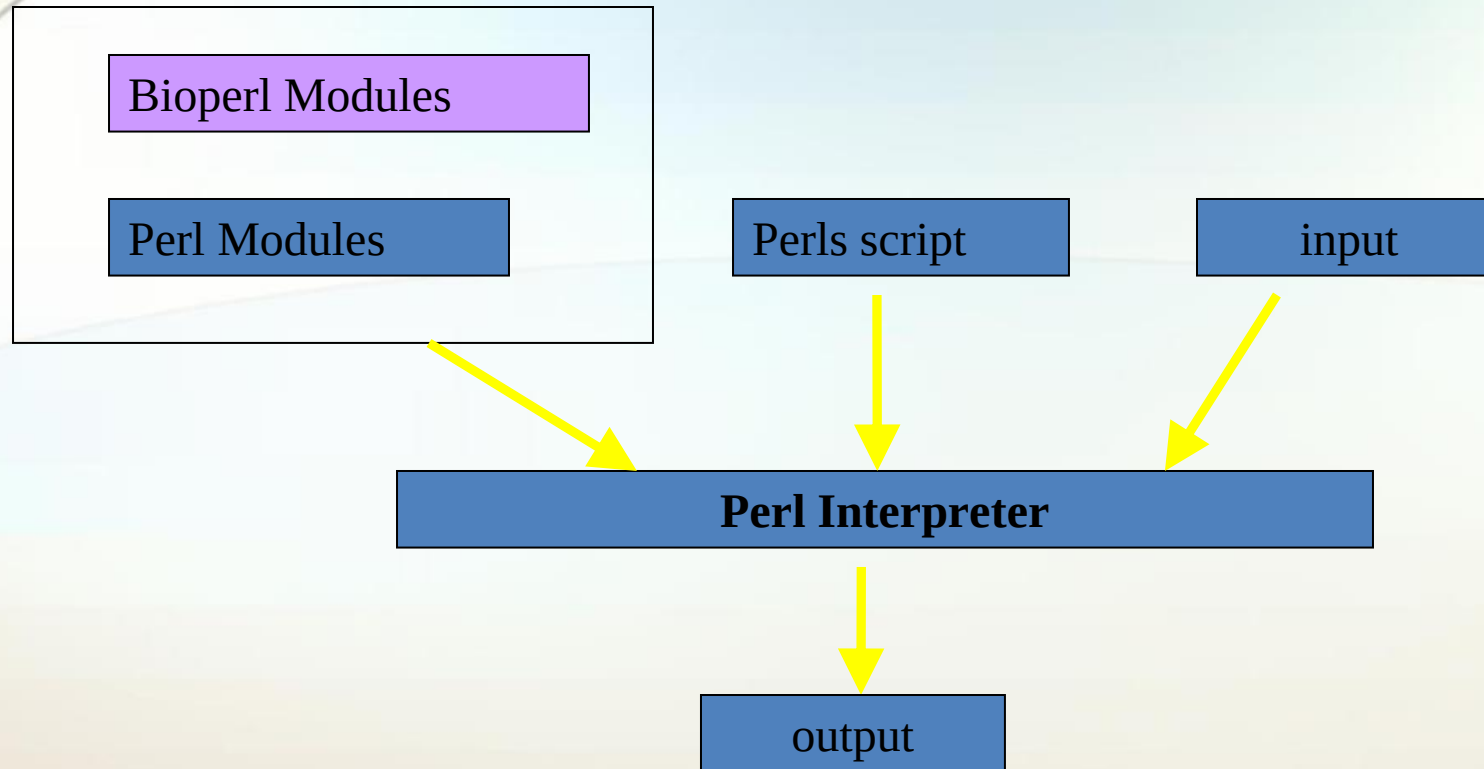
BioPerl

- BioPerl is an active open source software project supported by the Open Bioinformatics Foundation.
- BioPerl is a collection of Perl libraries for analyzing biological data.
- Sequence Analysis, Phylogenetic Analysis, Protein Structure Analysis, etc.
- It is NOT a separate programming language.
- Installation:

<http://strawberryperl.com/> - STRAWBERRY PERL

<https://medstrand.wordpress.com/installing-bioperl-on-windows/> -

STEPS FOR BIOPERL INSTALLATION



Bioperl and Perl

Why Bioperl for Bio-informatics?

- Perl is good at **file manipulation and text processing**, which make up a large part of the routine tasks in bio-informatics.
- Perl language, documentation and many Perl packages are **freely available**.
- Perl is easy to get started in, to **write small and medium-sized programs**.

Disadvantage of Bioperl(Future Aspects)

- Except sequence handling, all other **modules are not complete.**
- The **portability is not very good**, not all modules will work with on all platforms.

Bio::Seq Module

Bio::Seq

- It is central Module
- Using Bio::Seq module we can create Bio::Seq object

Bio::Seq

Bio::Seq - A sequence and a collection of sequence features
(an aggregate) with its own annotation.

```
$seq_obj = Bio::Seq->new(-seq => "aaaatggggggggggggcccccgtt",  
-display_id => "#12345",  
-desc => "example 1",  
-alphabet => "dna" );
```

Args	: -seq	=> sequence string
	-display_id	=> display id of the sequence
	-desc	=> alias for description
	-alphabet	=> skip alphabet guess and set it to dna, rna or protein

Title : seq

Usage : `$string = $seqobj->seq();`

Function: Get or set the sequence as a string of letters. An error is thrown if the sequence contains invalid characters

Returns : A scalar

Args : - Optional new sequence value (a string) to set
- Optional alphabet (it is guessed by default)

Title : subseq

Usage : `$substring = $seqobj->subseq(10,40);`
`$substring = $seqobj->subseq(10,40);`

Function: Return the subseq from start to end, where the first sequence character has coordinate 1 number is inclusive, ie 1-2 are the first two characters of the sequence.

Returns : a string

Args : integer for start position
integer for end position

Title : length

Usage : `$len = $seqobj->length();`

Function: Get the stored length of the sequence in number of symbols (bases or amino acids).

Returns : integer representing the length of the sequence.

Title : display_id or display_name

Usage : `$id_string = $seqobj->display_id();`

Function: Get or set the display id, the common name of the sequence object.

Returns : A string for the display ID

Args : Optional string for the display ID to set

Title : alphabet

Usage : `if( $seqobj->alphabet eq 'dna' ) { # Do something }`

Function: Get/set the alphabet of sequence, one of 'dna', 'rna' or 'protein'. This is case sensitive.

Returns : a string either 'dna','rna','protein'. NB - the object must make a call of the type - if there is no alphabet specified it has to guess.

Args : optional string to set : 'dna' | 'rna' | 'protein'

Title : desc or description

Usage : `$seqobj->desc($newval);`

Function: Get/set description of the sequence.

Returns : value of desc (a string)

Args : newvalue (a string or undef, optional)

Creating a sequence and an Object

Example 1:

use Bio::Seq;

```
$seq_obj = Bio::Seq->new(-seq => "aaaatggggggggggggcccggtt",  
                        -display_id => "#12345",  
                        -desc => "example 1",  
                        -alphabet => "dna" );
```

```
print $seq_obj->seq();
```

```
C:\Users\Nixon Mendez\Documents\Books\M.Tech\Semester 2\Bioperl>perl example1.pl
```

```
aaaatggggggggggggcccggtt
```

Bio::SeqIO Module

Bio::SeqIO

- Handler for SeqIO Formats
- Used for biological file handles
- Read or Write sequences
- Supported formats include fasta, genbank, embl, swiss (SwissProt), Entrez Gene and tracefile formats such as abi (ABI) and scf. quence objects

Sequence Input/Output

The *Bio::SeqIO* system was designed to make getting and storing sequences to and from the myriad of formats as easy as possible.

Bio::Perl has a number of other easy-to-use functions, including

get_sequence - gets a sequence from standard, internet accessible databases

read_sequence - reads a sequence from a file

read_all_sequences - reads all sequences from a file

new_sequence - makes a bioperl sequence just from a string

write_sequence - writes a single or an array of sequence to a file

translate - provides a translation of a sequence

translate_as_string - provides a translation of a sequence, returning back

just the sequence as a string

blast_sequence - BLASTs a sequence against standard databases at NCBI

write_blast - writes a blast report out to a file

write_seq - writes a sequence into a file

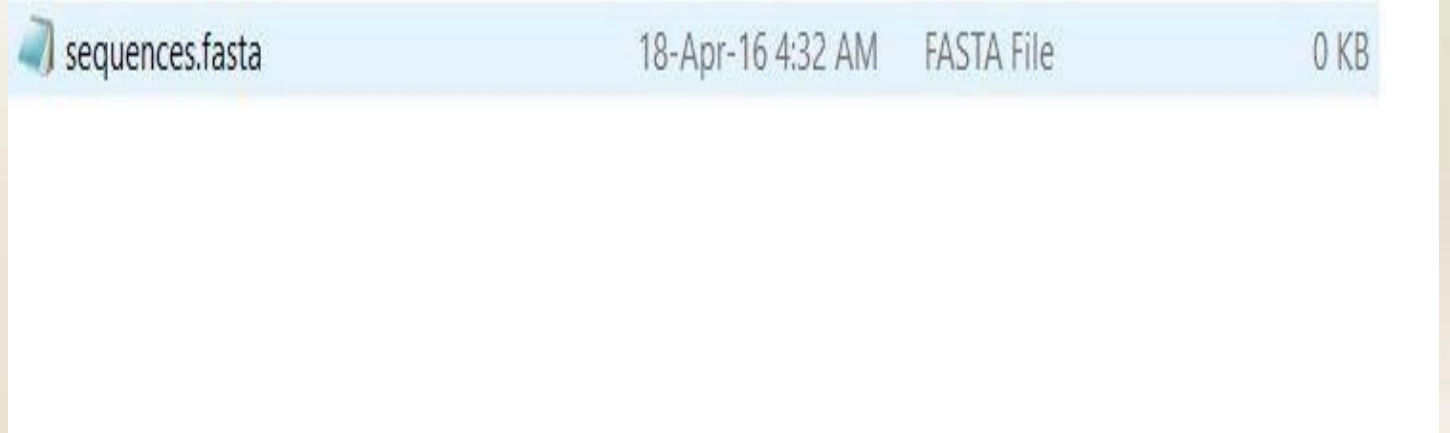
revcom – reverse compliments

Writing a sequence to a file

Example 2:

```
use Bio::SeqIO;
```

```
$seqio_obj = Bio::SeqIO->new(-file => ">sequence.fasta", -  
format  
=> 'fasta' );
```



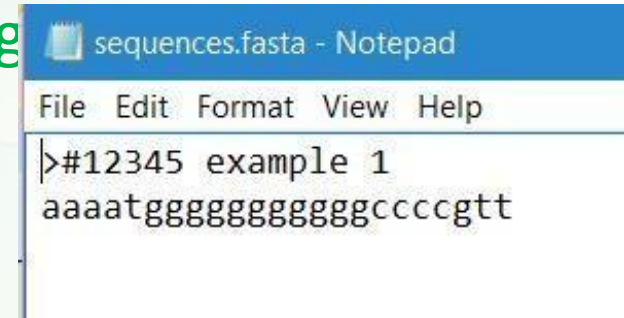
Note that > in the -file argument. This character indicates that we're going to write to the file named sequence.fasta, the same character we'd use if we were using Perl's open() function to write to a file. The -format argument, fasta, tells the object that it should create the file in fasta format.

Writing a sequence to a file

Example 3

```
use Bio::Seq; use Bio::SeqIO;
```

```
$seq_obj = Bio::Seq->new(-seq => "aaaatggggggggggggg  
-display_id => "#12345",  
-desc => "example 1",  
-alphabet => "dna" );
```



```
sequences.fasta - Notepad  
File Edit Format View Help  
>#12345 example 1  
aaaatgggggggggggggccccgtt
```

```
$seqio_obj = Bio::SeqIO->new(-file =>  
'>C:\\Users\\LINO\\Desktop\\sequence.fasta', - format => 'fasta' );  
$seqio_obj->write_seq($seq_obj);
```


Retrieving a sequence from a file

Example

```
use Bio::SeqIO;
```

```
$seqio_obj = Bio::SeqIO->new(-file => "  
C:\\Users\\LINO\\Desktop\\abc\\ seq.fasta", -format  
=> "fasta" );
```

```
$seq_obj = $seqio_obj->next_seq;
```

```
print $seq_obj->seq,"\n";
```

```
C:\Users\Nixon Mendez\Documents\Books\M.Tech\Semester 2\Bioperl>perl example4.pl  
MADGQLPFPCSYPSRL-RRDPFRDSPLSSRLDDGFGMDPFPDDL TAPWPEWALPRLSS----AWPGTLRSGMVPRGPTA  
TARFG-VPAEG----RNPPFPGE PWKVCNVHSEFKPEELMVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPAEVD  
PVTVFASLSPEGLLIEAPQVPPYSPFG-----ESSFNNELPQDNQEVTCSE-----
```

Retrieving multiple sequence from a file

Example 5:

```
use Bio::SeqIO;
```

```
$seqio_obj = Bio::SeqIO->new(-file => "  
C:\\Users\\LINO\\Desktop\\abc\\sequence.fasta", -format  
=> "fasta" );
```

```
while ($seq_obj = $seqio_obj->next_seq)  
{  
    print $seq_obj->seq, "\n";  
}
```

```
C:\Users\Nixon Mendez\Documents\Books\M.Tech\Semester 2\Bioperl>perl example5.pl  
MADGQLPFPCSYPSRL-RRDPFRDSPLSRLLDDGFGMDPFPDDLAPWPEWALPRLSS----AWPGTLRSGMVPRGPTA  
TARFG-VPAEG----RNPPFPFGEPPWKVCNVVHSFKPEELMVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPAEVD  
PVTVFASLSPEGLLIEAPQVPPYSPFG-----ESSFNNELPQDNQEVTCSE-----  
MADGQLPFPCSYPSRL-RRDPFRDSPLSRLLDDGFGMDPFPDDLAPWPEWALPRLSS----AWPGTLRSGMVPRGPPA  
TARFG-VPAEG----RSPFPFGEPPWKVCNVVHSFKPEELMVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPAEVD  
PATVFASLSPEGLLIEAPQVPPYSPFG-----ESSFNNELPQDNQEVTCSE-----  
MADGQMPFSCHYPSRL-RRDPFRDSPLSRLLDDGFGMDPFPDDL TASWPDWALPRLSS----AWPGTLRSGMVPRGPTA  
TARFG-VPAEG----RTPFPFGEPPWKVCNVVHSFKPEELMVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPAEVD  
PVTVFASLSPEGLLIEAPQVPPYSTFG-----ESSFNNELPQDSQEVTCSE-----  
MADGQMPFPCHYTSRR-RRDPFRDSPLSRLLDDGFGMDPFPDDL TASWPDWALPRLSS----AWPGTLRSGMVPRGPTA  
MTRFG-VPAEG----RSPFPFGEPPWKVCNVVHSFKPEELMVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPAEVD  
PVTVFASLSPEGLLIEAPQVPPYSPFG-----ESSFNNELPQDGQEVTCSE-----  
MADSQLPFSCHYPGRRSLRDPFREPGLSRLLDDDFGLSPFPGLD TADWPDWARPLTP----TWPGPLRARASAMAPGY  
STRFG-GYPES----RSPAPTSREPWKVCNVVHSFKPEELTVKTKDGYVEVSGKHEEKQQEGGIVSKNFTKKIQLPYEVD  
PITVFASLSPEGLLIEAPQIPPYQQYG-----EGGCSGEIPLESPEATCA-----  
MAEGDY-YTLGNRQRI-PRDPFREQSLASRFMDDDFGLPPFPNELSMDWPGWARPRLSHRLDAPWTGSLRSGFPRASMSS  
PQGFSSVYTESPRRASAPPTSDPEPPWKVCNVVHSFKPEELNVKTKDGFVEVSGKHEEKQDEGGIVTKNFTKKIQIPLDVO  
PVTVFASLSPEGLVLIIEARQTPPYLYSNEMPAESMEEPEARPEPSMTSTTTFEARE
```

Format the sequences

SeqIO object can read a stream of sequences in one format: Fasta, EMBL, GenBank, Swissprot, PIR, GCG, SCF, phd/phred, Ace, or raw (plain sequence), then write to another file in another format

Ex:

```
use Bio::SeqIO;
```

```
$in = Bio::SeqIO->new('-file' =>
```

```
"C:\\Users\\LINO\\Desktop\\abc\\sequence.fasta",  
                        '-format' => 'Fasta');
```

```
$out = Bio::SeqIO->new('-file' =>
```

```
">C:\\Users\\LINO\\Desktop\\abc\\seq123.embl", '-format' =>  
'EMBL');
```

```
while ( my $seq = $in->next_seq() )
```

```
{ $out->write_seq($seq); }
```

Manipulating sequence data

`$seqobj->trunc(5,10)` # truncation from 5 to 10 as new object

`$seqobj->revcom` # reverse complements sequence

`$seqobj->translate` # translation of the sequence

revcom() and translate() Example

```
use Bio::Seq;  
use Bio::SeqIO;  
$seqio_obj = Bio::SeqIO->new(-file =>  
"C:\\Users\\LINO\\Desktop\\abc\\seq.fasta", -format=> "fasta" );  
  
$seq_obj = $seqio_obj->next_seq;  
print $seq_obj->seq, "\n";  
$trans = $seq_obj->translate;  
print "\n", $trans->seq;  
$trans = $seq_obj->revcom;  
print "\n", $trans->seq;
```

trunc() Example

```
use Bio::Seq;  
use Bio::SeqIO;  
$seqio_obj = Bio::SeqIO->new(-file =>  
"C:\\Users\\LINO\\Desktop\\abc\\seq.fasta", -format=> "fasta" );  
$seq_obj = $seqio_obj->next_seq;  
print $seq_obj->seq, "\n";  
$trans = $seq_obj->trunc(10,14);  
print "\n", $trans->seq;
```

AlignIO

- You may already be familiar with the Bio.SeqIO module which deals with files containing one or more sequences. The purpose of the SeqIO module is to provide a simple uniform interface to assorted sequence file formats.
- Similarly, Bio.AlignIO deals with files containing one or more sequence alignments represented as Alignment objects.

It allows the user to:

- convert between alignment formats
- extracting specific regions of the alignment
- generating consensus sequences.

Example:

```
use Bio::AlignIO;
```

```
my $io = Bio::AlignIO->new(  
    -file => "receptors.aln",  
    -format => "clustalw" );
```


Bio::AlignIO->new()

1. -file

A file path to be opened for reading or writing. The usual Perl conventions apply:

'file' # open file for reading

'>file' # open file for writing

'>>file' # open file for appending

'+<file' # open file read/write

2. -format

Specify the format of the file.

SimpleAlign module

Description:

It handles multiple alignments of sequences

Method:

new

Usage : my \$aln = new Bio::SimpleAlign();

Function : Creates a new simple align object

Returns : Bio::SimpleAlign

Args : -source => string representing the source program where this alignment came from

next_seq

Usage : foreach \$seq (\$align->each_seq())

Function : Gets an array of Seq objects from the alignment

Returns : an array

length()

Usage : \$len = \$ali->length()

Function : Returns the maximum length of the alignment.

percentage_identity

Usage : \$id = \$align->percentage_identity

Function: The function calculates the average percentage identity. Percent identity refers to a quantitative measurement of the similarity between two sequences (DNA, amino acid or otherwise).

Returns : The average percentage identity

no_sequences

Usage : \$depth = \$ali->no_sequences

Function : number of sequence in the sequence alignment

Returns : integer

Functions

```
$in->next_aln();
```

```
$out->write_aln($aln);
```

```
$mini_aln = $aln->slice(20,22);
```

```
$new_aln = $aln->remove_columns([20,22]);
```

```
use Bio::AlignIO;
```

```
$out = Bio::AlignIO->new(-file => "C:\\Users\\LINO\\Desktop\\abc\\out.aln.pfam" ,  
    -format => 'pfam');
```

```
$aln = $out->next_aln();
```

```
print $aln->length,"\n";
```

```
print $aln->num_residues,"\n";
```

```
print $aln->no_sequences,"\n";
```

```
print $aln->percentage_identity,"\n";
```

```
$mini_aln = $aln->slice(20,22);
```

```
$new_aln = $aln->remove_columns([20,22]);
```

Read fasta file and write as Alignment file i.e .fasta to .pfam file

```
use Bio::AlignIO;
```

```
$in = Bio::AlignIO->new(-file => "C:\\Users\\LINO\\Desktop\\abc\\seq.fasta",  
                        -format => 'fasta');
```

```
$out = Bio::AlignIO->new(-file =>  
">C:\\Users\\LINO\\Desktop\\abc\\out1.aln.pfam" ,  
                        -format => 'pfam');
```

```
while ( my $aln = $in->next_aln() ) {  
    $out->write_aln($aln);  
}
```



THANK YOU

Introduction to R Language

Ms. Sermarani Nadar
Asst. Prof
G. N. Khalsa College

What is R

- R is a popular programming language used for statistical computing and graphical presentation.
- Its most common use is to analyze and visualize data.

Why Use R?

- It is a great resource for **data analysis, data visualization, data science and machine learning**
- It provides many **statistical techniques** (such as statistical tests, classification, clustering and data reduction)
- It is **easy to draw graphs** in R, like pie charts, histograms, box plot, scatter plot, etc
- It works on **different platforms** (Windows, Mac, Linux)
- It is **open-source and free**
- It has many packages (libraries of functions) that can be used to solve different problems

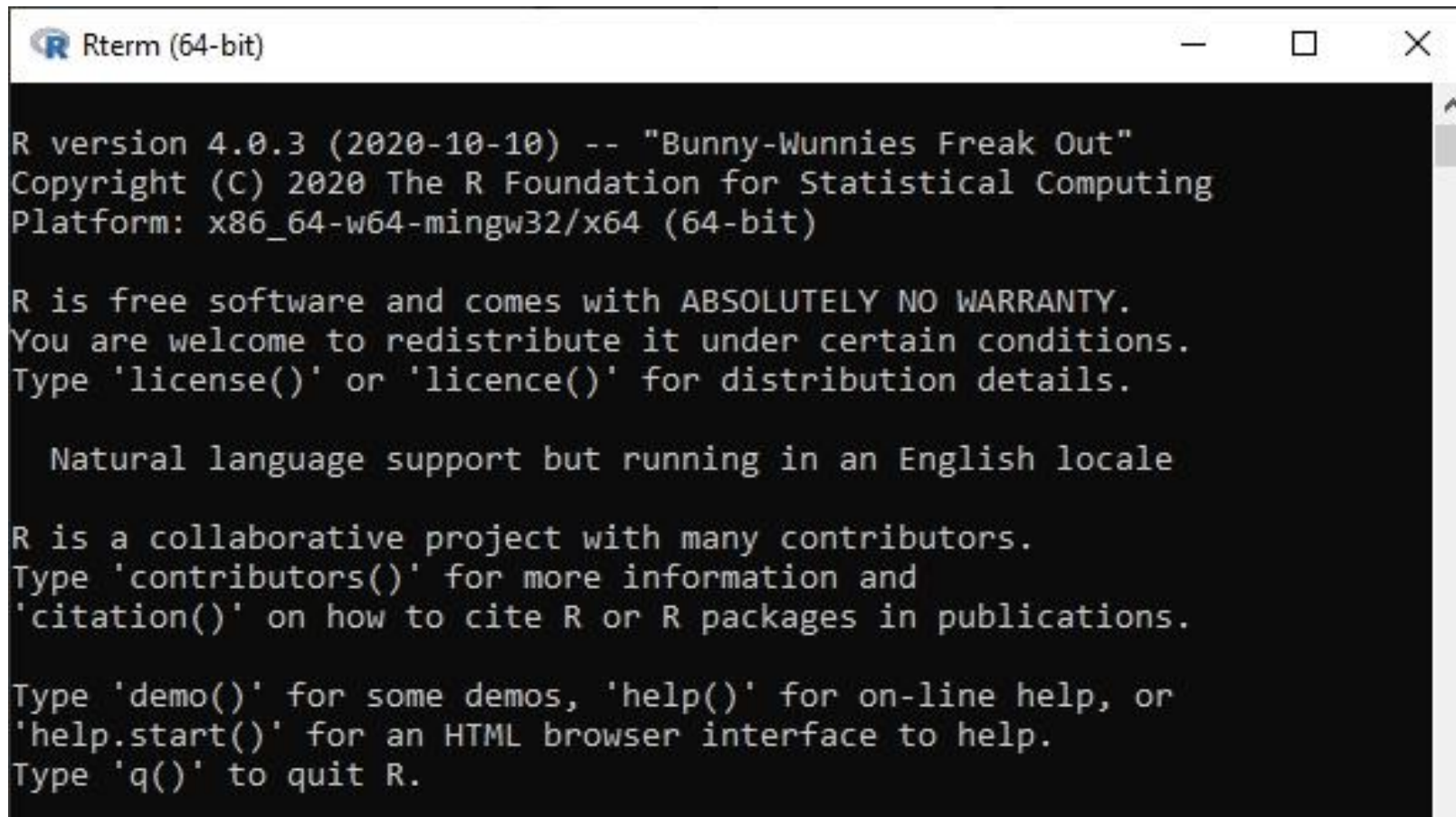
How to Install R

To install R, go to <https://cloud.r-project.org/> and download the latest version of R for Windows(), Mac or Linux.

Rstudio: <https://posit.co/download/rstudio-desktop/>

When you have downloaded and installed R, you can run R on your computer.

The screenshot below shows how it may look like when you run R on a Windows PC:

A screenshot of a Windows command prompt window titled "Rterm (64-bit)". The window has a black background with white text. The text displays the R version (4.0.3), copyright information (2020 The R Foundation for Statistical Computing), and platform details (x86_64-w64-mingw32/x64 (64-bit)). It also includes a disclaimer about the software being free and without warranty, and provides instructions on how to use various R functions like license(), contributors(), citation(), demo(), help(), and q().

```
R version 4.0.3 (2020-10-10) -- "Bunny-Wunnies Freak Out"
Copyright (C) 2020 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function

Addins

RunSource

Untitled1

1

1:1 (Top Level) R Script

EnvironmentHistoryConnectionsTutorial

Import Dataset96 MIB

RGlobal Environment

Environment is empty

FilesPlotsPackagesHelpViewerPresentation

New FolderNew FileDeleteRenameMore

Home

	Name	Size	Modified
☐	!qhlogs.doc	22 KB	Feb 8, 2023, 8:12 AM
☐	Adobe		
☐	Blackmagic Design		
☐	desktop.ini	402 B	Mar 3, 2024, 6:17 PM
☐	My Music		
☐	My Pictures		
☐	My Videos		
☐	OneNote Notebooks		

ConsoleTerminalBackground Jobs

R 4.4.2 ~/

R version 4.4.2 (2024-10-31 ucrt) -- "Pile of Leaves"
Copyright (c) 2024 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |

R - Data Types

- Vectors
- Lists
- Matrices
- Arrays
- Factors
- Data Frames

Vectors:

A vector(1 Dim) is a data structure that stores a collection of elements in an ordered list. Vectors are similar to arrays, but they can automatically resize themselves.

Data Type Example

Logical : TRUE, FALSE

```
v <- TRUE
```

```
print(class(v))
```

it produces the following result –
[1] "logical"

Numeric: 12.3, 5, 999

```
v <- 23.5
```

```
print(class(v))
```

it produces the following result –
[1] "numeric"

Integer: 2L, 34L, 0L

```
v <- 2L
```

```
print(class(v))
```

it produces the following result –
[1] "integer"

Complex: 3 + 2i

```
v <- 2+5i
```

```
print(class(v))
```

it produces the following result –
[1] "complex"

Character: 'a' , '"good", "TRUE", '23.4'

```
v <- "TRUE"
```

```
print(class(v))
```

it produces the following result –
[1] "character"

- When you want to create vector with more than one element, you should use `c()` function which means to combine the elements into a vector.

Create a vector.

```
seq <- c('attg','aggt','atgc')
```

```
print(seq)
```

Get the class of the vector.

```
print(class(seq))
```

When we execute the above code, it produces the following result –

```
[1] "attg"  "aggt"  "atgc"
```

```
[1] "character"
```


Lists

- A list is an R-object which can **contain many different types of elements** inside it like vectors, functions and even another list inside it.

Create a list.

```
list1 <- list(c(2,5,3),21.3,sin)
```

Print the list.

```
print(list1)
```

When we execute the above code, it produces the following result –

```
[[1]]
```

```
[1] 2 5 3
```

```
[[2]]
```

```
[1] 21.3
```

```
[[3]]
```

```
function (x) .Primitive("sin")
```

Matrices

- A matrix is a **two-dimensional rectangular data set**. It can be created using a vector input to the matrix function.

```
# Create a matrix.
```

```
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow = TRUE)
```

```
print(M)
```

- When we execute the above code, it produces the following result –

```
 [,1] [,2] [,3]  
[1,] "a"  "a"  "b"  
[2,] "c"  "b"  "a"
```

Arrays

- While matrices are confined to two dimensions, **arrays can be of any number of dimensions**. The array function takes a **dim attribute** which creates the required number of dimension. In the below example we create an array with two elements which are 3x3 matrices each.

Create an array.

```
a <- array(c('green','yellow'),dim = c(3,3,2))
```

```
print(a)
```

- When we execute the above code, it produces the following result –

```
, , 1
```

```
  [,1] [,2] [,3]
```

```
[1,] "green" "yellow" "green"
```

```
[2,] "yellow" "green" "yellow"
```

```
[3,] "green" "yellow" "green"
```

```
, , 2
```

```
  [,1] [,2] [,3]
```

```
[1,] "yellow" "green" "yellow"
```

```
[2,] "green" "yellow" "green"
```

```
[3,] "yellow" "green" "yellow"
```

Factors

- Factors are the r-objects which are created using a vector. It stores the vector along with the **distinct values of the elements in the vector as labels**. The **labels are always character** irrespective of whether it is numeric or character or Boolean etc. in the input vector. They are useful in statistical modeling.
- Factors are created using the **factor() function**. The **nlevels functions** gives the count of levels.

Create a vector.

```
dna_seq <- c('attg',' attg ','ataa','atgc','atgc','atgc',' attg ')
```

Create a factor object.

```
factor_seq <- factor(dna_seq )
```

Print the factor.

```
print(factor_seq)
```

```
print(nlevels(factor_seq))
```

When we execute the above code, it produces the following result –

```
[1] attg attg  ataa atgc atgc atgc attg
```

```
Levels: attg atgc ataa
```

```
[1] 3
```

Data Frames

- Data frames are **tabular data objects**. Unlike a matrix in data frame each column can contain different modes of data. The first column can be numeric while the second column can be character and third column can be logical. It is a **list of vectors of equal length**.
- Data Frames are created using the **data.frame()** function.

Create the data frame.

```
BMI <- data.frame(  
  gender = c("Male", "Male", "Female"),  
  height = c(152, 171.5, 165),  
  weight = c(81, 93, 78),  
  Age = c(42, 38, 26)  
)  
print(BMI)
```

When we execute the above code, it produces the following result –

	gender	height	weight	Age
1	Male	152.0	81	42
2	Male	171.5	93	38
3	Female	165.0	78	26

summary(BMI)

BMI[1]

BMI[["gender"]]

BMI\$Training

Add a new row

New_row_DF <- rbind(BMI, c("Female", 170.0, 50, 23))

Print the new row

New_row_DF

Add a new column

New_col_DF <- cbind(BMI, Name = c("Ramu", "Janu", "Gauri"))

Print the new column

New_col_DF

```
> summary(BMI)
   gender
Length:3
Class :character
Mode  :character
```

```
   height      weight      Age
Min.   :152.0   Min.   :78.0   Min.   :26.00
1st Qu.:158.5   1st Qu.:79.5   1st Qu.:32.00
Median :165.0   Median :81.0   Median :38.00
Mean   :162.8   Mean   :84.0   Mean   :35.33
3rd Qu.:168.2   3rd Qu.:87.0   3rd Qu.:40.00
Max.   :171.5   Max.   :93.0   Max.   :42.00
```

```
< |
```

```
> BMI[1]
```

```
gender
```

```
1    Male
```

```
2    Male
```

```
3 Female
```

```
> BMI[["gender"]]
```

```
[1] "Male"  "Male"  "Female"
```

```
>
```

Types of Operators

We have the following types of operators in R programming –

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Miscellaneous Operators

Arithmetic Operators		
Operator	Description	Example
+	Adds two vectors	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v+t) it produces the following result – [1] 10.0 8.5 10.0
–	Subtracts second vector from the first	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v-t) it produces the following result – [1] -6.0 2.5 2.0
*	Multiplies both vectors	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v*t) it produces the following result – [1] 16.0 16.5 24.0
/	Divide the first vector with the second	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v/t) When we execute the above code, it produces the following result – [1] 0.250000 1.833333 1.500000
%%	Give the remainder of the first vector with the second	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v%%t) it produces the following result – [1] 2.0 2.5 2.0
%/%	The result of division of first vector with second (quotient)	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v%/%t) it produces the following result – [1] 0 1 1
^	The first vector raised to the exponent of second vector	v <- c(2,5.5,6) t <- c(8, 3, 4) print(v^t) it produces the following result – [1] 256.000 166.375 1296.000

Relational Operators

Operator	Description	Example
>	Checks if each element of the first vector is greater than the corresponding element of the second vector.	v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>t) it produces the following result – [1] FALSE TRUE FALSE FALSE
<	Checks if each element of the first vector is less than the corresponding element of the second vector.	v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v < t) it produces the following result – [1] TRUE FALSE TRUE FALSE
==	Checks if each element of the first vector is equal to the corresponding element of the second vector.	v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v == t) it produces the following result – [1] FALSE FALSE FALSE TRUE
<=	Checks if each element of the first vector is less than or equal to the corresponding element of the second vector.	v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v<=t) it produces the following result – [1] TRUE FALSE TRUE TRUE
>=	Checks if each element of the first vector is greater than or equal to the corresponding element of the second vector.	v <- c(2,5.5,6,9) t <- c(8,2.5,14,9) print(v>=t) it produces the following result – [1] FALSE TRUE FALSE TRUE
		v <- c(2 5 5 6 9) t <-

Logical Operators

&	It is called Element-wise Logical AND operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if both the elements are TRUE.	<pre>v <- c(3,1,TRUE,2+3i) t <- c(4,1,FALSE,2+3i) print(v&t) it produces the following result – [1] TRUE TRUE FALSE TRUE</pre>
	It is called Element-wise Logical OR operator. It combines each element of the first vector with the corresponding element of the second vector and gives a output TRUE if one the elements is TRUE.	<pre>v <- c(3,0,TRUE,2+2i) t <- c(4,0,FALSE,2+3i) print(v t) it produces the following result – [1] TRUE FALSE TRUE TRUE</pre>
!	It is called Logical NOT operator. Takes each element of the vector and gives the opposite logical value.	<pre>v <- c(3,0,TRUE,2+2i) print(!v) it produces the following result – [1] FALSE TRUE FALSE FALSE</pre>

Miscellaneous Operators

These operators are used to for specific purpose and not general mathematical or logical computation.

:	Colon operator. It creates the series of numbers in sequence for a vector.	<pre>v <- 2:8 print(v) it produces the following result – [1] 2 3 4 5 6 7 8</pre>
%in%	This operator is used to identify if an element belongs to a vector.	<pre>v1 <- 8 v2 <- 12 t <- 1:10 print(v1 %in% t) print(v2 %in% t) it produces the following result – [1] TRUE [1] FALSE</pre>

Conditions and If Statements

If else

```
a <- 33
```

```
b <- 33
```

```
if (b > a) {  
  print("b is greater than a")  
} else if (a == b) {  
  print ("a and b are equal")  
}
```

AND(&)

```
a <- 200
```

```
b <- 33
```

```
c <- 500
```

```
if (a > b & c > a) {  
  print("Both conditions are true")  
}
```

OR(|)

```
a <- 200
```

```
b <- 33
```

```
c <- 500
```

```
if (a > b | a > c) {  
  print("At least one of the conditions is  
true")  
}
```

Loops

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable.

R has two loop commands:

while loops

for loops

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
  if (i == 4) {
    break
  }
}
```

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

```
fruits <- list("apple", "banana", "cherry")
```

```
for (x in fruits) {  
  print(x)  
}
```

R Functions

A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

Creating a Function

To create a function, use the `function()` keyword:

Example

```
my_function <- function() { # create a function with the name my_function
  print("Hello World!")
}
```

```
my_function() # call the function named
my_function
```

R Statistics

Statistics is the science of analyzing, reviewing and conclude data.

Some basic statistical numbers include:

- Mean, median and mode
- Minimum and maximum value
- Percentiles
- Variance and Standard Deviation
- Covariance and Correlation

Data Set

A data set is a collection of data, often presented in a table.

There is a popular built-in data set in R called "**mtcars**" (Motor Trend Car Road Tests), which is retrieved from the 1974 Motor Trend US Magazine.

We will use the mtcars data set, for statistical purposes:

```
# Print the mtcars data set  
mtcars
```

Information About the Data Set

You can use the question mark (?) to get information about the mtcars data set:

```
# Use the question mark to get information about the data set
```

```
?mtcars
```

```
> mtcars
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

```
> |
```

Description

The data was extracted from the 1974 *Motor Trend* US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models).

Usage

`mtcars`

Format

A data frame with 32 observations on 11 (numeric) variables.

```
[, 1] mpg  Miles/(US) gallon
[, 2] cyl  Number of cylinders
[, 3] disp Displacement (cu.in.)
[, 4] hp   Gross horsepower
[, 5] drat Rear axle ratio
[, 6] wt   Weight (1000 lbs)
[, 7] qsec 1/4 mile time
[, 8] vs   Engine (0 = V-shaped, 1 = straight)
[, 9] am   Transmission (0 = automatic, 1 = manual)
[,10] gear Number of forward gears
[,11] carb Number of carburetors
```

Note

Jenderson and Velleman (1981) comment in a footnote to Table 1: ‘Hocking [original transcriber]’s noncrucial coding of the Mazda’s rotary engine as a straight six-cylinder engine and the Porsche’s flat engine as a V engine, as well as the inclusion of the diesel Mercedes 240D, have been retained to enable direct comparisons to be made with previous analyses.’

Source

Jenderson and Velleman (1981), Building multiple regression models interactively. *Biometrics*, 37, 391–411.

Examples

[Run examples](#)

```
require(graphics)
pairs(mtcars, main = "mtcars data", gap = 1/4)
plot(mpg ~ disp | as.factor(cyl), data = mtcars,
     panel = panel.smooth, rows = 1)
# possibly more meaningful, e.g., for summary() or bivariate plots:
mtcars2 <- within(mtcars, {
  vs <- factor(vs, labels = c("V", "S"))
  am <- factor(am, labels = c("automatic", "manual"))
  cyl <- ordered(cyl)
```

Get Information

Use the **dim() function** to find the dimensions of the data set, and the **names() function** to view the names of the variables:

Example

```
Data_Cars <- mtcars # create a variable of the mtcars data set for better organization
```

```
# Use dim() to find the dimension of the data set  
dim(Data_Cars)
```

```
# Use names() to find the names of the variables from the data set  
names(Data_Cars)
```

Sort Variable Values

To sort the values, use the `sort()` function:

```
Data_Cars <- mtcars  
sort(Data_Cars$cyl)
```

Analyzing the Data

Now that we have some information about the data set, we can start to analyze it with some statistical numbers.

For example, we can use the **`summary()`** function to get a statistical summary of the data:

Example

```
Data_Cars <- mtcars  
  
summary(Data_Cars)
```

The `summary()` function returns six statistical numbers for each variable:

- Min
- First quantile (percentile)
- Median
- Mean
- Third quantile (percentile)
- Max

```
> summary(Data_Cars)
```

mpg	cyl	disp	hp	drat	wt	qsec
Min. :10.40	Min. :4.000	Min. : 71.1	Min. : 52.0	Min. :2.760	Min. :1.513	Min. :14.50
1st Qu.:15.43	1st Qu.:4.000	1st Qu.:120.8	1st Qu.: 96.5	1st Qu.:3.080	1st Qu.:2.581	1st Qu.:16.89
Median :19.20	Median :6.000	Median :196.3	Median :123.0	Median :3.695	Median :3.325	Median :17.71
Mean :20.09	Mean :6.188	Mean :230.7	Mean :146.7	Mean :3.597	Mean :3.217	Mean :17.85
3rd Qu.:22.80	3rd Qu.:8.000	3rd Qu.:326.0	3rd Qu.:180.0	3rd Qu.:3.920	3rd Qu.:3.610	3rd Qu.:18.90
Max. :33.90	Max. :8.000	Max. :472.0	Max. :335.0	Max. :4.930	Max. :5.424	Max. :22.90

vs	am	gear	carb
Min. :0.0000	Min. :0.0000	Min. :3.000	Min. :1.000
1st Qu.:0.0000	1st Qu.:0.0000	1st Qu.:3.000	1st Qu.:2.000
Median :0.0000	Median :0.0000	Median :4.000	Median :2.000
Mean :0.4375	Mean :0.4062	Mean :3.688	Mean :2.812
3rd Qu.:1.0000	3rd Qu.:1.0000	3rd Qu.:4.000	3rd Qu.:4.000
Max. :1.0000	Max. :1.0000	Max. :5.000	Max. :8.000

```
>
```

Mean

It is calculated by taking the sum of the values and dividing with the number of values in a data series.

The function **mean()** is used to calculate this in R.

Syntax

The basic syntax for calculating mean in R is –

```
mean(x, trim = 0, na.rm = FALSE)
```

- **x** is the input vector.
- **trim** is used to drop some observations from both end of the sorted vector.
- **na.rm** is used to remove the missing values from the input vector.

To drop the missing values from the calculation use na.rm = TRUE. which means remove the NA values.

```
# Create a vector.
```

```
x <- c(12,7,3,4.2,18,2,54,-21,8,-5,NA)
```

```
# Find mean.
```

```
result.mean <- mean(x)
```

```
print(result.mean)
```

```
# Find mean dropping NA values.
```

```
result.mean <- mean(x,na.rm = TRUE)
```

```
print(result.mean)
```

When we execute the above code, it produces the following result –

```
[1] NA
```

```
[1] 8.22
```


Median

The middle most value in a data series is called the median. The median() function is used in R to calculate this value.

Syntax

The basic syntax for calculating median in R is –

`median(x, na.rm = FALSE)`

Following is the description of the parameters used –

`x` is the input vector.

`na.rm` is used to remove the missing values from the input vector.

Example

```
# Create the vector.  
x <- c(12,7,3,4.2,18,2,54,-21,8,-5)
```

```
# Find the median.  
median.result <- median(x)  
print(median.result)
```

When we execute the above code, it produces the following result –

```
[1] 5.6
```

Mode

The mode is the value that has highest number of occurrences in a set of data. Unlike mean and median, mode can have both numeric and character data.

Create the function.

```
getmode <- function(v) {  
  uniqv <- unique(v)  
  uniqv[which.max(tabulate(match(v,  
  uniqv)))]  
}
```

Create the vector with numbers.

```
v <- c(2,1,2,3,1,2,3,4,1,5,5,3,2,3)
```

Calculate the mode using the user function.

```
result <- getmode(v)  
print(result)
```

Create the vector with characters.

```
charv <- c("o","it","the","it","it")
```

Calculate the mode using the user function.

```
result <- getmode(charv)  
print(result)
```

When we execute the above code, it produces the following result –

```
[1] 2
```

```
[1] "it"
```

Find the largest and smallest value of the variable hp (horsepower).

```
Data_Cars <- mtcars
```

```
max(Data_Cars$hp)
```

```
min(Data_Cars$hp)
```

How to Import a CSV File into R ?

A CSV file is used to store contents in a tabular-like format, which is organized in the form of rows and columns. The column values in each row are separated by a delimiter string. The CSV files can be loaded into the working space and worked using both in-built methods and external package imports.

Method 1: Using read.csv() method

The read.csv() method in base R is used to load a .csv file into the present script and work with it. The contents of the csv can be stored into the variable and further manipulated. Multiple files can also be accessed in different variables. The output is returned to the form of a data frame, where row numbers are assigned, integers beginning with 1.

Syntax:

```
read.csv(path, header = TRUE, sep = “,”)
```

Arguments :

path : The path of the file to be imported

header : By default : TRUE . Indicator of whether to import column headings.

sep = “,” : The separator for the values in each row.

```
# specifying the path
path <- "/Users/mallikagupta/Desktop/gfg.csv"
```

```
# reading contents of csv file
content <- read.csv(path)
# contents of the csv file
print (content)
```

Output:

	ID	Name	Post	Age
1	5	H	CA	67
2	6	K	SDE	39
3	7	Z	Admin	28

In case, the header is set to FALSE, the column names are ignored, and default variables names are displayed for each column beginning from V1.

THANK YOU

Subsetting in R Programming

Subsetting in R Using [] Operator

Using the '[]' operator, elements of vectors and observations from data frames can be accessed. To neglect some indexes, '-' is used to access all other indexes of vector or data frame.

Ex:

```
x <- 1:15
```

```
# Print vector
```

```
cat("Original vector: ", x, "\n")
```

```
# Subsetting vector
```

```
cat("First 5 values of vector: ", x[1:5], "\n")
```

```
cat("Without values present at index 1, 2 and 3: ",  
    x[-c(1, 2, 3)], "\n")
```

Output:

Original vector: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

First 5 values of vector: 1 2 3 4 5

Without values present at index 1, 2 and 3: 4 5 6 7 8 9 10 11 12
13 14 15

Example :

In this example, let us use mtcars data frame present in R base package for subsetting.

Dataset

```
cat("Original dataset: \n")
```

```
print(mtcars)
```

Subsetting data frame

```
cat("HP values of all cars:\n")
```

```
print(mtcars['hp'])
```

First 10 cars

```
cat("Without mpg and cyl column:\n")
```

```
print(mtcars[1:10, -c(1, 2)])
```


Subsetting in R Using [[]] Operator

[[]] operator is used for subsetting of list-objects. This operator is the same as [] operator but the only difference is that **[[]]** **selects only one element** whereas **[]** operator can select **more than 1 element** in a single command.

```
z <- list(a = list(x = 1, y = "GFG"), b = 1:10)
```

```
# Print list
```

```
cat("Original list:\n")
```

```
print(z)
```

```
# Print GFG using c() function
```

```
cat("Using c() function:\n")
```

```
print(z[[c(1, 2)]])
```

```
# Print GFG using only [[]] operator
```

```
cat("Using [[]] operator:\n")
```

```
print(z[[1]][[2]])
```

Output:

Original list:

```
$a
```

```
$a$x
```

```
[1] 1
```

```
$a$y
```

```
[1] "GFG"
```

```
$b
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Using c() function:

```
[1] "GFG"
```

Using [[]] operator:

```
[1] "GFG"
```

Ordering Factor Levels

Ordered factors levels are an extension of factors. It arranges the levels in increasing order. We use two functions: `factor()` and argument `ordered()`.

Syntax: `factor(data, levels = c(""), ordered = TRUE)`

Parameter:

data: input vector with explicitly defined values.

levels(): Mention the list of levels in `c` function.

ordered: It is set true for enabling ordering.

EXAMPLE:

```
# creating size vector
```

```
size = c("small", "large", "large", "small",  
        "medium", "large", "medium", "medium")
```

```
# converting to factor
```

```
size_factor <- factor(size)
```

```
print(size_factor)
```

```
# ordering the levels
```

```
ordered.size <- factor(size, levels = c(  
  "small", "medium", "large"), ordered = TRUE)
```

```
print(ordered.size)
```

Output:

```
[1] small large large small medium large medium medium  
Levels: large medium small
```

```
[1] small large large small medium large medium medium  
Levels: small < medium < large
```

DATA VISUALIZATION

Ms. Sermarani Nadar
Asst. Prof
G. N. Khalsa College

R Graphics

Plot

- The `plot()` function is used to draw points (markers) in a diagram.
- The function takes parameters for specifying points in the diagram.
- Parameter 1 specifies points on the x-axis.
- Parameter 2 specifies points on the y-axis.

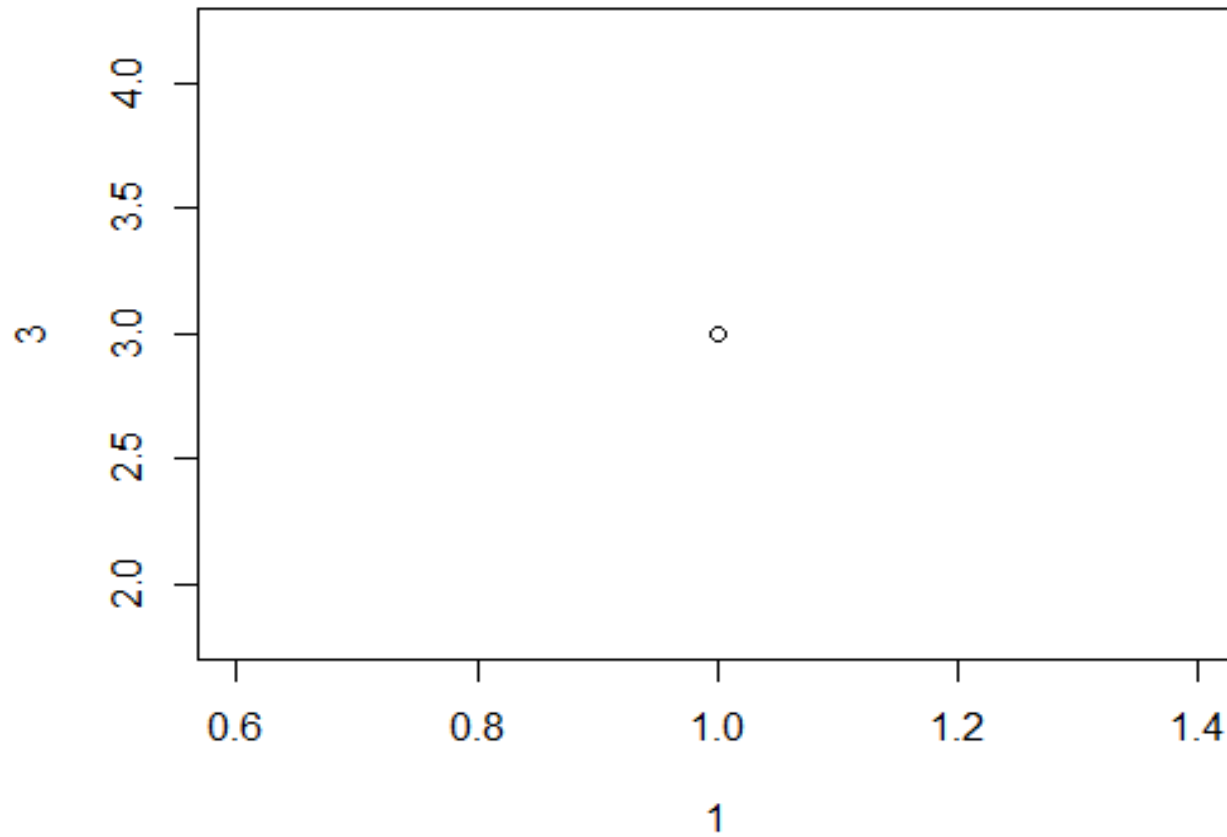
Syntax:

```
plot(x, y)
```

Example

Draw one point in the diagram, at position (1) and position (3):

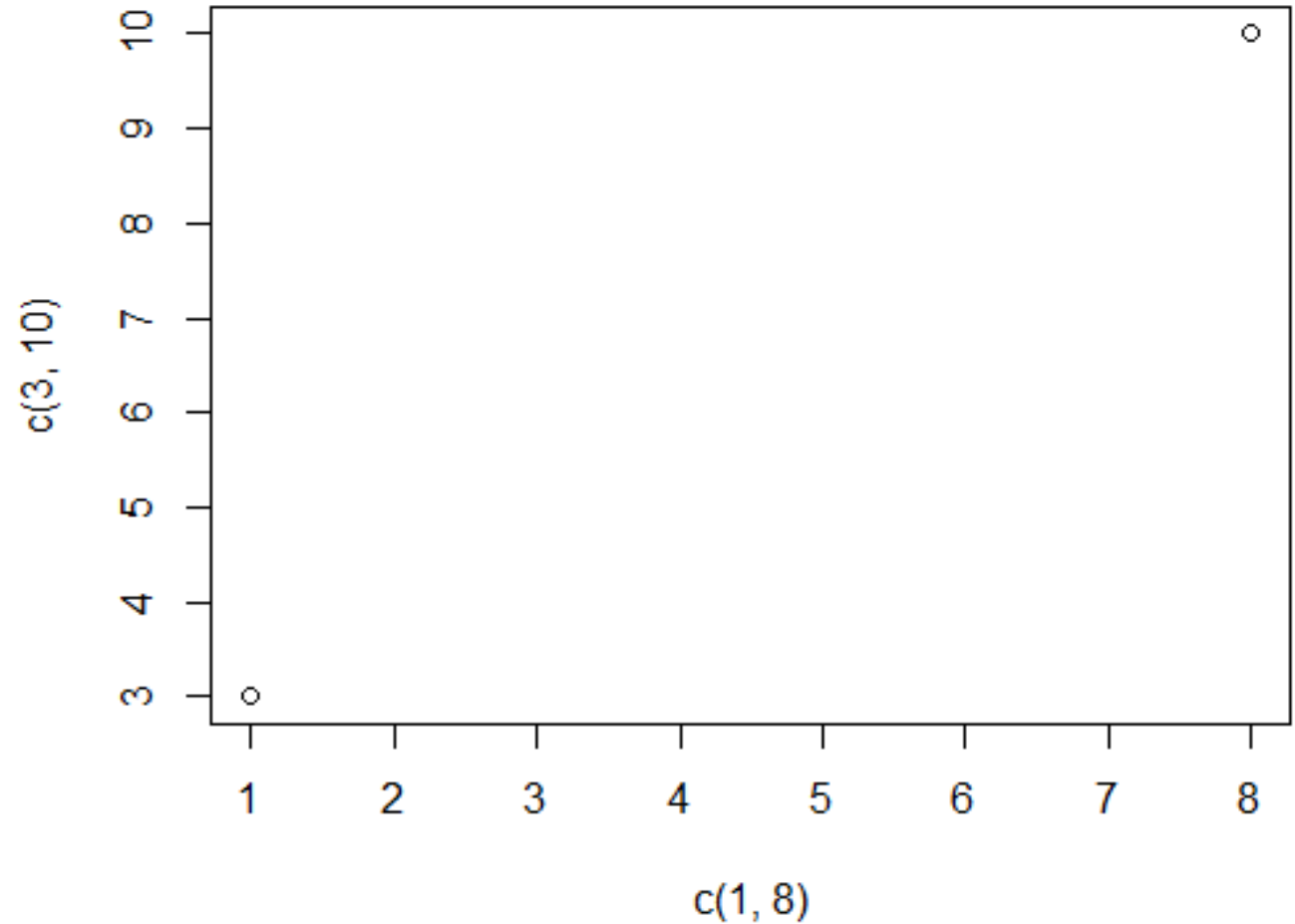
`plot(1, 3)`



Draw more points, use vectors:

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
plot(c(1, 8), c(3, 10))
```



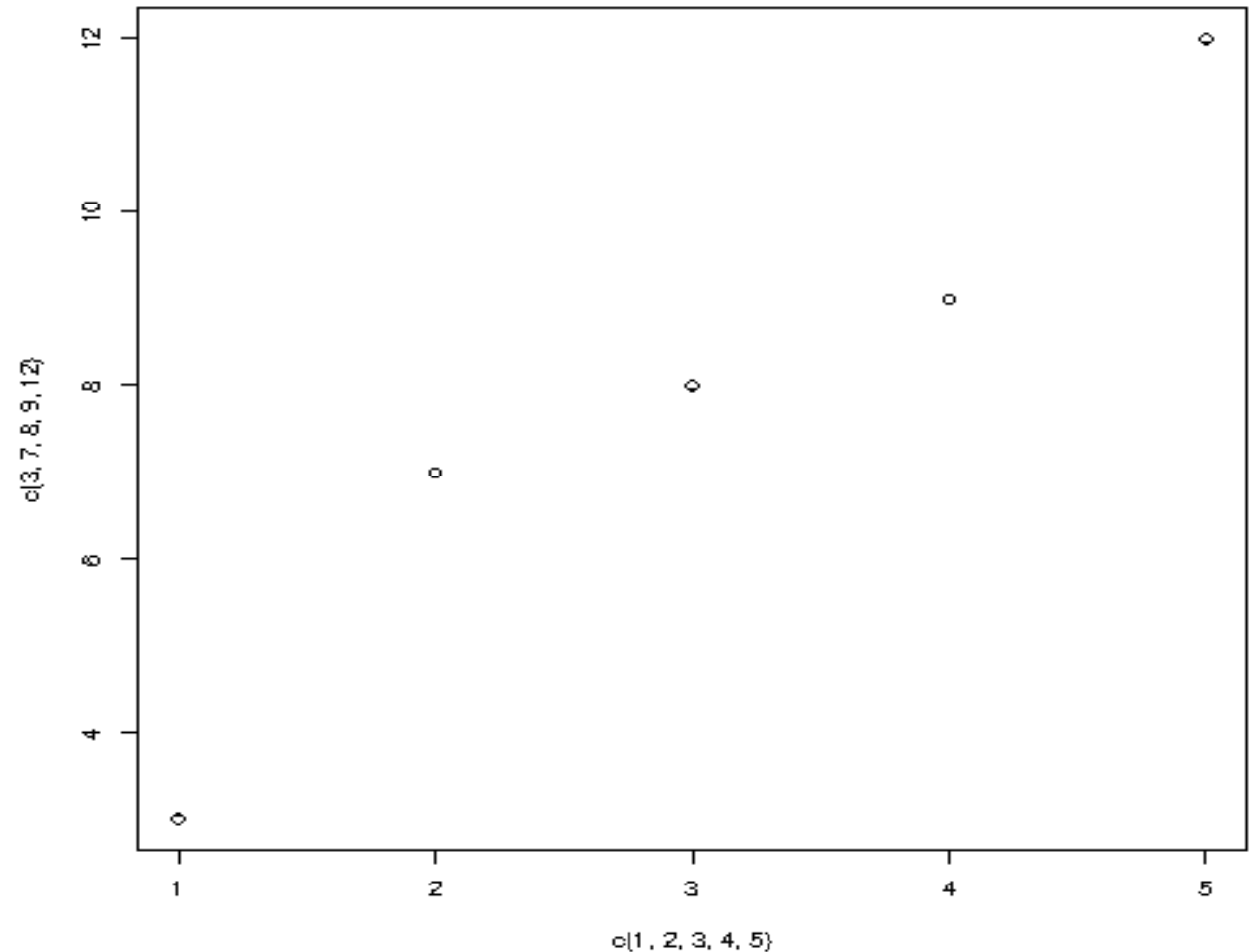
For better organization, when you have many values, it is better to use variables:

Example

```
x <- c(1, 2, 3, 4, 5)
```

```
y <- c(3, 7, 8, 9, 12)
```

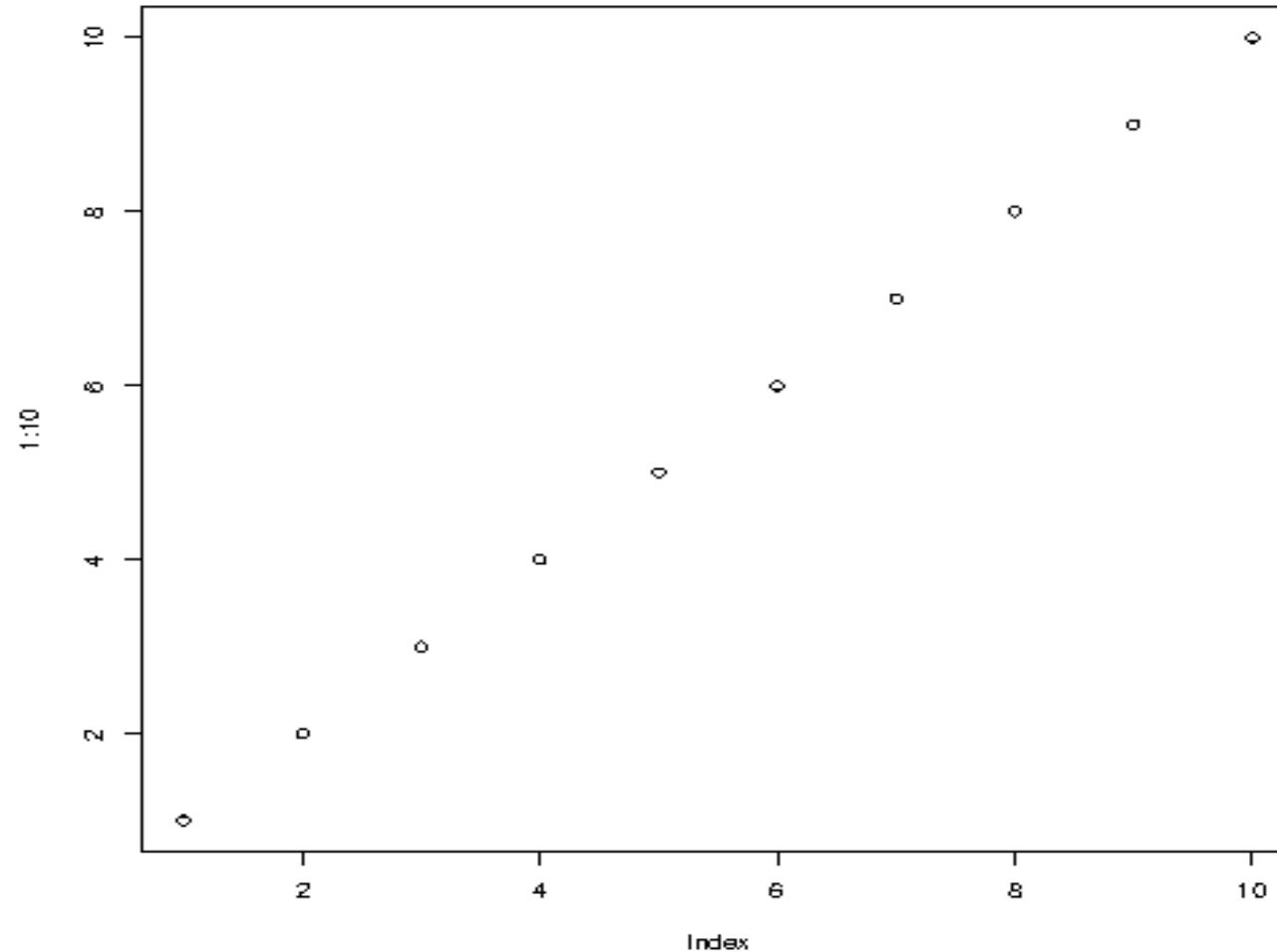
```
plot(x, y)
```



Sequences of Points

If you want to draw dots in a sequence, on both the x-axis and the y-axis, use the `:` operator:

Example
`plot(1:10)`



Useful Methods

- Draw a Line : `plot(1:10, type="l")`
- Plot Labels : `plot(1:10, main="My Graph", xlab="The x-axis", ylab="The y axis")`
- Graph Appearance Colour : `plot(1:10, col="red")`
- Size : `plot(1:10, cex=2)`
- Point Shape: Use `pch` with a value from 0 to 25 to change the point shape format: `plot(1:10, pch=25, cex=2)`
-

R Line

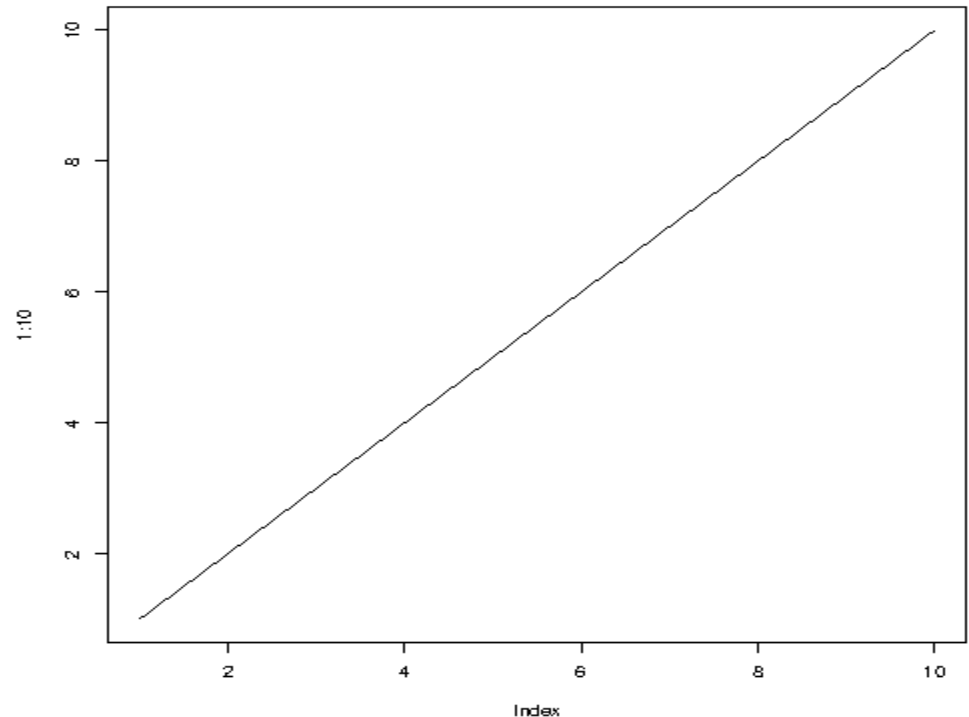
Line Graphs

A line graph has a line that connects all the points in a diagram.

To create a line, use the `plot()` function and add the `type` parameter with a value of `"l"`:

Example

```
plot(1:10, type="l")
```



Multiple Lines

- To display more than one line in a graph, use the `plot()` function together with the `lines()` function:

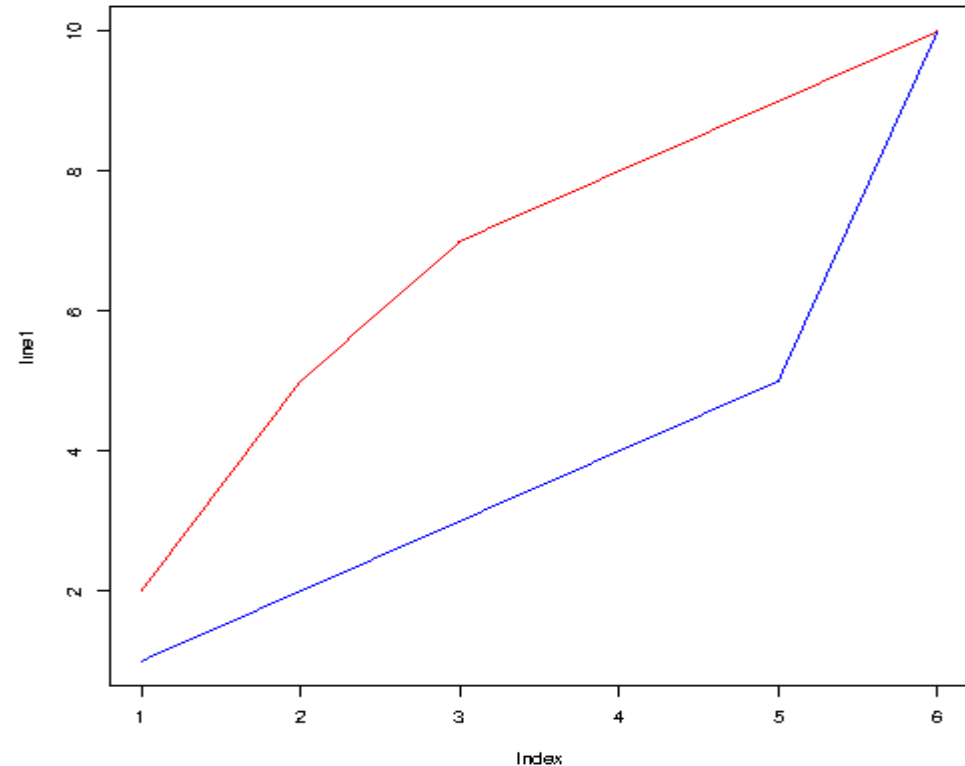
- Example

```
line1 <- c(1,2,3,4,5,10)
```

```
line2 <- c(2,5,7,8,9,10)
```

```
plot(line1, type = "l", col = "blue")
```

```
lines(line2, type="l", col = "red")
```



Useful Methods

- Line Color: `plot(1:10, type="l", col="blue")`
- Line Width : `plot(1:10, type="l", lwd=2)`
- Line Styles: `plot(1:10, type="l", lwd=5, lty=3)`

Available parameter values for `lty`:

0 removes the line

1 displays a solid line

2 displays a dashed line

3 displays a dotted line

4 displays a "dot dashed" line

5 displays a "long dashed" line

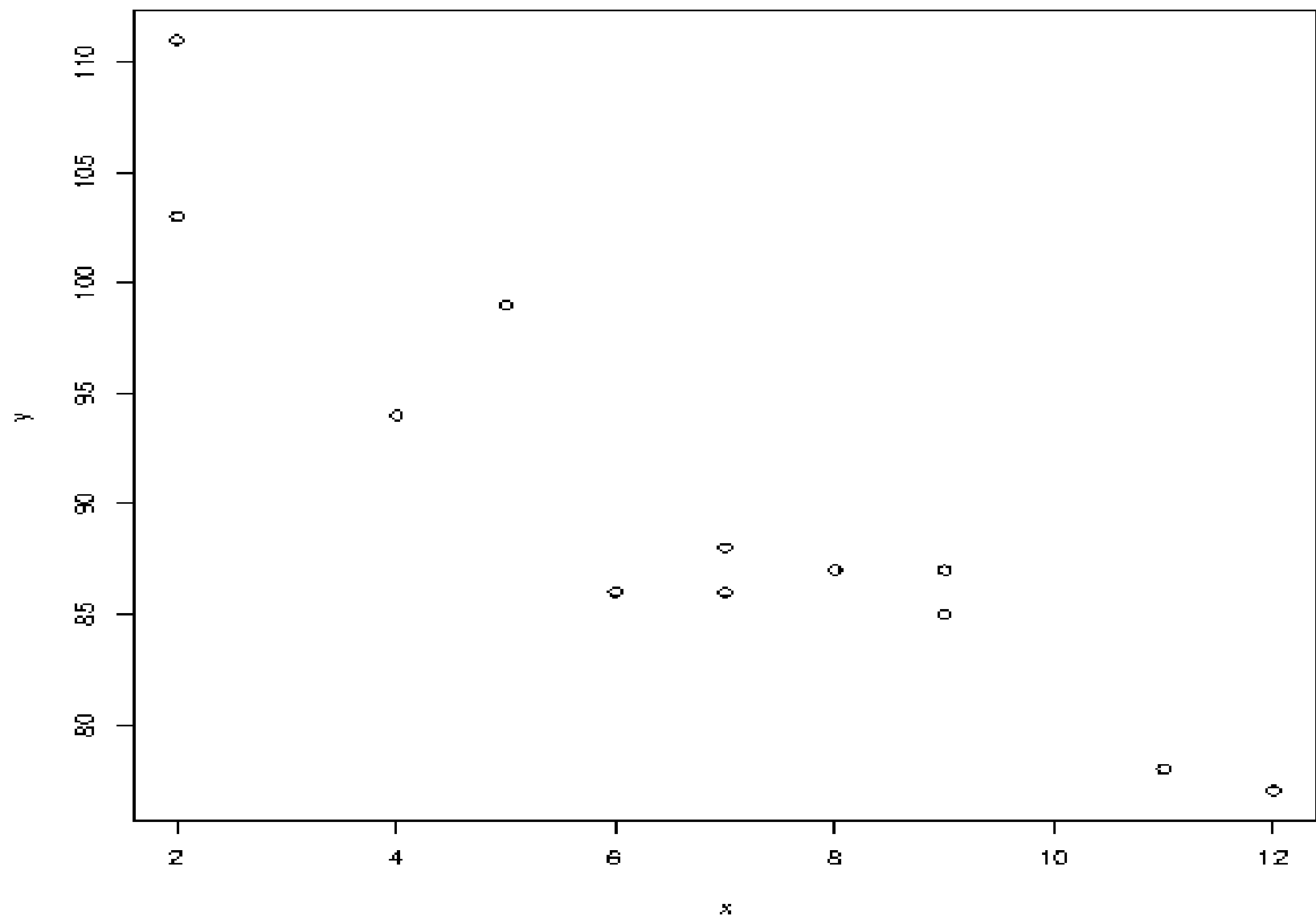
6 displays a "two dashed" line

Scatter Plots

- A "scatter plot" is a type of plot used to display the relationship between two numerical variables, and plots one dot for each observation.
- It needs two vectors of same length, one for the x-axis (horizontal) and one for the y-axis (vertical):
- Example
- ```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
```

```
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
```

```
plot(x, y)
```



Example

```
x <- c(5,7,8,7,2,2,9,4,11,12,9,6)
```

```
y <- c(99,86,87,88,111,103,87,94,78,77,85,86)
```

```
plot(x, y, main="Observation of Cars", xlab="Car age",
ylab="Car speed")
```

# R Pie Chart

A pie chart is a circular graphical view of data.

Use the `pie()` function to draw pie charts:

**Example**

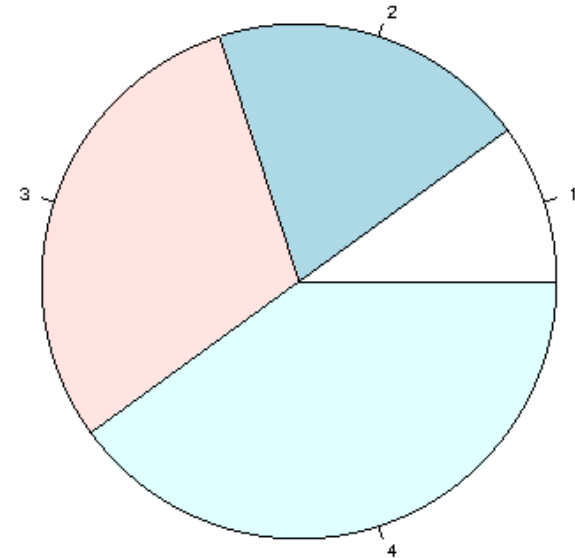
**# Create a vector of pies**

```
x <- c(10,20,30,40)
```

**# Display the pie chart**

```
pie(x)
```

**Result:**





## Labels and Header

Use the label parameter to add a label to the pie chart, and use the main parameter to add a header:

### Example

```
Create a vector of pies
```

```
x <- c(10,20,30,40)
```

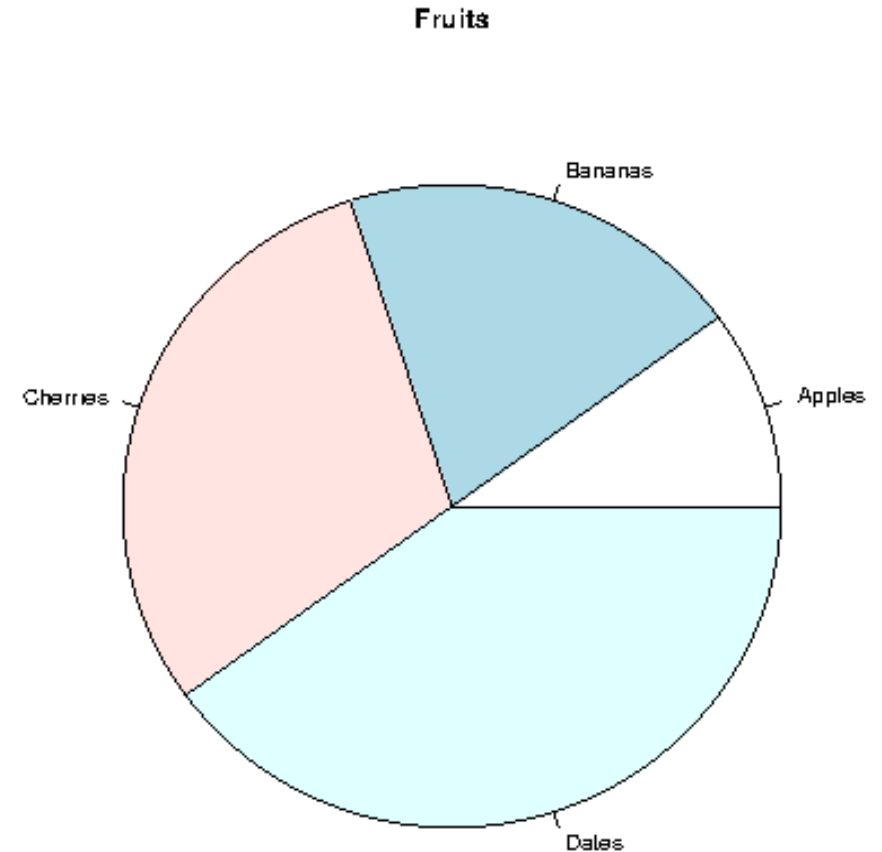
```
Create a vector of labels
```

```
mylabel <- c("Apples", "Bananas",
"Cherries", "Dates")
```

```
Display the pie chart with labels
```

```
pie(x, label = mylabel, main = "Fruits")
```

Result:



## Colors

You can add a color to each pie with the col parameter:

### Example

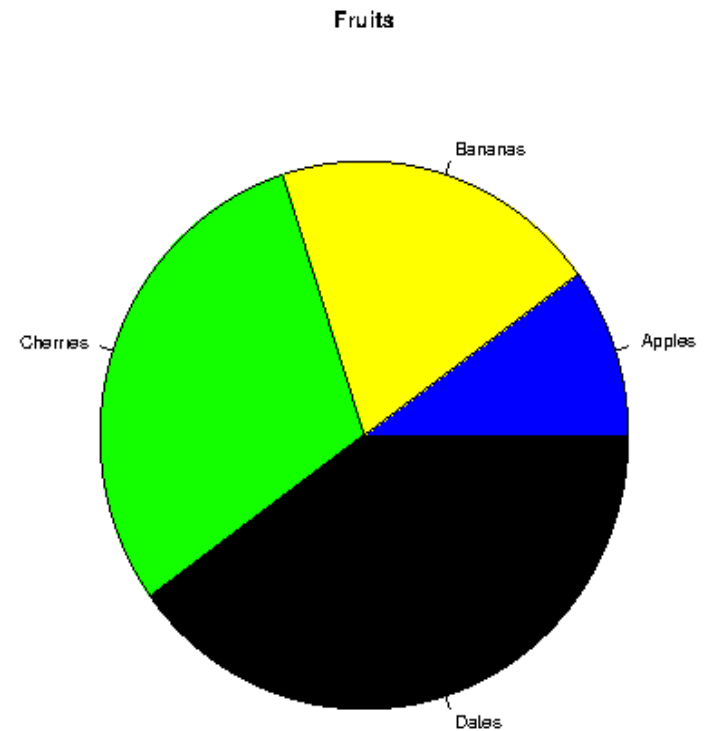
```
Create a vector of colors
```

```
colors <- c("blue", "yellow", "green", "black")
```

```
Display the pie chart with colors
```

```
pie(x, label = mylabel, main = "Fruits", col = colors)
```

Result:



## Legend

To add a list of explanation for each pie, use the `legend()` function:

### Example

```
Create a vector of labels
```

```
mylabel <- c("Apples", "Bananas", "Cherries",
"Dates")
```

```
Create a vector of colors
```

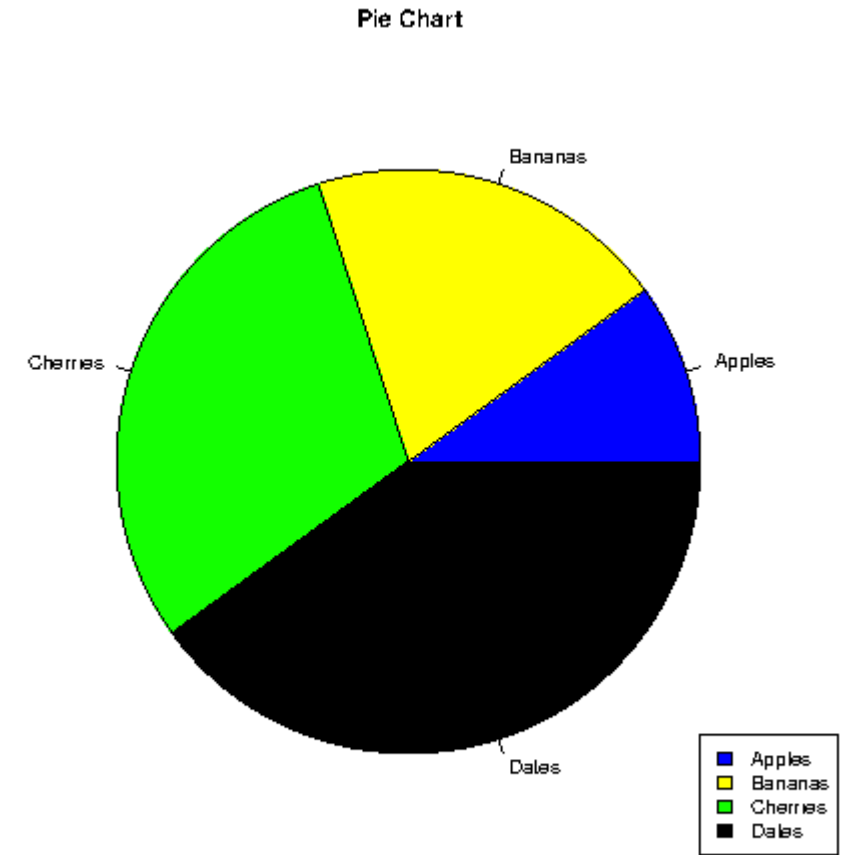
```
colors <- c("blue", "yellow", "green", "black")
```

```
Display the pie chart with colors
```

```
pie(x, label = mylabel, main = "Pie Chart", col =
colors)
```

```
Display the explanation box
```

```
legend("bottomright", mylabel, fill = colors)
```



The legend can be positioned as either:  
bottomright, bottom, bottomleft, left, topleft,  
top, topright, right, center

# R Bar Charts

- A bar chart uses rectangular bars to visualize data. Bar charts can be displayed horizontally or vertically. The height or length of the bars are proportional to the values they represent.
- Use the `barplot()` function to draw a vertical bar chart:  
The `x` variable represents values in the x-axis (A,B,C,D)  
The `y` variable represents values in the y-axis (2,4,6,8)  
Then we use the `barplot()` function to create a bar chart of the values  
`names.arg` defines the names of each observation in the x-axis

## Example

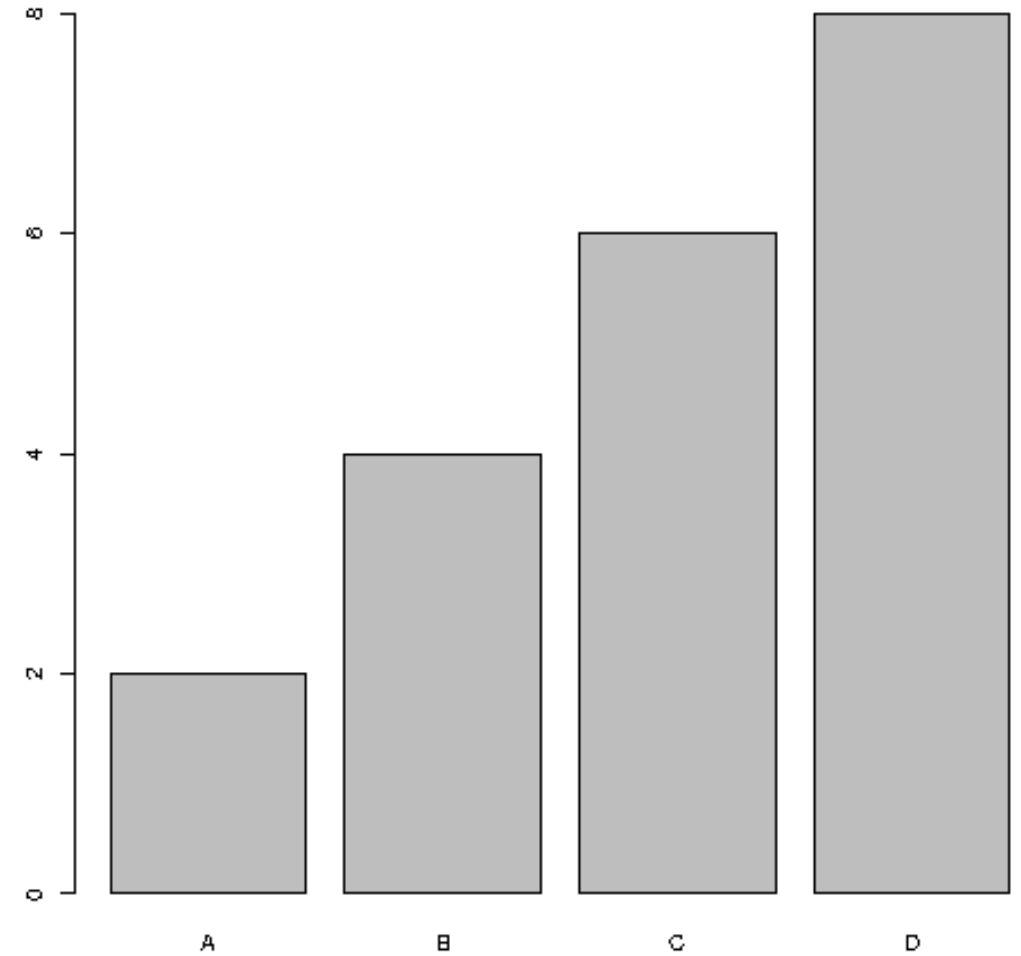
# x-axis values

```
x <- c("A", "B", "C", "D")
```

# y-axis values

```
y <- c(2, 4, 6, 8)
```

```
barplot(y, names.arg = x)
```



# Bar Color

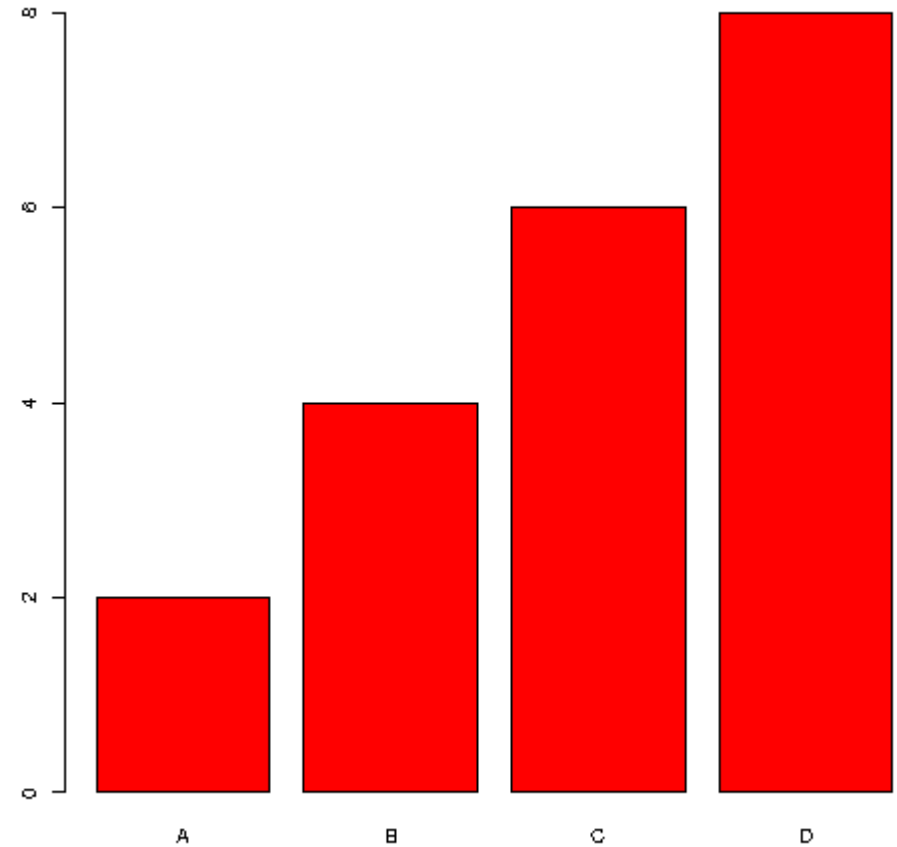
Use the col parameter to change the color of the bars:

## Example

```
x <- c("A", "B", "C", "D")
```

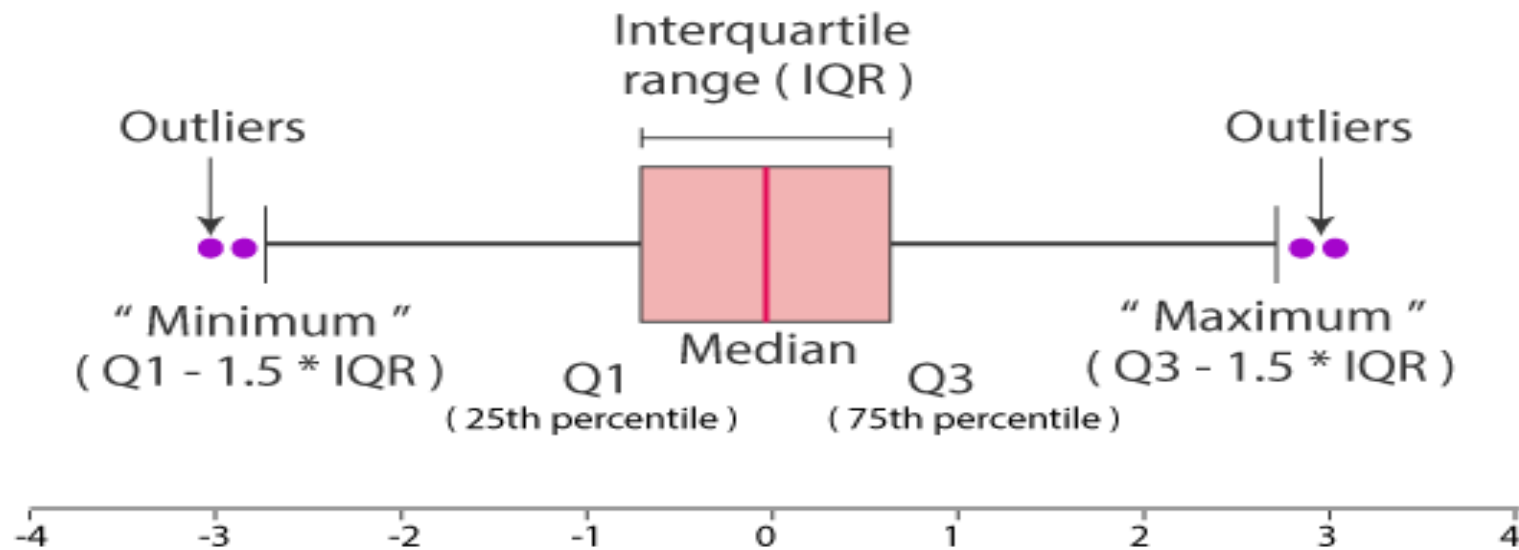
```
y <- c(2, 4, 6, 8)
```

```
barplot(y, names.arg = x, col = "red")
```



# What is Boxplot? Why we use Boxplot?

Box plots, also known as box and whisker plots, are used to visually summarize data sets and compare them. They are useful for showing the distribution of data, including outliers, the median, and the range of values.



**Different parts of boxplot**

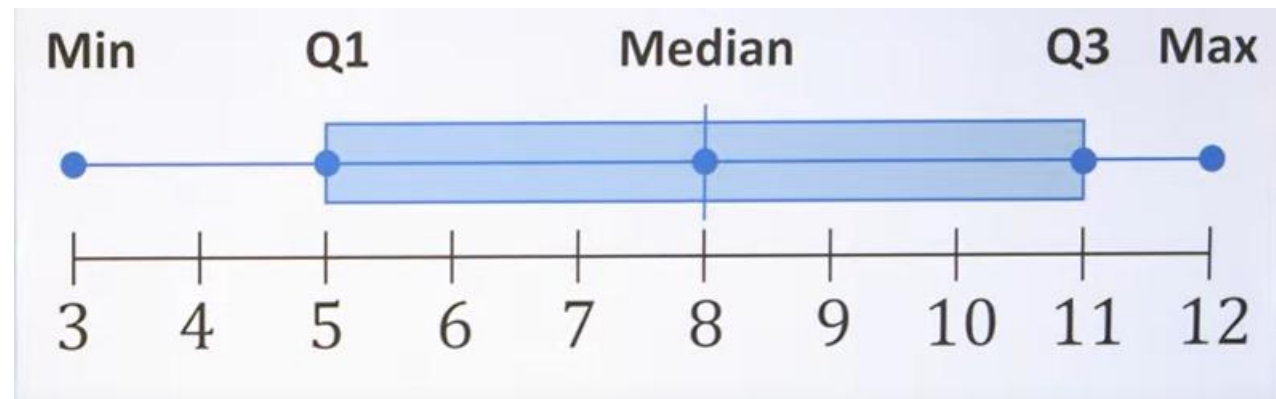
# IQR

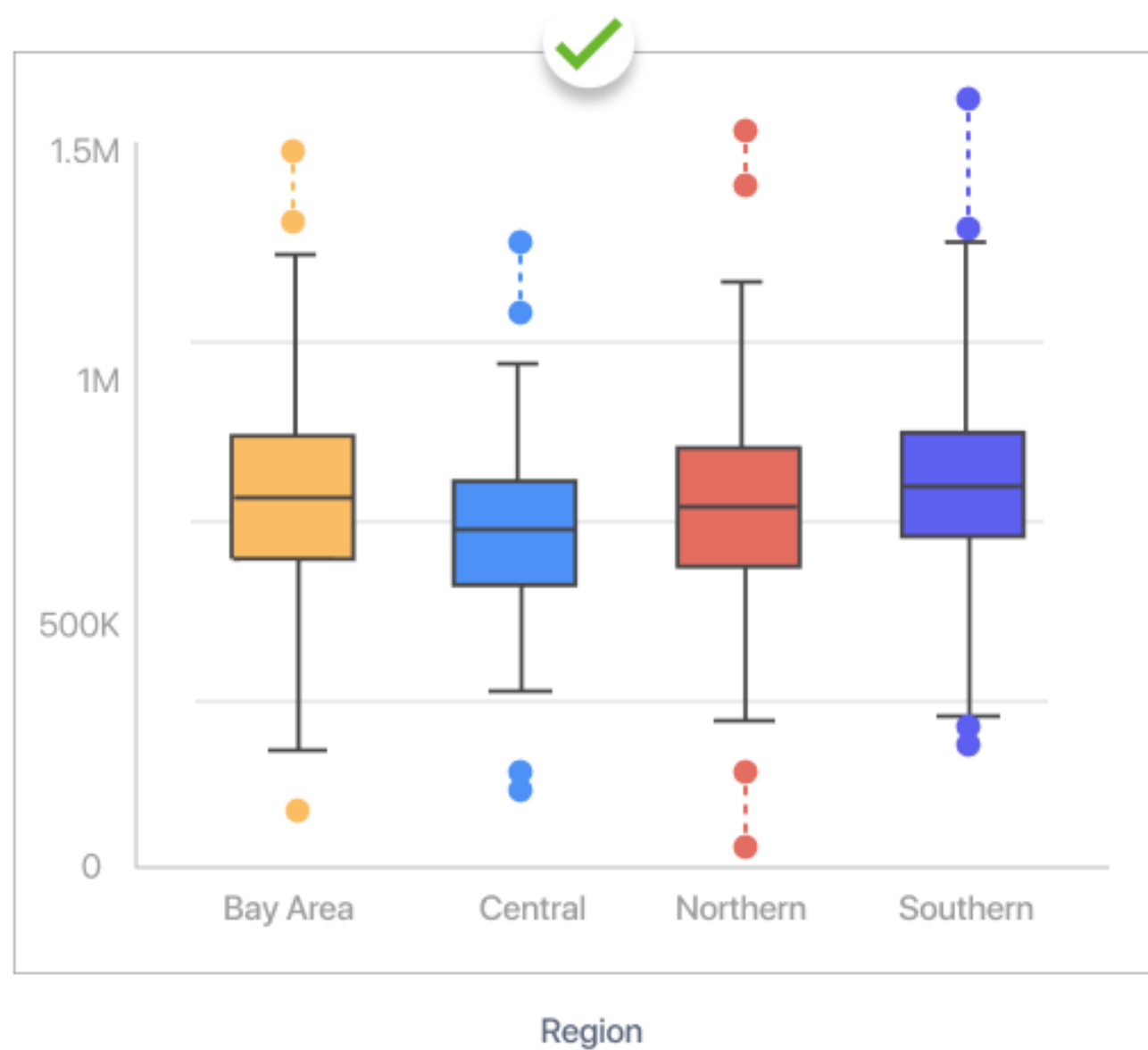
10, 4, 11, 7, 3, 12, 11, 9, 5, 5, 8

3, 4, 5, 5, 7, 8, 9, 10, 11, 11, 12

3, 4, 5, 5, 7, 8, 9, 10, 11, 11, 12

Q1 = 5      Q2 = 8      Q3 = 11







## Boxplots in R Language

A box graph is a chart that is used to display information in the form of distribution by drawing boxplots for each of them. This distribution of data is based on five sets (minimum, first quartile, median, third quartile, and maximum).

**Syntax:** `boxplot(x, data, notch, varwidth, names, main)`

### Parameters:

- x:** This parameter sets as a vector or a formula.
- data:** This parameter sets the data frame.
- notch:** This parameter is the label for horizontal axis.
- varwidth:** This parameter is a logical value. Set as true to draw width of the box proportionate to the sample size.
- main:** This parameter is the title of the chart.
- names:** This parameter are the group labels that will be showed under each boxplot.

```
input <- mtcars[, c('mpg', 'cyl')]
print(head(input))
```

Output:

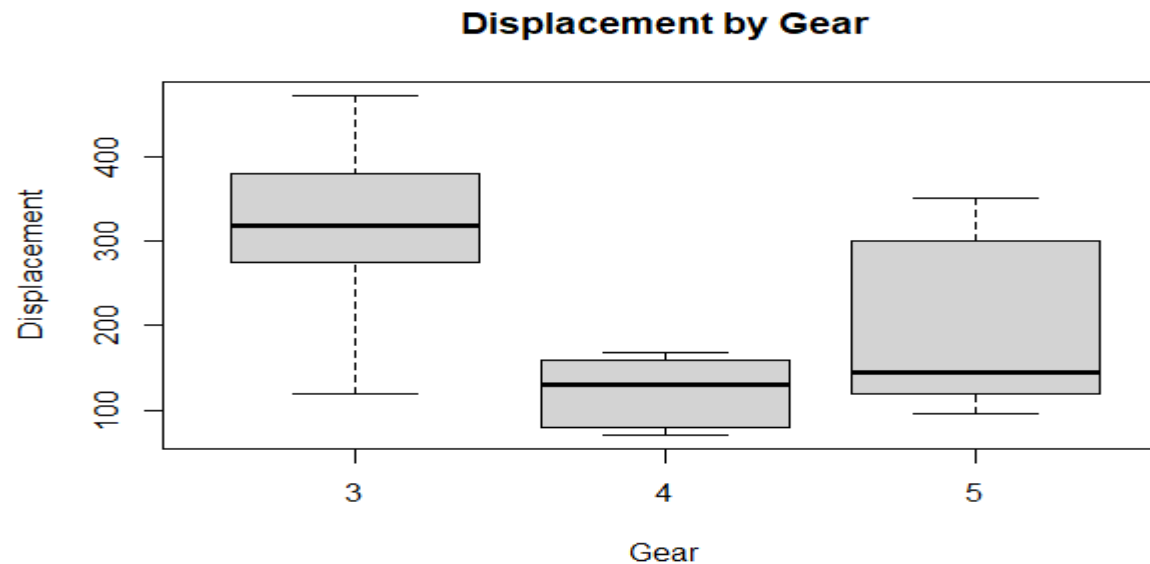
|                   | mpg  | cyl |
|-------------------|------|-----|
| Mazda RX4         | 21.0 | 6   |
| Mazda RX4 Wag     | 21.0 | 6   |
| Datsun 710        | 22.8 | 4   |
| Hornet 4 Drive    | 21.4 | 6   |
| Hornet Sportabout | 18.7 | 8   |
| Valiant           | 18.1 | 6   |

## Creating the Boxplot

- Take the parameters which are required to make a boxplot.
- Now we draw a graph for the relation between “mpg” and “cyl”.

```
Load the dataset
data(mtcars)
```

```
Create the box plot
boxplot(dis ~ gear, data = mtcars,
 main = "Displacement by Gear",
 xlab = "Gear",
 ylab = "Displacement")
```



Create a data frame of student grades with ordered and plot the box plot and interpret which class got high score and low score?

Students Marks: 75, 82, 68, 92, 89, 78, 85, 90, 72, 81, 94, 87, 79, 86, 91

Students Level: freshman(First 5), sophomore (Next 4), junior (Next 3) , senior (Next 3).

```
Create a sample dataset of student grades
```

```
grades <- data.frame(
 grade = c(75, 82, 68, 92, 89, 78, 85, 90, 72, 81, 94, 87, 79, 86, 91),
 level = factor(c(rep("freshman", 5), rep("sophomore", 4), rep("junior", 3),
 rep("senior", 3)))
)
```

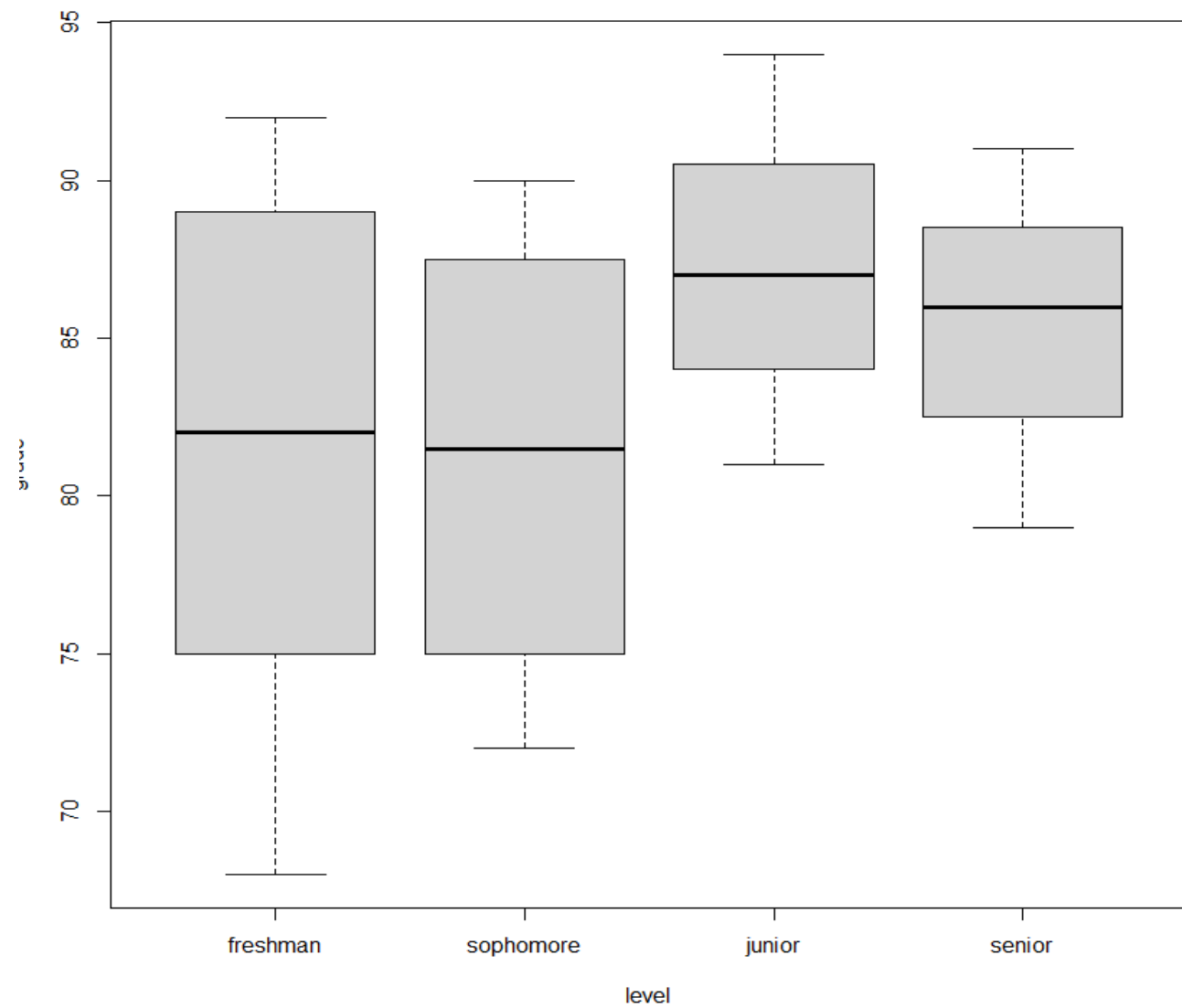
```
Specify level ordering for the "level" factor
```

```
grades$level <- ordered(grades$level, levels = c("freshman", "sophomore",
 "junior", "senior"))
```

```
Create a boxplot of grades by class level
```

```
boxplot(grade ~ level, data = grades, main = "Student Grades by Class
Level")
```

Student Grades by Class Level



# R Programming Advanced Tools

| Integrated Development Environments (IDEs) |                                                                                                                                                                                                                                 |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RStudio                                    | <ol style="list-style-type: none"><li>1. User-Friendly Interface</li><li>2. Integrated Tools</li><li>3. Project Management</li><li>4. Extensions and Plugins</li></ol> <p>Interactive Code</p> <p>Sharing and Collaboration</p> |
| Jupyter Notebooks                          |                                                                                                                                                                                                                                 |

## Data Manipulation Tools

|              |                                                                                                                                                |                       |                                                                                             |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------------------------------------------------------------|
| <b>dplyr</b> | dplyr is a grammar of data manipulation, providing a consistent set of verbs that help you solve the most common data manipulation challenges. | Data Transformation   | Functions like <b>filter()</b> , <b>select()</b> , <b>mutate()</b> , and <b>summarize()</b> |
|              |                                                                                                                                                | Pipeline Operations   | <b>%&gt;%</b>                                                                               |
|              |                                                                                                                                                | Efficient Handling    | Optimized for performance with <b>large datasets</b> .                                      |
| <b>tidyr</b> | tidyr helps to tidy your data, ensuring that it's structured in a way that facilitates analysis.                                               | Reshaping Data        | <b>gather()</b> , <b>spread()</b> , <b>unite()</b> , and <b>separate()</b>                  |
|              |                                                                                                                                                | Handling Missing Data | <b>drop_na()</b> and <b>fill()</b> .                                                        |

| Package Management Tools |                                                                                                 |                       |
|--------------------------|-------------------------------------------------------------------------------------------------|-----------------------|
| CRAN                     | The <i>Comprehensive R Archive Network (CRAN)</i> is the primary repository for R packages.     | Extensive Repository  |
|                          |                                                                                                 | Easy Installation     |
| Bioconductor             | Bioconductor provides tools for the analysis and comprehension of high-throughput genomic data. | Specialized Packages  |
|                          |                                                                                                 | Integration with CRAN |
|                          |                                                                                                 | Handling Missing Data |

| Data Import and Export Tools |                                                                                                                                                    |                   |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| readr                        | <i>readr</i> provides a fast and friendly way to read rectangular data.                                                                            | Efficient Parsing |
|                              |                                                                                                                                                    | read_csv()        |
| data.table                   | <i>data.table</i> is an extension of data.frame, providing a high-performance version of base R's data.frame with syntax and feature enhancements. | Speed             |
|                              |                                                                                                                                                    | Flexibility       |



Interactive Web Applications with R

|                             |                                                                                                                                                                                              |                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Shiny                       | Shiny allows you to turn your R scripts into interactive web applications effortlessly.                                                                                                      | Extensive Repository                                 |
|                             |                                                                                                                                                                                              | Easy Installation                                    |
| Appsilon's styling packages | In Shiny, the default UI components are functional but can sometimes look plain and lack the polish needed for professional applications. This is where Appsilon's styling packages come in. | shiny.semantic<br>shiny.fluent<br>semantic.dashboard |
|                             |                                                                                                                                                                                              | Integration with CRAN                                |
|                             |                                                                                                                                                                                              | Handling Missing Data                                |

## Mtcars Dataset Explorer

**X-axis variable**

disp

Y-axis variable

hp

Scatter plot of disp vs hp



Controls

X-axis variable

disp



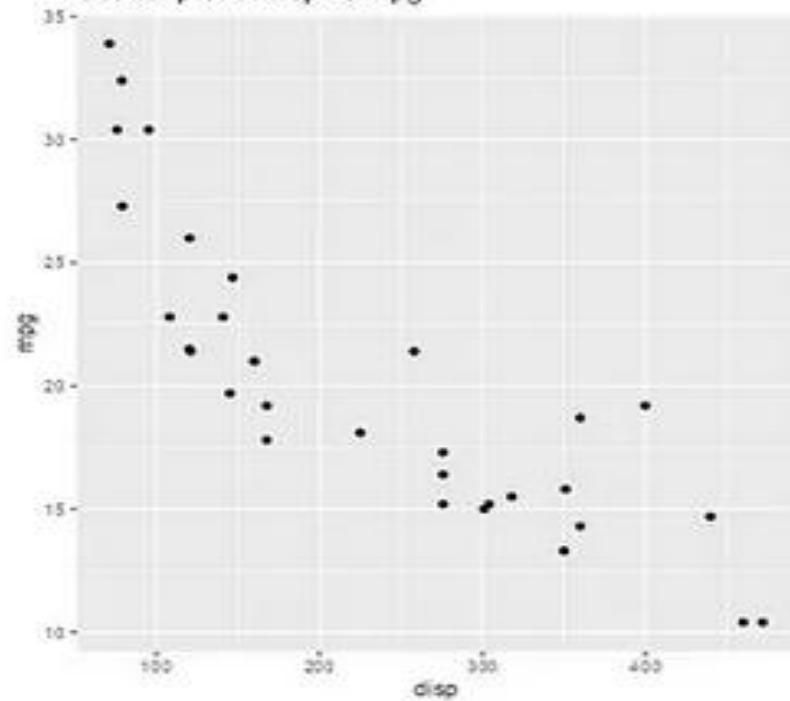
Y-axis variable

mpg



Scatter Plot

Scatter plot of disp vs mpg



# Key areas where R shines in bioinformatics

- **Data Preprocessing and Quality Control**
- **Statistical Analysis**
- **Gene Expression Analysis**
- **Genomic Data Manipulation**
- **Data Visualization**
- **Pathway Enrichment Analysis**
- **Machine Learning and Prediction**

**THANK YOU**

# Bioconductor: "open source software for bioinformatics"

Bioconductor is a comprehensive project that provides tools for the analysis and comprehension of high-throughput genomic data. Built on the R programming language, Bioconductor facilitates the integration and analysis of biological data, offering a robust platform for bioinformatics and computational biology. This article provides a detailed overview of Bioconductor, its features, and how to use it effectively in R Programming Language.

## What is Bioconductor?

Bioconductor is an open-source, open-development software project designed for the analysis and comprehension of genomic data. Launched in 2001, it has grown into a major repository with over 2,000 R packages tailored to various bioinformatics tasks. These packages cover a broad spectrum of applications, including genomic data analysis, annotation, and visualization. It contains thousands of specialized packages for tasks like differential gene expression, pathway analysis, and visualization of biological networks.

### How Did Bioconductor Start?

Bioconductor was launched in 2001 by Robert Gentleman (one of the co-creators of R) and other bioinformatics experts. The goal was to provide a standardized framework for genomic data analysis in R, replacing the scattered custom scripts used at the time.

### Why is Bioconductor Important?

- Handles large-scale high-throughput sequencing data (e.g., RNA-Seq, DNA-Seq).
- Provides statistical models tailored for biological experiments.
- Ensures reproducibility—each package follows strict version control and documentation.
- Supports integration with public genomic databases like Ensembl, NCBI, and KEGG.

## Key Features of Bioconductor

Bioconductor packages are designed to facilitate the integration and analysis of biological data, including gene expression, SNPs, and sequencing data. Here are the main key features of Bioconductor.

Bioconductor packages are specifically designed to handle biological data, including:

- Genomic Data: DNA sequences, gene expression data, and genomic annotations.
- Proteomic Data: Protein expression and structure data.
- Metabolomic Data: Data related to metabolic processes and pathways.
- Epigenomic Data: Information on epigenetic modifications.

## Bioconductor is opensource:

The Bioconductor project has a commitment to full open source discipline, with distribution via a public git (version control) server. Almost all contributions exist under an open source license. There are many different reasons why open source software is beneficial to the analysis of microarray data and to computational biology in general. The reasons include:

- To provide full access to algorithms and their implementation
- To facilitate software improvements through bug fixing and software extension
- To encourage good scientific computing and statistical practice by providing appropriate tools and instruction
- To provide a workbench of tools that allow researchers to explore and expand the methods used to analyze biological data
- To ensure that the international scientific community is the owner of the software tools needed to carry out research
- To lead and encourage commercial support and development of those tools that are successful
- To promote reproducible research by providing open and accessible tools with which to carry out that research (reproducible research is distinct from independent verification)

## What do we measure and why?

The Bioconductor project is an open source repository for R packages, datasets, and workflows that are specific for analyzing biological data. The Bioconductor project is a useful extension on CRAN, the R Archive, because it provides us with the software tools to explore, understand, and solve molecular biology questions. Hence, Bioconductor's tagline is "open source software for bioinformatics".

Molecular biology questions are usually about either the structure or the function of each of the building blocks of an organism, and very often how they interconnect.

1. Structure of biological data: Elements, regions, size, order, relationships
2. Function of biological data: expression, levels, regulation, phenotypes

## How to install Bioconductor packages?

The Bioconductor package collection forms its own repository and has its own release schedule. Because Bioconductor has its own base functions and updates, packages are installed differently.

To install Bioconductor packages we need two lines of code.

1. Install BiocManager: a package management system through which we can install other Bioconductor packages.
2. Call BiocManager: to install any Bioconductor package we desire. Here, we install the popular GenomicRanges package. Updating packages regularly is important to access new features and improvements.

```
Install BiocManager
install.packages("BiocManager")

Install the GenomicRanges package
BiocManager::install("GenomicRanges")
```

```
> BiocManager::valid() # Checks for outdated or missing packages
Error in loadNamespace(name) : there is no package called 'BiocManager'
```

## Installing packages from somewhere else besides CRAN?

Not all R packages are available on CRAN. For bioinformatics-related packages in particular, there is another repository that has many powerful packages that you can install. It is called Bioconductor and it is a repository specifically focused on bioinformatics packages. Bioconductor has a mission of promoting the statistical analysis and comprehension of current and emerging high-throughput biological assays. This means that many if not all of the packages available on Bioconductor are focused on the analysis of biological data, and that it can be a great place to look for tools to help you analyze your -omics datasets.

Since access to the Bioconductor repository is not built in to base R 'out of the box', there are a couple steps needed to install packages from this alternative source. We will work through the steps (only 2!) to install a package to help with the VCF analysis we are working on, but you can use the same approach to install any of the many thousands of available packages.

1. install the BiocManager package
2. install the vcfR package from Bioconductor using BiocManager

## Finding Bioconductor Packages

Bioconductor packages can be searched and installed via the Bioconductor website or using the BiocManager package within R.

```
BiocManager::available()
```

Website for available packages:

[https://bioconductor.org/packages/release/BiocViews.html#\\_Software](https://bioconductor.org/packages/release/BiocViews.html#_Software)

## Popular Bioconductor Packages

Here are some popular Bioconductor packages and their functionalities:

- GenomicRanges: GenomicRanges provides efficient and flexible tools for representing and manipulating genomic intervals and variables defined along a genome.

```
library(GenomicRanges)
```

```
gr <- GRanges(seqnames = "chr1",
```



```

ranges = IRanges(start = c(100, 200),
 end = c(150, 250)),
strand = c("+", "-"))
gr

```

**OUTPUT:**

**GRanges object with 2 ranges and 0 metadata columns:**

|     | seqnames | ranges    | strand |
|-----|----------|-----------|--------|
|     | <Rle>    | <IRanges> | <Rle>  |
| [1] | chr1     | [100,150] | +      |
| [2] | chr1     | [200,250] | -      |

This means:

The first genomic interval spans 100-150 on the positive strand.

The second genomic interval spans 200-250 on the negative strand.

- DESeq2: DESeq2 is used for differential gene expression analysis based on negative binomial distribution.
- edgeR: edgeR is another package for differential expression analysis of RNA-seq and other count data.
- limma: limma provides tools for the analysis of gene expression data, especially from microarray and RNA-seq technologies.
- Biostrings: Biostrings offers efficient manipulation of large biological sequences.

**R programming** – Data structures (vectors, matrices, lists, data frames), functions, and basic visualization.

**Statistics** – Probability distributions, hypothesis testing, regression models.

**Genomics/Bioinformatics concepts** – DNA, RNA, gene expression, sequencing technologies (microarrays, RNA-Seq), and functional annotations.

## S4 system:

In R, particularly within the Bioconductor project, "S3" and "S4" refer to two different object-oriented programming systems, with S3 being a simpler, more flexible approach while S4 is a more rigorous and formal system, heavily used by Bioconductor packages to define complex data structures with strict validation checks; essentially, most Bioconductor packages utilize S4 classes for their data objects due to the need for precise data representation and quality control.

## Key points about S3 and S4:

### S3:

- Easier to learn and implement, with a more relaxed structure.
- Relies on conventions and class attributes to determine how to handle objects.

- Widely used in base R packages and for simpler data analysis tasks.

#### **S4:**

- Provides more control and validation by explicitly defining slots (data components) and their allowed data types within a class.
- Preferred for complex data structures in Bioconductor packages where accurate data representation is crucial.
- Requires more code to define classes and methods due to its stricter structure.

#### **How Bioconductor uses S4:**

##### **Data objects:**

- Most major Bioconductor data classes like
  - "ExpressionSet" and
  - "SummarizedExperiment"
 are built using S4 classes, allowing for complex data structures with defined attributes and validation checks.

##### **Package development:**

- Developers in Bioconductor heavily utilize S4 to create robust and well-defined data analysis pipelines.

| <b>Feature</b>    | <b>S3 (Simpler Object-Oriented System)</b>              | <b>S4 (Formal and More Rigid System)</b>                 |
|-------------------|---------------------------------------------------------|----------------------------------------------------------|
| Definition        | Informal, flexible object-oriented system in R          | Formal, stricter object-oriented system in R             |
| Object Structure  | Uses lists with class attributes                        | Uses a structured class definition                       |
| Class Declaration | Implicit (just adding a class attribute)                | Explicit (setClass())                                    |
| Method Definition | Uses generic functions with UseMethod()                 | Uses setMethod() and setGeneric()                        |
| Encapsulation     | Weak encapsulation (any field can be modified directly) | Strong encapsulation (slots must be defined explicitly)  |
| Field Access      | Direct access using \$                                  | Must use @ to access slots                               |
| Inheritance       | Implicit, less strict                                   | Explicit, supports multiple inheritance                  |
| Flexibility       | Very flexible, but less structured                      | More rigid but ensures data consistency                  |
| Performance       | Generally faster due to its simplicity                  | Slightly slower due to validation overhead               |
| Error Checking    | Minimal error checking                                  | Strong type checking and validation                      |
| Usage             | Used for quick prototyping and less strict applications | Used for complex applications like Bioconductor packages |

|                    |                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Example Definition | <pre>my_s3_object &lt;- list(name = "Example", value = 10)  class(my_s3_object) &lt;- "MyS3Class"</pre> | <pre>MyEpicProject &lt;- setClass(# Define class name with UpperCamelCase "MyEpicProject", # Define slots, helpful for validation slots = c(ini = "Date", end = "Date", milestone = "character"), # Define inheritance contains = "MyProject")  setClass("MyS4Class", slots = list(name = "character", value = "numeric"))  my_s4_object &lt;- new("MyS4Class", name = "Example", value = 10)</pre> |
|                    | <pre>plot() methods(plot)</pre>                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                     |
| Example Method     | <pre>print.MyS3Class &lt;- function(obj) { print(paste("S3 Object:", obj\$name, obj\$value)) }</pre>    | <pre>setMethod("show", "MyS4Class", function(object) { print(paste("S4 Object:", object@name, object@value)) })</pre>                                                                                                                                                                                                                                                                               |

## Exploring Bioconductor Packages

Bioconductor provides packages for various aspects of bioinformatics:

| Category                | Examples of Packages                    |
|-------------------------|-----------------------------------------|
| Sequencing Data         | GenomicRanges, Rsamtools, ShortRead     |
| Differential Expression | DESeq2, edgeR, limma                    |
| Pathway Analysis        | clusterProfiler, ReactomePA, GAGE       |
| Microarray Analysis     | affy, oligo, limma                      |
| Visualization           | ggbio, Gviz, pheatmap                   |
| Variant Analysis        | VariantAnnotation, BSgenome, Biostrings |
| ChIP-Seq Analysis       | ChIPseeker, DiffBind                    |

## Working with Genomic Data

One of the key features of Bioconductor is working with genomic ranges. The GenomicRanges package provides an efficient way to work with genomic intervals.

Example: Using GenomicRanges

```
library(GenomicRanges)
```

```
Define genomic ranges
gr <- GRanges(seqnames = "chr1", ranges = IRanges(start = c(100,
200, 300), end = c(150, 250, 350)), strand = c("+", "-", "+"))
```

```
Print GenomicRanges object
print(gr)
```

## RNA-Seq Data Analysis with DESeq2

DESeq2 is widely used for differential gene expression analysis.

Example: Running DESeq2

```
library(DESeq2)

Sample metadata
colData <- data.frame(
 row.names = c("sample1", "sample2", "sample3", "sample4"),
 condition = c("control", "control", "treatment", "treatment")
)

names condition
Sample1 control [1.2, 2.2, 3.3....]
sample2 control
sample3 treatment
sample4 treatment

Generate example count matrix
countData <- matrix(sample(1:1000, 1000, replace = TRUE), nrow =
500)

Create DESeq2 object
dds <- DESeqDataSetFromMatrix(countData = countData, colData =
colData, design = ~ condition)

Run DESeq2 analysis
dds <- DESeq(dds)

Get results
res <- results(dds)

View top differentially expressed genes
head(res)
```

## What is dplyr?

The package dplyr is a fairly new (2014) package that tries to provide easy tools for the most common data manipulation tasks. This package is also included in the tidyverse package,

which is a collection of eight different packages (dplyr, ggplot2, tibble, tidyr, readr, purrr, stringr, and forcats). It is built to work directly with data frames. The thinking behind it was largely inspired by the package plyr which has been in use for some time but suffered from being slow in some cases. dplyr addresses this by porting much of the computation to C++. An additional feature is the ability to work with data stored directly in an external database. The benefits of doing this are that the data can be managed natively in a relational database, queries can be conducted on that database, and only the results of the query returned.

This addresses a common problem with R in that all operations are conducted in memory and thus the amount of data you can work with is limited by available memory. The database connections essentially remove that limitation in that you can have a database that is over 100s of GB, conduct queries on it directly and pull back just what you need for analysis in R.

## dplyr: Data Manipulation

`dplyr` is a powerful package for filtering, summarizing, and transforming data.

### Key Functions

| Function                 | Description                     |
|--------------------------|---------------------------------|
| <code>filter()</code>    | Select rows based on conditions |
| <code>select()</code>    | Choose specific columns         |
| <code>mutate()</code>    | Add new calculated columns      |
| <code>summarize()</code> | Aggregate and summarize data    |
| <code>arrange()</code>   | Sort data                       |
| <code>group_by()</code>  | Group data for analysis         |

```
library(dplyr)

df <- mtcars %>%
 filter(cyl == 6) %>% # Filter cars with 6 cylinders
 select(mpg, hp, wt) %>% # Select relevant columns
 mutate(hp_per_wt = hp / wt) %>% # Add a new column
 arrange(desc(hp_per_wt)) # Sort in descending order

print(df)
```

## Introduction to ggplot2

ggplot2 is a powerful and flexible visualization package in R, designed for creating publication-quality graphs. It follows the Grammar of Graphics principle, which provides a systematic approach to building plots layer by layer.

## Key Features of ggplot2

- Layered Approach: You can add elements like points, lines, and labels iteratively.
- Aesthetic Mappings (aes()): Maps variables to visual properties (color, shape, size).
- Faceting: Allows dividing data into multiple small plots.
- Customization: Themes, color palettes, axis labels, and legends.

## Installing tidyverse

ggplot2 is a part of the tidyverse package, which includes several data manipulation and visualization tools.

```
Installs ggplot2, dplyr, tidyr, etc.
install.packages("tidyverse")

Loads all packages in the tidyverse
library(tidyverse)
library(ggplot2)
```

## Loading a Dataset

```
data(mpg) # Inbuilt dataset

head(mpg) # View first few rows
```

## Creating Basic ggplot2 Visualizations

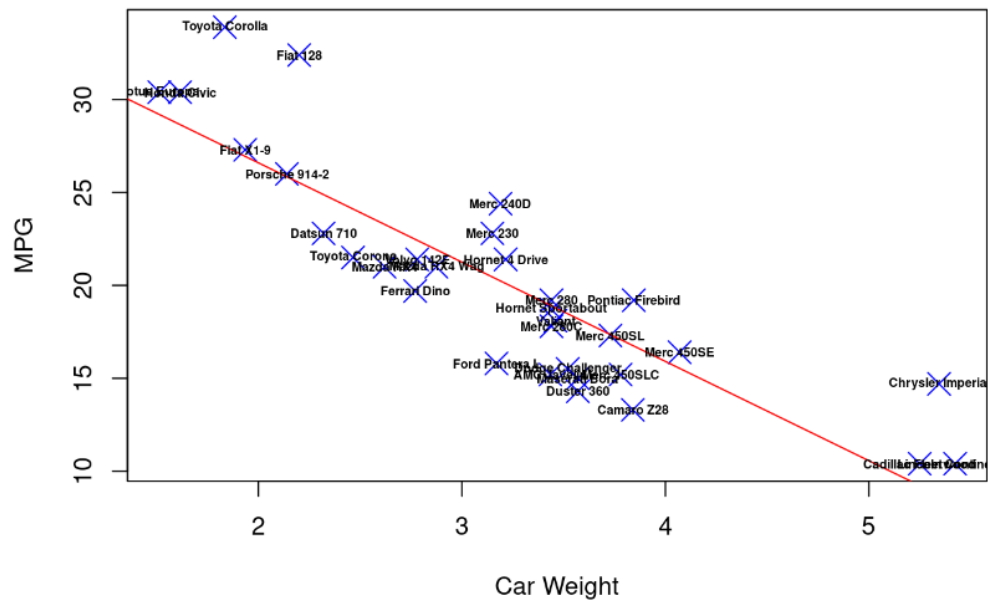
### Scatter Plot

```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) + geom_point()
```

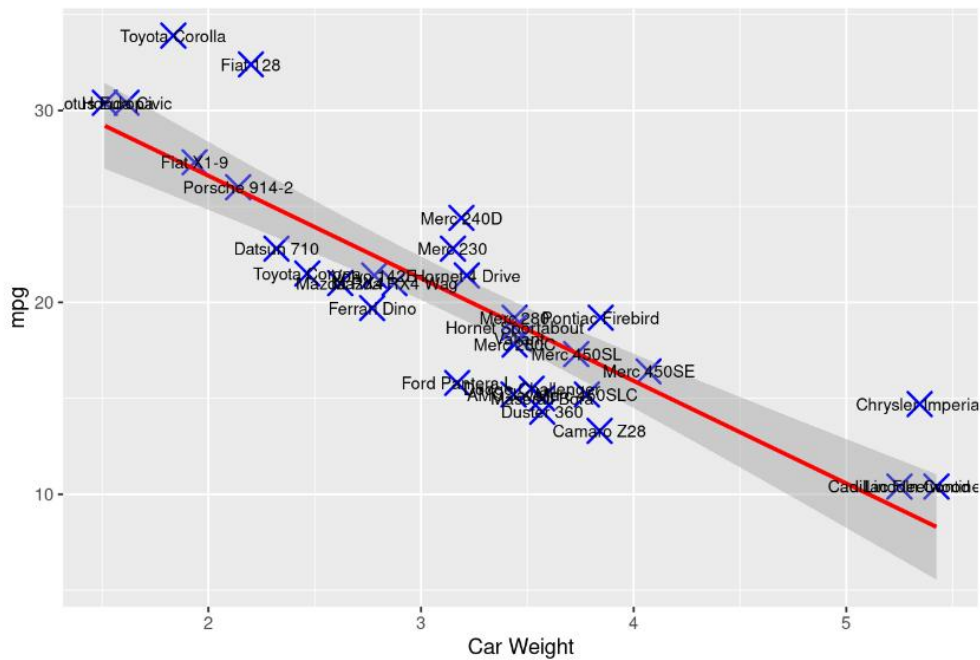
ggplot2 Vs Base R visualization:

<https://jtr13.github.io/cc21fall2/base-r-vs.-ggplot2-visualization.html#:~:text=From%20the%20comparison%20of%20two,different%20between%20ggplot2%20and%20R.>

## Scatterplot in Base R



## Scatterplot in ggplot2



## Bar Plot

```
ggplot(mpg, aes(x = class)) + geom_bar()
```

## Density Plot

```
ggplot(mpg, aes(x = hwy, fill = class)) + geom_density(alpha = 0.5)
```

## Advanced Plot Customization

- **Faceting (Splitting plots into sub-plots)**

```
ggplot(mpg, aes(x = displ, y = hwy)) +
 geom_point() +
 facet_wrap(~class)
```

- **Themes in ggplot2**

```
ggplot(mpg, aes(x = displ, y = hwy)) +
 geom_point() +
 theme_minimal()
```

## Using Shiny and dplyr

**Shiny** is used for interactive web apps, while **dplyr** is for data manipulation.

### Applications of Shiny

- Interactive Genomic Data Analysis.
- Visualization of RNA-Seq results.
- Web-based tools for protein structure prediction.

### Installing and Loading Shiny

```
install.packages("shiny")

library(shiny)
```

### Basic dplyr Operations

```
library(dplyr)

Filtering Data
```



```

filtered_data <- mpg %>% filter(class == "suv")

Selecting Columns

selected_data <- mpg %>% select(manufacturer, model, hwy)

Summarizing Data

summary_data <- mpg %>% group_by(class) %>% summarize(avg_hwy =
mean(hwy))

print(filtered_data)

print(selected_data)

print(summary_data)

```

### Filtering Genes with High Expression

```

library(dplyr)

gene_data <- data.frame(Gene = c("A", "B", "C"), Expression = c(5.2,
10.1, 3.6))

high_expr <- gene_data %>% filter(Expression > 5)

print(high_expr)

```

# SHINY

## What is Shiny?

Shiny is an R package that allows you to create interactive web applications directly from R, without needing web development experience (HTML, CSS, JavaScript).

Think of Shiny as Excel on steroids: Instead of static plots and tables, you can build dynamic dashboards where users can adjust sliders, enter text, and interact with data in real time.

## Installing & Setting Up Shiny

1. First, install Shiny if you haven't already: `install.packages("shiny")`
2. To load the package: `library(shiny)`
3. Start with a **built-in demo app** to see how Shiny works:  
`runExample("01_hello")`

This will open a simple Shiny app in your browser.

# Shiny's Core Structure

Every Shiny app has two main parts:

- UI (User Interface): Controls how the app looks
- Server (Backend Logic): Handles computations & interactivity

These are combined inside a `shinyApp()` function.

## Real-World Applications & Industries

Shiny is widely used in data science, research, business analytics, and interactive reporting. It allows users to build interactive dashboards and web applications without needing extensive web development skills. Below are some of the key use cases across different domains.

### 1. Data Visualization & Dashboards

- Use Case: Interactive Data Dashboards
  - Shiny helps convert static plots into dynamic, interactive dashboards that allow users to:
    1. Filter, zoom, and select subsets of data.
    2. Adjust visualization parameters using sliders or dropdown menus.
    3. Compare datasets in real-time.
  - Example: Interactive Sales Dashboard
  - Application: A company can use Shiny to monitor sales trends, filter by region, and display product performance.

### 2. Bioinformatics & Healthcare

- Use Case: Genomic Data Analysis
  - Shiny is used in bioinformatics for visualizing gene expression, DNA sequencing data, and patient statistics.
  - Example: Researchers use Shiny apps to visualize gene expression patterns in cancer patients.
- Use Case: Patient Monitoring & Medical Dashboards
  - Hospitals and healthcare providers use Shiny to create interactive patient monitoring dashboards to track:
    - ICU patient statistics (heart rate, blood pressure, oxygen levels).
    - Disease Surveillance: COVID-19 or flu tracking apps.
- Example: A Shiny app that takes patient data (e.g., blood pressure, glucose levels) and generates risk scores for diabetes.

### 3. Financial Analytics

- Shiny is widely used in finance for analyzing stocks, risk assessment, and credit scoring.
  - Stock Market Analysis: Track stock prices, calculate moving averages, and simulate investment returns.
  - Risk Assessment: Build financial models that predict loan default risk.
  - Portfolio Optimization: Adjust investment allocations dynamically.

- Example: A Shiny app that allows investors to track stock price trends and calculate expected returns.
- 4. Machine Learning & AI
  - Shiny is frequently used to create interactive machine learning applications:
    - Real-time Model Training: Users can upload datasets and train ML models interactively.
    - Hyperparameter Tuning Dashboards: Adjust ML parameters like learning rate or tree depth dynamically.
    - Real-time Predictions: Upload customer data to predict churn rates or fraud likelihood.
  - Example: A churn prediction model where users input customer data, and the app predicts whether they are likely to leave.
- 5. Government & Public Policy
  - Shiny is used for census analysis, crime tracking, and demographic research:
    - Census & Demographic Data Analysis: Interactive maps showing population trends.
    - Crime Rate Monitoring: Track crime patterns across cities using visual heatmaps.
    - Policy Impact Analysis: Simulate the effect of new government policies.
  - Example: A crime-tracking dashboard that shows real-time data for different regions.
- 6. Education & E-Learning
  - Shiny is useful for interactive learning tools in various subjects:
    - Statistics & Data Science: Simulate probability distributions and hypothesis testing.
    - Physics & Engineering: Create simulations of projectile motion and circuit designs.
    - Economics & Finance: Interactive case studies on market behavior.
  - Example: A normal distribution simulator where students adjust the mean and standard deviation.
- 7. Marketing & Business Intelligence
  - Shiny allows businesses to analyze customer behavior and sales trends dynamically:
    - Website Analytics: Monitor page visits, session durations, and user engagement.
    - A/B Testing Analysis: Compare conversion rates for different marketing strategies.
    - Customer Retention Models: Predict churn and identify key drivers.
  - Example: A customer segmentation tool where businesses can filter customers based on purchase history and demographics.
- 8. Logistics & Supply Chain
  - Companies use Shiny to optimize supply chains and manage inventory.
    - Delivery Route Optimization: Simulate and visualize the best routes for fleet management.
    - Warehouse Inventory Monitoring: Track product availability in real-time.

- Example: A dashboard showing delivery efficiency based on real-time GPS tracking data.
- 9. Cybersecurity & Fraud Detection
  - Shiny is used in cybersecurity for real-time threat monitoring and anomaly detection:
    - Detect Suspicious Login Attempts: Monitor failed logins and access patterns.
    - Fraud Detection in Banking: Build models to flag unusual transactions.
    - Network Traffic Analysis: Visualize incoming and outgoing traffic anomalies.
  - Example: A Shiny app that alerts security teams when multiple failed logins occur from the same IP address.

## Why Use Shiny?

- No web development required - Anyone familiar with R can build interactive applications.
- Real-time interactivity - Data updates dynamically based on user input.
- Seamless integration - Works with R's ecosystem (ggplot2, dplyr, ML packages).
- Deployable on the web - Apps can be shared via ShinyApps.io, RStudio Connect, or Docker.

## Applications of R Programming in Bioinformatics

R is widely used in bioinformatics for analyzing and visualizing biological data.

### Applications:

#### Genomic Data Analysis

- Biostrings for DNA/RNA sequence analysis.
- vcfR for variant calling data.
- GenomicRanges for handling genomic coordinates.

### Compute GC Content (Bioconductor)

```
library(Biostrings)

Define a DNA sequence
dna_seq <- DNASTring("ATGCGCGATCGTAGCTAGCGTATGCGCGATCGTAGCTAGCGT")

Calculate GC content
gc_content <- letterFrequency(dna_seq, letters = c("G", "C"),
as.prob = TRUE)
gc_content "G" = 0.22
```

```
gc_content "C" = 0.28
```

```
gc_percentage <- sum(gc_content) * 100
```

```
0.50 * 100 = 50%
```

```
print(gc_percentage) # Output in percentage
```

```
50
```

### Variant Calling and Annotation

- vcfR for handling VCF files (genetic variants).
- VariantAnnotation for annotating mutations.

```
library(vcfR)
```

```
vcf_data <- read.vcfR("variants.vcf")
```

```
print(vcf_data)
```

### Data Visualization in Bioinformatics

- ggplot2 for plotting gene expression.
- shiny for interactive dashboards.

### Difference Between Bioconductor and CRAN

- **CRAN (Comprehensive R Archive Network):** A general repository for R packages, covering a wide range of statistical and computational applications.
- **Bioconductor:** A specialized repository for bioinformatics tools, particularly for high-throughput genomic data analysis.

| Feature      | CRAN                                     | Bioconductor                                 |
|--------------|------------------------------------------|----------------------------------------------|
| Focus        | General R applications                   | Bioinformatics and computational biology     |
| Packages     | General-purpose R packages               | Specialized bioinformatics packages          |
| Installation | <code>install.packages("package")</code> | <code>BiocManager::install("package")</code> |
| Update Cycle | Frequent, user-contributed               | Regular updates with strict quality control  |

### Applications of R Programming in Bioinformatics

- Genomic Data Analysis: Analyzing DNA, RNA, and protein sequences.

- Statistical Analysis: Gene expression data normalization, clustering, and classification.
- Machine Learning in Drug Discovery: QSAR modeling.
- Visualization: ggplot2 for gene expression heatmaps.
- Network Analysis: Protein-protein interaction networks using igraph.

### Applications of Bioconductor

- **Sequence Alignment:** Handling FASTA, FASTQ files.
- **Gene Expression Analysis:** Normalization, clustering.
- **Variant Analysis:** SNP annotation.
- **Epigenomics:** DNA methylation analysis.

### Role of ggplot2 in Bioinformatics

ggplot2 is widely used for visualizing biological data, including:

- **Expression Data:** Boxplots for differential expression.
- **Genomic Distribution:** Histogram for GC content.
- **Cluster Analysis:** Heatmaps of gene expression.

### Example: Visualizing GC Content Distribution

```
library(ggplot2)

Simulated GC content data
data <- data.frame(Sample = 1:100, GC_Content = runif(100, 35, 65))

Plot
ggplot(data, aes(x = Sample, y = GC_Content)) +
 geom_point(color = "blue") +
 labs(title = "GC Content Distribution", x = "Sample", y = "GC
Content (%)")
```

### Key Advantages and Limitations of Bioconductor

- **Advantages:**
  - High-quality bioinformatics tools.
  - Strong support for genomic and transcriptomic data.
  - Regular updates and active community.
- **Limitations:**
  - Requires **BiocManager** for installation.
  - Some packages may have dependencies that need manual resolution.

## Use of vcfR in Bioinformatics

**vcfR** is used for handling VCF (Variant Call Format) files in R. A VCF (Variant Call Format) file is a widely used file format in bioinformatics for storing genetic variants. It contains information about SNPs (Single Nucleotide Polymorphisms), insertions, deletions, and structural variations in a genome.

- **Parsing VCF Files:** Extract SNP and INDEL variants.
- **Filtering Variants:** Removing low-quality mutations.
- **Visualization:** Plotting genomic variant distributions.

### VCF Download:

- 1000 Genomes Project
- NCBI dbSNP
- Ensembl (Genome Variation Data)

### Structure of a VCF File

A VCF file consists of three sections:

- Header Section → Metadata about the file.
- Column Header Line → Describes the data fields.
- Variant Data Section → The actual genetic variant data.

```
##fileformat=VCFv4.2
```

```
##source=MockVCFGenerator
```

```
##reference=GRCh38
```

```
##contig=<ID=chr1,length=248956422>
```

```
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
```

| #CHROM  | POS     | ID      | REF | ALT | QUAL | FILTER | INFO | FORMAT |
|---------|---------|---------|-----|-----|------|--------|------|--------|
| Sample1 | Sample2 |         |     |     |      |        |      |        |
| chr1    | 10176   | rs12345 | A   | G   | 50   | PASS   | .    | GT 0/1 |
| chr1    | 10583   | rs67890 | C   | T   | 99   | PASS   | .    | GT 0/0 |
| chr1    | 10611   | .       | G   | A   | 60   | PASS   | .    | GT 1/1 |

chr1      13110      rs54321 T            C            70            PASS      .            GT            0/1  
0/0

| Column           | Description                                                            |
|------------------|------------------------------------------------------------------------|
| CHROM            | Chromosome where variant occurs (chr1).                                |
| POS              | Position of the variant in base pairs (10176).                         |
| ID               | SNP ID from dbSNP (rs12345).                                           |
| REF              | Reference allele (A).                                                  |
| ALT              | Alternate allele (G).                                                  |
| QUAL             | Quality score of variant call (50).                                    |
| FILTER           | Whether the variant passed quality control (PASS).                     |
| INFO             | Additional metadata (empty here).                                      |
| FORMAT           | Specifies genotype data format (GT = Genotype).                        |
| Sample1, Sample2 | Genotype of each sample (0/1 = heterozygous, 1/1 = homozygous mutant). |

| Applications of VCF Files:     |                                                             |
|--------------------------------|-------------------------------------------------------------|
| Application                    | Description                                                 |
| Genetic Variant Identification | Identifies SNPs, insertions, and deletions in genomes.      |
| Disease Association Studies    | Links mutations to diseases (e.g., cancer, rare disorders). |
| Pharmacogenomics               | Identifies genetic variants affecting drug response.        |
| Population Genetics            | Studies genetic variations across populations.              |
| Evolutionary Biology           | Analyzes genetic differences among species.                 |

Example Code to Read a VCF File

```
library(vcfR)

vcf <- read.vcfR("variants.vcf")
```



```
head(vcf)
```

## Expected Output

```
***** Object of Class vcfR *****
Number of samples: 2
Number of variants: 4
CHROM POS REF ALT
chr1 10176 A G
chr1 10583 C T
chr1 10611 G A
chr1 13110 T C
```

## Extract Meta Information (Header)

The **header section** contains metadata about the VCF file.

```
Extract header (metadata)
vcf@meta
```

## Expected Output

```
[1] "##fileformat=VCFv4.2"
[2] "##source=GATK"
[3] "##reference=GRCh38"
[4] "##contig=<ID=chr1,length=248956422>"
```

## Extract Fixed Variant Data

The **fixed section** contains **chromosome, position, reference, and alternate alleles**.

```
Convert fixed fields to a data frame
fixed_data <- as.data.frame(vcf@fix)

Display first few rows
head(fixed_data)
```

## Expected Output

```
CHROM POS ID REF ALT QUAL FILTER INFO
```

|      |       |         |   |   |    |      |   |
|------|-------|---------|---|---|----|------|---|
| chr1 | 10176 | rs12345 | A | G | 50 | PASS | . |
| chr1 | 10583 | rs67890 | C | T | 99 | PASS | . |
| chr1 | 10611 | .       | G | A | 60 | PASS | . |
| chr1 | 13110 | rs54321 | T | C | 70 | PASS | . |

## Extract Genotype Information

The **genotype (GT)** column tells us whether a sample has the variant.

```
Extract genotype (GT) information
genotype_data <- extract.gt(vcf, element = "GT")

Display first few rows
head(genotype_data)
```

### Expected Output

| Sample1 | Sample2 |
|---------|---------|
|---------|---------|

|     |     |
|-----|-----|
| 0/1 | 1/1 |
|-----|-----|

|     |     |
|-----|-----|
| 0/0 | 0/1 |
|-----|-----|

|     |     |
|-----|-----|
| 1/1 | 0/0 |
|-----|-----|

|     |     |
|-----|-----|
| 0/1 | 0/0 |
|-----|-----|

### Explanation:

- 0/0 → Homozygous reference (No mutation)
- 0/1 → Heterozygous (One reference, one mutant allele)
- 1/1 → Homozygous alternate (Mutation in both alleles)

## Plot Distribution of Variants Across Chromosome

```
library(ggplot2)
```

```
ggplot(vcf_df, aes(x = Position)) +
 geom_histogram(bins = 30, fill = "blue", color = "black") +
 labs(title = "Distribution of Variants Across Chromosome",
 x = "Genomic Position",
 y = "Variant Count") +
 theme_minimal()
```

### Count Nucleotide Frequencies in a DNA Sequence

```
library(Biostrings)

Load DNA sequence
dna_seq <- DNAString("AGCTTAGGCT")

Count nucleotides
nuc_freq <- alphabetFrequency(dna_seq, baseOnly = TRUE)
print(nuc_freq)
```

### Reverse Complement of a DNA Sequence (Bioconductor)

```
library(Biostrings)
dna_seq <- DNAString("AGCTTAGGCT")
Complement = TCGAATCCGA
Reverse_complement = AGCCTAAGCT
rev_comp <- reverseComplement(dna_seq)
AGCCTAAGCT

cat("Reverse complement:", rev_comp, "\n")
> Reverse complement: AGCCTAAGCT
```

### Display Sequence Length & Translate into Protein

```
library(Biostrings)
dna_seq <- DNAString("ATGCGTACGTAG")
protein_seq <- translate(dna_seq)
print(protein_seq)
> MLYK
```

### Find Overlapping Genes Between Two Genomic Regions

```
library(GenomicRanges)
```

```
gr1 <- GRanges(seqnames = "chr1", ranges = IRanges(start = c(100,
500), end = c(200, 600)))
gr2 <- GRanges(seqnames = "chr1", ranges = IRanges(start = c(150,
550), end = c(250, 650)))
```

```
100 , 200, 300, 400, 500, 600, 700,800
```

```
overlap <- findOverlaps(gr1, gr2)
print(overlap)
```

```
150, 600
```

### Read a DNA Sequence from a FASTA File and Display its Length

```
library(Biostrings)

Load FASTA file
dna_seq <- readDNAStringSet("sequence.fasta")

length <- width(dna_seq)

Display the length of the sequence
cat("Length of the DNA sequence:", length, "\n")
```

```
> "Length of the DNA sequence:" 100
```

### Practice Questions:

1. Write a script to find the reverse complement of a given DNA sequence using the Bioconductor package.
2. Demonstrate how to install packages from Bioconductor.
3. Write a script to display the sequence length from a FASTA file.
4. How does ggplot2 integrate with Bioconductor for visualizing bioinformatics data?
5. dplyr package with an example.
6. Write a script to compute the GC content of a DNA sequence using the Bioconductor package.
7. Write a script to find the reverse complement of a given DNA sequence using the Bioconductor package.
8. Write a script to display the length of the given DNA sequence and translate it into a protein sequence.
9. Write an R script to count the frequency of each nucleotide in a DNA sequence using the Biostrings package

10. Write a script to read a DNA sequence from a FASTA file and display its length
11. Explain the role of ggplot2. Give necessary examples.
12. Write a script to count the frequency of each nucleotide in a DNA sequence.